

WEBSITE LUYỆN TẬP TỪ VỰNG THI TOEIC (VOCABOOST)

THIẾT KẾ CHI TIẾT BACKEND

Phiên bản hiệu chỉnh từ template *DetailDesignTemplateV1.0*

Thông tin tài liệu

Thuộc tính	Mô tả
Mã dự án	VOCAB-PRACTICE
Tên dự án	TOEIC Vocabulary Learning Platform / VocaBoost
Phạm vi tài liệu	Thiết kế chi tiết backend, API, middleware, service và luồng xử lý nghiệp vụ
Phiên bản	V3.0 - Backend Detailed Design + Full Pseudocode
Ngày	23/06/2026

Vai trò	Người phụ trách	Ngày
Người lập	Codex / nhóm phát triển	23/06/2026
Người kiểm tra	Nhóm dự án	
Người phê duyệt	Chủ nhiệm đề tài	

MỤC LỤC

1. Giới thiệu
2. Tổng quan hệ thống
3. Kiến trúc hệ thống backend
4. Thiết kế lớp và module backend
5. Giả mã chi tiết từng hàm backend
6. Thuật toán nghiệp vụ trọng yếu
7. Thiết kế dữ liệu backend
8. Liên kết frontend - backend
9. Kết luận và khuyến nghị mở rộng

1. GIỚI THIỆU

1.1. Mục đích

Tài liệu này mô tả chi tiết phần backend của hệ thống VocaBoost - nền tảng học từ vựng TOEIC có áp dụng Spaced Repetition System (SRS). Mục tiêu của tài liệu là giúp nhóm phát triển, kiểm thử và giảng viên/nhà hướng dẫn hiểu rõ cấu trúc mã nguồn, trách nhiệm của từng module, các hàm quan trọng, các endpoint API và các luồng xử lý nghiệp vụ cốt lõi.

1.2. Phạm vi

Phạm vi tài liệu tập trung vào backend Node.js/Express trong thư mục backend/src, bao gồm: cấu hình hệ thống, middleware, route, controller, service, các thuật toán SRS, quản lý nội dung, gamification, lộ trình học và các API hỗ trợ AI. Phần frontend chỉ được nhắc đến ở mức liên kết API tiêu biểu.

1.3. Vai trò và trách nhiệm

- Nhà phát triển backend: tra cứu trách nhiệm từng file, từng service và luồng nghiệp vụ để bảo trì/mở rộng hệ thống.
- Nhóm kiểm thử: dựa trên route/controller/service để xây dựng test case cho API và kiểm thử tích hợp.
- Giảng viên hoặc người phản biện đồ án: theo dõi đầy đủ cách hệ thống hiện thực yêu cầu phân quyền, quản trị nội dung và SRS.
- Nhóm frontend: đối chiếu các API được backend công bố và chuẩn dữ liệu trả về.

1.4. Tài liệu tham khảo

Bảng tài liệu tham khảo

Thuộc tính	Mô tả
README.md	Mô tả tổng quan dự án, stack công nghệ, chức năng lõi và hướng dẫn chạy.
Database/prototype_database.sql	Script tạo CSDL nền ban đầu của dự án.
Database/TOEIC_Schema_dbdiagram.dbml	Sơ đồ logic và các bảng chính dùng cho backend.
Database/giaithichbang.md	Giải thích nghiệp vụ và tác dụng của các bảng dữ liệu.

1.5. Từ và thuật ngữ

Thuật ngữ	Viết tắt	Diễn giải
SRS	Spaced Repetition System	Cơ chế lặp lại ngắt quãng dùng để xác định thời điểm ôn từ vựng tối ưu.
JWT	JSON Web Token	Token xác thực mà backend phát hành sau khi đăng nhập thành công.

CRUD	Create/Read/Update/Delete	Bốn thao tác cơ bản trên dữ liệu quản trị nội dung.
Learner	Người học	Vai trò sử dụng hệ thống để học và kiểm tra từ vựng.
Content Creator	Biên tập viên	Vai trò tạo chủ đề, từ vựng, câu hỏi, mini test và gửi duyệt.

2. TỔNG QUAN HỆ THỐNG

VocaBoost là website học từ vựng TOEIC theo mô hình full-stack. Frontend Next.js gọi backend Express thông qua các API REST dưới tiền tố /api. Backend đảm nhiệm xác thực, phân quyền, điều phối logic SRS, quản lý nội dung, quản lý báo lỗi, duyệt nội dung, gamification và hỗ trợ AI sinh ví dụ/định nghĩa.

2.1. Các phân hệ chức năng chính

Bảng phân hệ

Thuộc tính	Mô tả
Phân hệ người học	Học flashcard, mini test, dashboard tiến độ, notebook, notifications, daily goal và hàng đợi SRS.
Phân hệ biên tập viên	Tạo chủ đề/từ vựng/câu hỏi/mini test, quản lý media và gửi nội dung đi duyệt.
Phân hệ quản trị viên	Duyệt nội dung, quản trị người dùng, báo cáo, thông báo, thống kê và import dữ liệu.
Phân hệ AI	Gợi ý nghĩa, phiên âm và câu ví dụ TOEIC bằng OpenAI + Dictionary API + Google Translate.

2.2. Luồng request backend tổng quát

Luồng backend chuẩn

Thuộc tính	Mô tả
B1	Frontend gọi apiClient tới endpoint /api/... tương ứng.
B2	Express nhận request tại index.js và định tuyến sang route module phù hợp.
B3	Middleware verifyToken/checkPermission/validate được áp dụng trước khi vào controller.
B4	Controller tách tham số, gọi service, ánh xạ lỗi nghiệp vụ sang HTTP status.
B5	Service truy vấn SQL Server, chạy transaction/stored procedure và gom business result.
B6	Controller trả JSON về frontend; errorHandler xử lý lỗi còn sót ở cuối pipeline.

3. KIẾN TRÚC HỆ THỐNG BACKEND

Backend sử dụng kiến trúc phân lớp mỏng: Route -> Middleware -> Controller -> Service -> SQL Server. Mỗi lớp có trách nhiệm rõ ràng, giúp dễ mở rộng chức năng và dễ kiểm thử tích hợp.

Phân rã backend

Thuộc tính	Mô tả
Máy chủ ứng dụng	Express.js khởi tạo tại backend/src/index.js, mount toàn bộ route dưới /api.
Tầng Route	8 file route, khai báo khoảng 123 endpoint REST.
Tầng Controller	10 controller chịu trách nhiệm request/response.
Tầng Service	11 service xử lý nghiệp vụ và SQL trực tiếp.
Middleware dùng chung	3 middleware cho auth, permission, validation, error handling.
Config/utility	3 module cấu hình/kết nối/health check.

3.1. Thành phần chính

Mô tả component

Thuộc tính	Mô tả
ApplicationServer	Khởi tạo Express, cấu hình CORS/Helmet/JSON parser, route mount, health endpoint và graceful shutdown.
Auth Middleware	Giải mã JWT, kiểm tra quyền, kiểm tra quyền sở hữu nội dung.
Validate Middleware	Dùng Zod schemas để chặn dữ liệu request không hợp lệ trước khi vào controller.
Route Modules	Định nghĩa đầy đủ endpoint cho auth, learner, admin, creator, AI, progress và review.
Controller Modules	Nhận request, chọn status code, chuyển dữ liệu sang service, ánh xạ lỗi nghiệp vụ sang response.
Service Modules	Chứa toàn bộ logic truy vấn SQL, transaction, stored procedure, SRS, import, gamification, report và AI.
SQL Server	Lưu người dùng, từ vựng, câu hỏi, mini test, tiến độ học, logs, notifications và cấu hình gamification.

4. THIẾT KẾ LỚP VÀ MODULE BACKEND

Mục này mô tả đầy đủ các file backend theo đúng code hiện tại của dự án. Mỗi module được trình bày theo phong cách tương tự tài liệu mẫu: nêu vai trò, thuộc tính cấu hình quan trọng và danh sách hàm/endpoint cùng chức năng.

4.1. Application

ApplicationServer

Khởi tạo máy chủ Express, gắn middleware toàn cục, mount route và xử lý vòng đời tiến trình.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Src
File	index.js
Trách nhiệm	Khởi tạo máy chủ Express, gắn middleware toàn cục, mount route và xử lý vòng đời tiến trình.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	gracefulShutdown	Hàm hỗ trợ xử lý graceful shutdown trong module backend.

4.2. Config

DBConfig

Cấu hình và tiện ích hệ thống phục vụ kết nối cơ sở dữ liệu.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Config
File	config/db.js
Trách nhiệm	Cấu hình và tiện ích hệ thống phục vụ kết nối cơ sở dữ liệu.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	serverClean	Hàm hỗ trợ xử lý server clean trong module backend.
Property	config	Hàm hỗ trợ xử lý config trong module backend.
Property	poolPromise	Khởi tạo và tái sử dụng connection pool SQL Server cho toàn bộ backend.
Property	sql	Hàm hỗ trợ xử lý sql trong module backend.

HealthCheckConfig

Cấu hình và tiện ích hệ thống phục vụ kiểm tra sức khỏe hệ thống.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Config
File	config/health-check.js
Trách nhiệm	Cấu hình và tiện ích hệ thống phục vụ kiểm tra sức khỏe hệ thống.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	runHealthCheck	Hàm hỗ trợ xử lý run health check trong module backend.

IntegrationTestConfig

Cấu hình và tiện ích hệ thống phục vụ kiểm thử tích hợp.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Config
File	config/integration-test.js
Trách nhiệm	Cấu hình và tiện ích hệ thống phục vụ kiểm thử tích hợp.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	runIntegrationTest	Hàm hỗ trợ xử lý run integration test trong module backend.

4.3. Middleware**AuthMiddleware**

Cung cấp middleware cắt ngang cho xác thực, tập trung vào xác thực, phân quyền, validate và chuẩn hóa lỗi.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Middleware
File	middlewares/auth.js
Trách nhiệm	Cung cấp middleware cắt ngang cho xác thực, tập trung vào xác thực, phân quyền, validate và chuẩn hóa lỗi.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
------	------------------	-------

Method	normalizePermission	Thực thi middleware để chuẩn hóa permission.
Method	getUserPermissions	Thực thi middleware để lấy user permissions.
Method	verifyToken	Kiểm tra Bearer token JWT, giải mã payload và gắn thông tin người dùng vào req.
Method	checkPermission	Tạo middleware kiểm tra một quyền cụ thể trước khi cho phép truy cập endpoint.
Method	checkAnyPermission	Tạo middleware cho phép truy cập nếu người dùng có ít nhất một quyền trong danh sách.
Method	checkOwnership	Kiểm tra quyền sở hữu bản ghi khi người dùng không có đặc quyền quản trị hệ thống.

ErrorhandlerMiddleware

Cung cấp middleware cắt ngang cho errorHandler, tập trung vào xác thực, phân quyền, validate và chuẩn hóa lỗi.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Middleware
File	middlewares/errorHandler.js
Trách nhiệm	Cung cấp middleware cắt ngang cho errorHandler, tập trung vào xác thực, phân quyền, validate và chuẩn hóa lỗi.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	errorHandler	Hàm hỗ trợ xử lý error handler trong module backend.

ValidateMiddleware

Cung cấp middleware cắt ngang cho validate, tập trung vào xác thực, phân quyền, validate và chuẩn hóa lỗi.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Middleware
File	middlewares/validate.js

Trách nhiệm	Cung cấp middleware cắt ngang cho validate, tập trung vào xác thực, phân quyền, validate và chuẩn hóa lỗi.
--------------------	--

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	validate	Hàm hỗ trợ xử lý validate trong module backend.
Schema	schemas.register	Hàm hỗ trợ xử lý schemas register trong module backend.
Schema	schemas.login	Hàm hỗ trợ xử lý schemas login trong module backend.
Schema	schemas.createWord	Hàm hỗ trợ xử lý schemas create word trong module backend.
Schema	schemas.createQuestion	Hàm hỗ trợ xử lý schemas create question trong module backend.
Schema	schemas.miniTest	Hàm hỗ trợ xử lý schemas mini test trong module backend.
Schema	schemas.createReport	Hàm hỗ trợ xử lý schemas create report trong module backend.

4.4. Routes**AdminRoute**

Khai báo các endpoint HTTP cho phân hệ quản trị hệ thống, gắn middleware quyền và nối sang controller.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Route layer
File	routes/admin.routes.js
Trách nhiệm	Khai báo các endpoint HTTP cho phân hệ quản trị hệ thống, gắn middleware quyền và nối sang controller.
Base path	/api/admin

Danh sách endpoint

HTT P	Path	Handler	Mô tả
GET	/api/admin/topics	getTopics	Endpoint GET /api/admin/topics dùng để xử lý get topics, có gắn

			middleware checkAnyPermission.
POST	/api/admin/topics	createTopic	Endpoint POST /api/admin/topics dùng để xử lý create topic, có gắn middleware checkAnyPermission, validate.
PUT	/api/admin/topics/:id	updateTopic	Endpoint PUT /api/admin/topics/:id dùng để xử lý update topic, có gắn middleware checkAnyPermission, validate.
DELETE	/api/admin/topics/:id	deleteTopic	Endpoint DELETE /api/admin/topics/:id dùng để xử lý delete topic, có gắn middleware checkAnyPermission.
GET	/api/admin/topic-categories	getTopicCategories	Endpoint GET /api/admin/topic-categories dùng để xử lý get topic categories, có gắn middleware checkAnyPermission.
POST	/api/admin/topic-categories	createTopicCategory	Endpoint POST /api/admin/topic-categories dùng để xử lý create topic category, có gắn middleware checkAnyPermission, validate.
PUT	/api/admin/topic-categories/:id	updateTopicCategory	Endpoint PUT /api/admin/topic-categories/:id dùng để xử lý update topic category, có gắn middleware checkAnyPermission, validate.
DELETE	/api/admin/topic-categories/:id	deleteTopicCategory	Endpoint DELETE /api/admin/topic-categories/:id dùng để xử lý delete topic category, có gắn

			middleware checkAnyPermission.
GET	/api/admin/words	getWords	Endpoint GET /api/admin/words dùng để xử lý get words, có gắn middleware checkPermission.
POST	/api/admin/words	createWord	Endpoint POST /api/admin/words dùng để xử lý create word, có gắn middleware checkPermission, validate.
POST	/api/admin/words/import-preview	previewWordImport	Phân tích CSV import từ vựng và trả về kết quả kiểm tra trước khi ghi dữ liệu.
POST	/api/admin/words/bulk-import	bulkImportWords	Thực thi import hàng loạt từ vựng vào cơ sở dữ liệu.
GET	/api/admin/words/:id	getWordDetail	Endpoint GET /api/admin/words/:id dùng để xử lý get word detail, có gắn middleware checkPermission.
PUT	/api/admin/words/:id	updateWord	Endpoint PUT /api/admin/words/:id dùng để xử lý update word, có gắn middleware checkPermission, validate.
DELETE	/api/admin/words/:id/hard	hardDeleteWord	Endpoint DELETE /api/admin/words/:id/hard dùng để xử lý hard delete word, có gắn middleware checkPermission.
DELETE	/api/admin/words/:id	deleteWord	Endpoint DELETE /api/admin/words/:id dùng để xử lý delete word, có gắn middleware checkPermission.
POST	/api/admin/questions/bulk-import	bulkImportQuestions	Endpoint POST /api/admin/questions/bulk-import dùng để xử lý bulk import questions, có gắn middleware

			checkPermission, express.text.
GET	/api/admin/questions/:wordId	getQuestionsByWord	Endpoint GET /api/admin/questions/:wordId dùng để xử lý get questions by word, có gắn middleware checkPermission.
POST	/api/admin/questions	createQuestion	Endpoint POST /api/admin/questions dùng để xử lý create question, có gắn middleware checkPermission, validate.
PUT	/api/admin/questions/:id	updateQuestion	Endpoint PUT /api/admin/questions/:id dùng để xử lý update question, có gắn middleware checkPermission, validate.
DELETE	/api/admin/questions/:id	deleteQuestion	Endpoint DELETE /api/admin/questions/:id dùng để xử lý delete question, có gắn middleware checkPermission.
GET	/api/admin/minitests	getMiniTests	Endpoint GET /api/admin/minitests dùng để xử lý get mini tests, có gắn middleware checkPermission.
POST	/api/admin/minitests	createMiniTest	Endpoint POST /api/admin/minitests dùng để xử lý create mini test, có gắn middleware checkPermission, validate.
PUT	/api/admin/minitests/:id	updateMiniTest	Endpoint PUT /api/admin/minitests/:id dùng để xử lý update mini test, có gắn middleware checkPermission, validate.
DELETE	/api/admin/minitests/:id	deleteMiniTest	Endpoint DELETE /api/admin/minitests/:id dùng để xử lý delete mini test, có

			gắn middleware checkPermission.
PATCH	/api/admin/minitests/:id/publish	publishMiniTest	Endpoint PATCH /api/admin/minitests/:id/publish dùng để xử lý publish mini test, có gắn middleware checkPermission.
PATCH	/api/admin/minitests/:id/archive	archiveMiniTest	Endpoint PATCH /api/admin/minitests/:id/archive dùng để xử lý archive mini test, có gắn middleware checkPermission.
GET	/api/admin/stats	getStats	Endpoint GET /api/admin/stats dùng để xử lý get stats, có gắn middleware checkPermission.
GET	/api/admin/students	getStudents	Endpoint GET /api/admin/students dùng để xử lý get students, có gắn middleware checkPermission.
POST	/api/admin/students	createUser	Endpoint POST /api/admin/students dùng để xử lý create user, có gắn middleware checkPermission, validate.
PUT	/api/admin/students/:id	updateUser	Endpoint PUT /api/admin/students/:id dùng để xử lý update user, có gắn middleware checkPermission, validate.
DELETE	/api/admin/students/:id	deleteUser	Endpoint DELETE /api/admin/students/:id dùng để xử lý delete user, có gắn middleware checkPermission.
PATCH	/api/admin/students/:id/toggle	toggleStudentStatus	Endpoint PATCH /api/admin/students/:id/toggle dùng để xử lý toggle student status, có gắn

			middleware checkPermission.
PATCH	/api/admin/students/:id/role	updateUserRole	Endpoint PATCH /api/admin/students/:id/role dùng để xử lý update user role, có gắn middleware checkPermission, validate.
GET	/api/admin/analytics	getAnalytics	Endpoint GET /api/admin/analytics dùng để xử lý get analytics, có gắn middleware checkAnyPermission.
GET	/api/admin/content- management	getContentManag ement	Endpoint GET /api/admin/content- management dùng để xử lý get content management, có gắn middleware checkAnyPermission.
PATCH	/api/admin/content-status	updateContentStat us	Endpoint PATCH /api/admin/content-status dùng để xử lý update content status, có gắn middleware checkAnyPermission, validate.
GET	/api/admin/content- review/pending	getPendingConten t	Endpoint GET /api/admin/content- review/pending dùng để xử lý get pending content, có gắn middleware checkAnyPermission.
POST	/api/admin/content- review/:entityType/:entityId/ approve	approveContent	Endpoint POST /api/admin/content- review/:entityType/:entityId/ approve dùng để xử lý approve content, có gắn middleware checkAnyPermission, validate.
POST	/api/admin/content- review/:entityType/:entityId/ reject	rejectContent	Endpoint POST /api/admin/content- review/:entityType/:entityId/ reject dùng để xử lý reject

			content, có gắn middleware checkAnyPermission, validate.
POST	/api/admin/content- review/:entityType/:entityId/ archive	archiveContent	Endpoint POST /api/admin/content- review/:entityType/:entityId/ archive dùng để xử lý archive content, có gắn middleware checkAnyPermission, validate.
GET	/api/admin/content- review/:entityType/:entityId/ logs	getContentReview Logs	Endpoint GET /api/admin/content- review/:entityType/:entityId/ logs dùng để xử lý get content review logs, có gắn middleware checkAnyPermission, validate.
GET	/api/admin/audit-logs	getAuditLogs	Endpoint GET /api/admin/audit-logs dùng để xử lý get audit logs, có gắn middleware checkAnyPermission.
GET	/api/admin/reports	getReports	Endpoint GET /api/admin/reports dùng để xử lý get reports, có gắn middleware checkAnyPermission.
PATCH	/api/admin/reports/:id	updateReport	Endpoint PATCH /api/admin/reports/:id dùng để xử lý update report, có gắn middleware checkAnyPermission, validate.
GET	/api/admin/notifications	getNotifications	Endpoint GET /api/admin/notifications dùng để xử lý get notifications, có gắn middleware checkPermission.

POST	/api/admin/notifications	sendAnnouncement	Endpoint POST /api/admin/notifications dùng để xử lý send announcement, có gắn middleware checkPermission, validate.
POST	/api/admin/notifications/daily-reminders	createDailyReminders	Endpoint POST /api/admin/notifications/daily-reminders dùng để xử lý create daily reminders, có gắn middleware checkPermission.

AIRoute

Khai báo các endpoint HTTP cho phân hệ AI hỗ trợ nội dung, gắn middleware quyền và nối sang controller.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Route layer
File	routes/ai.routes.js
Trách nhiệm	Khai báo các endpoint HTTP cho phân hệ AI hỗ trợ nội dung, gắn middleware quyền và nối sang controller.
Base path	/api/ai

Danh sách endpoint

HTTP	Path	Handler	Mô tả
POST	/api/ai/word-suggestions	suggestWordContent	Endpoint POST /api/ai/word-suggestions dùng để xử lý suggest word content, có gắn middleware checkAnyPermission, validate.

AuthRoute

Khai báo các endpoint HTTP cho phân hệ xác thực, gắn middleware quyền và nối sang controller.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Route layer
File	routes/auth.routes.js

Trách nhiệm	Khai báo các endpoint HTTP cho phân hệ xác thực, gắn middleware quyền và nối sang controller.
Base path	/api/auth

Danh sách endpoint

HTTP	Path	Handler	Mô tả
POST	/api/auth/register	register	Endpoint POST /api/auth/register dùng để xử lý register, có gắn middleware validate.
POST	/api/auth/login	login	Endpoint POST /api/auth/login dùng để xử lý login, có gắn middleware validate.

CategoriesRoute

Khai báo các endpoint HTTP cho phân hệ danh mục nền tảng, gắn middleware quyền và nối sang controller.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Route layer
File	routes/categories.routes.js
Trách nhiệm	Khai báo các endpoint HTTP cho phân hệ danh mục nền tảng, gắn middleware quyền và nối sang controller.
Base path	/api/categories

Danh sách endpoint

HTTP	Path	Handler	Mô tả
GET	/api/categories/part-of-speeches	getPartOfSpeeches	Endpoint GET /api/categories/part-of-speeches dùng để xử lý get part of speeches.
GET	/api/categories/topics	getTopics	Endpoint GET /api/categories/topics dùng để xử lý get topics.

CreatorRoute

Khai báo các endpoint HTTP cho phân hệ biên tập nội dung, gắn middleware quyền và nối sang controller.

Thuộc tính module

Thuộc tính	Mô tả
------------	-------

Tầng	Route layer
File	routes/creator.routes.js
Trách nhiệm	Khai báo các endpoint HTTP cho phân hệ biên tập nội dung, gắn middleware quyền và nối sang controller.
Base path	/api/creator

Danh sách endpoint

HTTP	Path	Handler	Mô tả
GET	/api/creator/dashboard	getDashboard	Endpoint GET /api/creator/dashboard dùng để xử lý get dashboard, có gắn middleware checkPermission.
GET	/api/creator/content-summary	getContentSummary	Endpoint GET /api/creator/content-summary dùng để xử lý get content summary, có gắn middleware checkPermission.
GET	/api/creator/topics/:id/analitics	getTopicAnalytics	Endpoint GET /api/creator/topics/:id/analitics dùng để xử lý get topic analytics, có gắn middleware checkPermission.
GET	/api/creator/mini-tests/:id/analytics	getMiniTestAnalytics	Endpoint GET /api/creator/mini-tests/:id/analytics dùng để xử lý get mini test analytics, có gắn middleware checkPermission.
GET	/api/creator/topic-categories	getTopicCategories	Endpoint GET /api/creator/topic-categories dùng để xử lý get topic categories, có gắn middleware checkAnyPermission.
GET	/api/creator/topics	getTopics	Endpoint GET /api/creator/topics dùng để xử lý get topics, có gắn

			middleware checkAnyPermission.
POST	/api/creator/topics	createTopic	Endpoint POST /api/creator/topics dùng để xử lý create topic, có gắn middleware checkPermission, validate.
PUT	/api/creator/topics/:id	updateTopic	Endpoint PUT /api/creator/topics/:id dùng để xử lý update topic, có gắn middleware checkPermission, validate.
DELETE	/api/creator/topics/:id	deleteTopic	Endpoint DELETE /api/creator/topics/:id dùng để xử lý delete topic, có gắn middleware checkPermission.
POST	/api/creator/topics/:id/sub mit-review	submitTopicForReview	Endpoint POST /api/creator/topics/:id/sub mit-review dùng để xử lý submit topic for review, có gắn middleware checkPermission.
GET	/api/creator/words	getWords	Endpoint GET /api/creator/words dùng để xử lý get words, có gắn middleware checkAnyPermission.
POST	/api/creator/words	createWord	Endpoint POST /api/creator/words dùng để xử lý create word, có gắn middleware checkPermission, validate.
PUT	/api/creator/words/:id	updateWord	Endpoint PUT /api/creator/words/:id dùng để xử lý update word, có gắn middleware checkPermission, validate.
DELETE	/api/creator/words/:id	deleteWord	Endpoint DELETE /api/creator/words/:id dùng để xử lý delete word, có

			gắn middleware checkPermission.
POST	/api/creator/words/:id/sub mit-review	submitWordForReview	Endpoint POST /api/creator/words/:id/sub mit-review dùng để xử lý submit word for review, có gắn middleware checkPermission.
GET	/api/creator/questions	getQuestions	Endpoint GET /api/creator/questions dùng để xử lý get questions, có gắn middleware checkAnyPermission.
POST	/api/creator/questions	createQuestion	Endpoint POST /api/creator/questions dùng để xử lý create question, có gắn middleware checkPermission, validate.
PUT	/api/creator/questions/:id	updateQuestion	Endpoint PUT /api/creator/questions/:id dùng để xử lý update question, có gắn middleware checkPermission, validate.
DELETE	/api/creator/questions/:id	deleteQuestion	Endpoint DELETE /api/creator/questions/:id dùng để xử lý delete question, có gắn middleware checkPermission.
POST	/api/creator/questions/:id/s ubmit-review	submitQuestionForReview	Endpoint POST /api/creator/questions/:id/s ubmit-review dùng để xử lý submit question for review, có gắn middleware checkPermission.
GET	/api/creator/mini-tests	getMiniTests	Endpoint GET /api/creator/mini-tests dùng để xử lý get mini tests, có gắn middleware checkAnyPermission.

POST	/api/creator/mini-tests	createMiniTest	Endpoint POST /api/creator/mini-tests dùng để xử lý create mini test, có gắn middleware checkPermission, validate.
PUT	/api/creator/mini-tests/:id	updateMiniTest	Endpoint PUT /api/creator/mini-tests/:id dùng để xử lý update mini test, có gắn middleware checkPermission, validate.
DELETE	/api/creator/mini-tests/:id	deleteMiniTest	Endpoint DELETE /api/creator/mini-tests/:id dùng để xử lý delete mini test, có gắn middleware checkPermission.
POST	/api/creator/mini-tests/:id/items	addMiniTestItem	Endpoint POST /api/creator/mini-tests/:id/items dùng để xử lý add mini test item, có gắn middleware checkPermission, validate.
DELETE	/api/creator/mini-tests/:id/items/:questionId	removeMiniTestItem	Endpoint DELETE /api/creator/mini-tests/:id/items/:questionId dùng để xử lý remove mini test item, có gắn middleware checkPermission.
POST	/api/creator/mini-tests/:id/submit-review	submitMiniTestForReview	Endpoint POST /api/creator/mini-tests/:id/submit-review dùng để xử lý submit mini test for review, có gắn middleware checkPermission.
GET	/api/creator/media	getMedia	Endpoint GET /api/creator/media dùng để xử lý get media, có gắn middleware checkAnyPermission.
POST	/api/creator/media	createMedia	Endpoint POST /api/creator/media dùng để

			xử lý create media, có gắn middleware checkAnyPermission, validate.
DELETE	/api/creator/media/:id	deleteMedia	Endpoint DELETE /api/creator/media/:id dùng để xử lý delete media, có gắn middleware checkAnyPermission.

ProgressRoute

Khai báo các endpoint HTTP cho phân hệ tiến độ học tập, gắn middleware quyền và nối sang controller.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Route layer
File	routes/progress.routes.js
Trách nhiệm	Khai báo các endpoint HTTP cho phân hệ tiến độ học tập, gắn middleware quyền và nối sang controller.
Base path	/api/progress

Danh sách endpoint

HTTP	Path	Handler	Mô tả
GET	/api/progress/	getProgress	Endpoint GET /api/progress/ dùng để xử lý get progress.
GET	/api/progress/stats	getStats	Endpoint GET /api/progress/stats dùng để xử lý get stats.

ReviewRoute

Khai báo các endpoint HTTP cho phân hệ duyệt nội dung, gắn middleware quyền và nối sang controller.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Route layer
File	routes/review.routes.js
Trách nhiệm	Khai báo các endpoint HTTP cho phân hệ duyệt nội dung, gắn middleware quyền và nối sang controller.
Base path	/api/admin/content-review

Danh sách endpoint

HTT P	Path	Handler	Mô tả
GET	/api/admin/content-review/pending	getPending	Endpoint GET /api/admin/content-review/pending dùng để xử lý get pending, có gắn middleware checkPermission.
POST	/api/admin/content-review/:entityType/:entityId/approve	approve	Endpoint POST /api/admin/content-review/:entityType/:entityId/approve dùng để xử lý approve, có gắn middleware checkPermission.
POST	/api/admin/content-review/:entityType/:entityId/reject	reject	Endpoint POST /api/admin/content-review/:entityType/:entityId/reject dùng để xử lý reject, có gắn middleware checkPermission.
POST	/api/admin/content-review/:entityType/:entityId/archive	archive	Endpoint POST /api/admin/content-review/:entityType/:entityId/archive dùng để xử lý archive, có gắn middleware checkPermission.
GET	/api/admin/content-review/:entityType/:entityId/logs	getReviewLogs	Endpoint GET /api/admin/content-review/:entityType/:entityId/logs dùng để xử lý get review logs, có gắn middleware checkPermission.

UserRoute

Khai báo các endpoint HTTP cho phân hệ người học, gắn middleware quyền và nối sang controller.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Route layer
File	routes/user.routes.js
Trách nhiệm	Khai báo các endpoint HTTP cho phân hệ người học, gắn middleware quyền và nối sang controller.
Base path	/api/user

Danh sách endpoint

HTT P	Path	Handler	Mô tả
POS T	/api/user/submit-answer	submitAnswer	Nhận kết quả trả lời bài tập/flashcard từ người học, cập nhật attempt và logic SRS.
GET	/api/user/flashcards	getFlashcards	Trả về bộ flashcard hiện tại cho learner dựa trên tiến độ và lịch ôn.
GET	/api/user/stats	getStats	Endpoint GET /api/user/stats dùng để xử lý get stats.
GET	/api/user/gamification/profile	getProfile	Endpoint GET /api/user/gamification/profile dùng để xử lý get profile.
POS T	/api/user/gamification/practice-complete	completePractice	Endpoint POST /api/user/gamification/practice-complete dùng để xử lý complete practice.
PUT	/api/user/gamification/achievements/seen	markAchievementsSeen	Endpoint PUT /api/user/gamification/achievements/seen dùng để xử lý mark achievements seen.
GET	/api/user/dashboard/mastery-timeline	getMasteryTimeline	Endpoint GET /api/user/dashboard/mastery-timeline dùng để xử lý get mastery timeline.
GET	/api/user/learning-path	getRoadmap	Endpoint GET /api/user/learning-path dùng để xử lý get roadmap.
GET	/api/user/topics/:topicId/words	getTopicWords	Endpoint GET /api/user/topics/:topicId/words dùng để xử lý get topic words.
GET	/api/user/minitests	getMiniTests	Endpoint GET /api/user/minitests dùng để xử lý get mini tests.
GET	/api/user/minitests/history	getTestHistory	Endpoint GET /api/user/minitests/history dùng để xử lý get test history.

GET	/api/user/minitests/session-details	getTestSessionDetails	Endpoint GET /api/user/minitests/session-details dùng để xử lý get test session details.
GET	/api/user/minitests/:id	getMiniTestDetails	Endpoint GET /api/user/minitests/:id dùng để xử lý get mini test details.
PUT	/api/user/profile	updateProfile	Endpoint PUT /api/user/profile dùng để xử lý update profile.
POST	/api/user/minitests/:id/submit	submitMiniTest	Endpoint POST /api/user/minitests/:id/submit dùng để xử lý submit mini test.
POST	/api/user/reports	createReport	Endpoint POST /api/user/reports dùng để xử lý create report, có gắn middleware validate.
GET	/api/user/activity/heatmap	getActivityHeatmap	Endpoint GET /api/user/activity/heatmap dùng để xử lý get activity heatmap.
GET	/api/user/progress/analytics	getProgressAnalytics	Endpoint GET /api/user/progress/analytics dùng để xử lý get progress analytics.
GET	/api/user/goals/daily-progress	getDailyProgress	Endpoint GET /api/user/goals/daily-progress dùng để xử lý get daily progress.
GET	/api/user/goals/daily-goal	getDailyGoal	Endpoint GET /api/user/goals/daily-goal dùng để xử lý get daily goal.
PUT	/api/user/goals/daily-goal	updateDailyGoal	Endpoint PUT /api/user/goals/daily-goal dùng để xử lý update daily goal.
PUT	/api/user/goals/srs-config	updateSRSSConfig	Cập nhật giới hạn số lượng mục ôn SRS trong mỗi phiên học.
GET	/api/user/review/smart-queue	getSmartReviewQueue	Trả danh sách ôn ưu tiên theo điểm nguy cơ quên.

GET	/api/user/notifications	getNotifications	Endpoint GET /api/user/notifications dùng để xử lý get notifications.
PUT	/api/user/notifications/:id/read	markNotificationRead	Endpoint PUT /api/user/notifications/:id/read dùng để xử lý mark notification read.
PUT	/api/user/notifications/read-all	markAllNotificationsRead	Endpoint PUT /api/user/notifications/read-all dùng để xử lý mark all notifications read.
GET	/api/user/review/session-summary	getSessionSummary	Endpoint GET /api/user/review/session-summary dùng để xử lý get session summary.
GET	/api/user/review/mistakes	getMistakeReviewQueue	Endpoint GET /api/user/review/mistakes dùng để xử lý get mistake review queue.
GET	/api/user/notebook	getNotebook	Endpoint GET /api/user/notebook dùng để xử lý get notebook.
GET	/api/user/notebook/check	checkNotebookEntry	Endpoint GET /api/user/notebook/check dùng để xử lý check notebook entry.
POST	/api/user/notebook	addNotebookEntry	Endpoint POST /api/user/notebook dùng để xử lý add notebook entry.
PUT	/api/user/notebook/:id	updateNotebookEntry	Endpoint PUT /api/user/notebook/:id dùng để xử lý update notebook entry.
DELETE	/api/user/notebook/:id	deleteNotebookEntry	Endpoint DELETE /api/user/notebook/:id dùng để xử lý delete notebook entry.

4.5. Controllers

AdminController

Nhận request/response cho phân hệ quản trị hệ thống, gọi service và chuẩn hóa HTTP status trả về.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Controller
File	controllers/admin.controller.js
Trách nhiệm	Nhận request/response cho phân hệ quản trị hệ thống, gọi service và chuẩn hóa HTTP status trả về.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	getTopics	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách chủ đề.
Method	getTopicCategories	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh mục chủ đề.
Method	createTopic	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới chủ đề.
Method	updateTopic	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật chủ đề.
Method	deleteTopic	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa chủ đề.
Method	createTopicCategory	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới danh mục chủ đề.
Method	updateTopicCategory	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật danh mục chủ đề.
Method	deleteTopicCategory	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa danh mục chủ đề.
Method	getWords	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách từ vựng.
Method	createWord	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới từ vựng.
Method	getWordDetail	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy chi tiết từ vựng.

Method	bulkImportWords	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để nhập hàng loạt danh sách từ vựng.
Method	previewWordImport	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xem trước dữ liệu import từ vựng.
Method	updateWord	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật từ vựng.
Method	deleteWord	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa từ vựng.
Method	hardDeleteWord	Hàm hỗ trợ xử lý hard delete word trong module backend.
Method	getQuestionsByWord	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy các câu hỏi của một từ vựng.
Method	createQuestion	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới câu hỏi.
Method	updateQuestion	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật câu hỏi.
Method	deleteQuestion	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa câu hỏi.
Method	getOverviewStats	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy thống kê tổng quan.
Method	getWeeklyActivity	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy hoạt động theo tuần.
Method	getTodayActivity	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy hoạt động trong ngày.
Method	getRecentUsers	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách người dùng gần đây.
Method	getTopCourses	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách chủ đề/bộ học nổi bật.

Method	bulkImportQuestions	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để nhập hàng loạt danh sách câu hỏi.
Method	getMiniTests	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách bài kiểm tra ngắn.
Method	createMiniTest	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới bài kiểm tra ngắn.
Method	updateMiniTest	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật bài kiểm tra ngắn.
Method	deleteMiniTest	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa bài kiểm tra ngắn.
Method	publishMiniTest	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xuất bản bài kiểm tra ngắn.
Method	archiveMiniTest	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lưu trữ bài kiểm tra ngắn.
Method	getStats	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy thống kê.
Method	getStudents	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách người dùng/học viên.
Method	createUser	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới user.
Method	updateUser	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật user.
Method	deleteUser	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa user.
Method	toggleStudentStatus	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để bật/tắt trạng thái tài khoản học viên.
Method	updateUserRole	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật vai trò người dùng.

Method	getAnalytics	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy dữ liệu phân tích.
Method	getContentManagement	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy content management.
Method	updateContentStatus	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật trạng thái nội dung.
Method	getPendingContent	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách nội dung chờ duyệt.
Method	approveContent	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để phê duyệt content.
Method	rejectContent	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để từ chối content.
Method	archiveContent	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lưu trữ content.
Method	getContentReviewLogs	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy content review logs.
Method	getNotifications	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách thông báo.
Method	getAuditLogs	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy nhật ký audit quản trị.
Method	getReports	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách báo lỗi/nội dung.
Method	updateReport	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật báo lỗi/nội dung.
Method	sendAnnouncement	Hàm hỗ trợ xử lý send announcement trong module backend.

Method	createDailyReminders	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới daily reminders.
--------	----------------------	--

AIController

Nhận request/response cho phân hệ AI hỗ trợ nội dung, gọi service và chuẩn hóa HTTP status trả về.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Controller
File	controllers/ai.controller.js
Trách nhiệm	Nhận request/response cho phân hệ AI hỗ trợ nội dung, gọi service và chuẩn hóa HTTP status trả về.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	suggestWordContent	Hàm hỗ trợ xử lý suggest word content trong module backend.

AuthController

Nhận request/response cho phân hệ xác thực, gọi service và chuẩn hóa HTTP status trả về.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Controller
File	controllers/auth.controller.js
Trách nhiệm	Nhận request/response cho phân hệ xác thực, gọi service và chuẩn hóa HTTP status trả về.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	register	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để đăng ký register.
Method	login	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xác thực login.

CategoriesController

Nhận request/response cho phân hệ danh mục nền tảng, gọi service và chuẩn hóa HTTP status trả về.

Thuộc tính module

Thuộc tính	Mô tả
------------	-------

Tầng	Controller
File	controllers/categories.controller.js
Trách nhiệm	Nhận request/response cho phân hệ danh mục nền tảng, gọi service và chuẩn hóa HTTP status trả về.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	getPartOfSpeeches	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh mục từ loại.
Method	getTopics	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách chủ đề.

CreatorController

Nhận request/response cho phân hệ biên tập nội dung, gọi service và chuẩn hóa HTTP status trả về.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Controller
File	controllers/creator.controller.js
Trách nhiệm	Nhận request/response cho phân hệ biên tập nội dung, gọi service và chuẩn hóa HTTP status trả về.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	getDashboard	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy dashboard.
Method	getContentSummary	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy tổng hợp trạng thái nội dung.
Method	getTopicAnalytics	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy thống kê theo chủ đề.
Method	getMiniTestAnalytics	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy thống kê bài kiểm tra ngắn.

Method	getTopicCategories	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh mục chủ đề.
Method	getTopics	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách chủ đề.
Method	createTopic	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới chủ đề.
Method	updateTopic	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật chủ đề.
Method	deleteTopic	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa chủ đề.
Method	submitTopicForReview	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gửi topic for review.
Method	getWords	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách từ vựng.
Method	createWord	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới từ vựng.
Method	updateWord	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật từ vựng.
Method	deleteWord	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa từ vựng.
Method	submitWordForReview	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gửi word for review.
Method	getQuestions	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách câu hỏi.
Method	createQuestion	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới câu hỏi.
Method	updateQuestion	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật câu hỏi.
Method	deleteQuestion	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa câu hỏi.

Method	submitQuestionForReview	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gửi question for review.
Method	getMiniTests	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách bài kiểm tra ngắn.
Method	createMiniTest	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới bài kiểm tra ngắn.
Method	updateMiniTest	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật bài kiểm tra ngắn.
Method	deleteMiniTest	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa bài kiểm tra ngắn.
Method	addMiniTestItem	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để thêm một phần tử câu hỏi của bài kiểm tra.
Method	removeMiniTestItem	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gỡ bỏ một phần tử câu hỏi của bài kiểm tra.
Method	submitMiniTestForReview	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gửi mini test for review.
Method	getMedia	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách media metadata.
Method	createMedia	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới danh sách media metadata.
Method	deleteMedia	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa danh sách media metadata.
Method	getCreatorErrorMessage	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy creator error message.

GamificationController

Nhận request/response cho phân hệ gamification và phần thưởng, gọi service và chuẩn hóa HTTP status trả về.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Controller
File	controllers/gamification.controller.js
Trách nhiệm	Nhận request/response cho phân hệ gamification và phần thưởng, gọi service và chuẩn hóa HTTP status trả về.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	getProfile	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy hồ sơ người dùng.
Method	completePractice	Hàm hỗ trợ xử lý complete practice trong module backend.
Method	markAchievementsSeen	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để đánh dấu trạng thái đã xem huy hiệu.

LearningPathController

Nhận request/response cho phân hệ lộ trình học tập, gọi service và chuẩn hóa HTTP status trả về.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Controller
File	controllers/learning-path.controller.js
Trách nhiệm	Nhận request/response cho phân hệ lộ trình học tập, gọi service và chuẩn hóa HTTP status trả về.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	getRoadmap	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy lộ trình học tập.

ProgressController

Nhận request/response cho phân hệ tiến độ học tập, gọi service và chuẩn hóa HTTP status trả về.

Thuộc tính module

Thuộc tính	Mô tả
------------	-------

Tầng	Controller
File	controllers/progress.controller.js
Trách nhiệm	Nhận request/response cho phân hệ tiến độ học tập, gọi service và chuẩn hóa HTTP status trả về.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	getProgress	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy progress.
Method	getStats	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy thống kê.

ReviewController

Nhận request/response cho phân hệ duyệt nội dung, gọi service và chuẩn hóa HTTP status trả về.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Controller
File	controllers/review.controller.js
Trách nhiệm	Nhận request/response cho phân hệ duyệt nội dung, gọi service và chuẩn hóa HTTP status trả về.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	getPending	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách chờ xử lý.
Method	approve	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để phê duyệt approve.
Method	reject	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để từ chối reject.
Method	archive	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lưu trữ archive.
Method	getReviewLogs	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy nhật ký duyệt nội dung.

UserController

Nhận request/response cho phân hệ người học, gọi service và chuẩn hóa HTTP status trả về.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Controller
File	controllers/user.controller.js
Trách nhiệm	Nhận request/response cho phân hệ người học, gọi service và chuẩn hóa HTTP status trả về.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	submitAnswer	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gửi answer.
Method	getFlashcards	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy bộ flashcard.
Method	getTopicWords	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách từ vựng của chủ đề.
Method	getStats	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy thống kê.
Method	getMasteryTimeline	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy dòng thời gian mức độ ghi nhớ.
Method	getMiniTests	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách bài kiểm tra ngắn.
Method	getMiniTestDetails	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy chi tiết bài kiểm tra ngắn.
Method	updateProfile	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật hồ sơ người dùng.
Method	getTestHistory	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy test history.
Method	getTestSessionDetails	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy test session details.
Method	createReport	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới báo lỗi/nội dung.

Method	getDailyGoal	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy mục tiêu hằng ngày.
Method	updateDailyGoal	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật mục tiêu hằng ngày.
Method	updateSRSSConfig	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật cấu hình giới hạn ôn SRS.
Method	submitMiniTest	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gửi bài kiểm tra ngắn.
Method	getActivityHeatmap	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy activity heatmap.
Method	getProgressAnalytics	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy phân tích tiến độ học tập.
Method	getDailyProgress	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy tiến độ mục tiêu hằng ngày.
Method	getSmartReviewQueue	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy hàng đợi ôn tập thông minh.
Method	getNotebook	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy sổ tay từ vựng.
Method	addNotebookEntry	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để thêm một mục trong sổ tay từ vựng.
Method	updateNotebookEntry	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật một mục trong sổ tay từ vựng.
Method	deleteNotebookEntry	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa một mục trong sổ tay từ vựng.

Method	checkNotebookEntry	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để kiểm tra một mục trong sổ tay từ vựng.
Method	getNotifications	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách thông báo.
Method	markNotificationRead	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để đánh dấu notification read.
Method	markAllNotificationsRead	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để đánh dấu all notifications read.
Method	getSessionSummary	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy tóm tắt phiên ôn tập.
Method	getMistakeReviewQueue	Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy hàng đợi ôn lại câu sai.

4.6. Services

AdminService

Thực hiện toàn bộ nghiệp vụ chính của phân hệ quản trị hệ thống, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Service
File	services/admin.service.js
Trách nhiệm	Thực hiện toàn bộ nghiệp vụ chính của phân hệ quản trị hệ thống, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	logAdminAction	Hàm hỗ trợ xử lý log admin action trong module backend.
Method	logContentReview	Hàm hỗ trợ xử lý log content review trong module backend.
Method	getTopics	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách chủ đề.

Method	getTopicCategories	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh mục chủ đề.
Method	createTopic	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới chủ đề.
Method	updateTopic	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật chủ đề.
Method	deleteTopic	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa chủ đề.
Method	createTopicCategory	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới danh mục chủ đề.
Method	updateTopicCategory	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật danh mục chủ đề.
Method	deleteTopicCategory	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa danh mục chủ đề.
Method	getWords	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách từ vựng.
Method	attachWordRelations	Hàm hỗ trợ xử lý attach word relations trong module backend.
Method	getWordDetail	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy chi tiết từ vựng.
Method	createWord	Tạo mới từ vựng trong transaction, gán chủ đề, thêm ví dụ minh họa và ghi nhật ký quản trị.
Method	updateWord	Cập nhật từ vựng, làm mới liên kết chủ đề/câu ví dụ trong transaction và lưu audit log.
Method	archiveWord	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lưu trữ từ vựng.
Method	deleteWord	Xóa mềm hoặc xóa logic từ vựng theo ràng buộc nghiệp vụ và cập nhật nhật ký thao tác.
Method	bulkInsertWords	Đọc dữ liệu import hàng loạt, ánh xạ topic/từ loại và tạo từng từ vựng vào CSDL.
Method	previewWordImport	Tiền kiểm dữ liệu CSV import từ vựng, ánh xạ reference data và trả danh sách lỗi theo từng dòng.

Method	getQuestionsByWord	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy các câu hỏi của một từ vựng.
Method	createQuestion	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới câu hỏi.
Method	updateQuestion	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật câu hỏi.
Method	deleteQuestion	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa câu hỏi.
Method	getMiniTests	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách bài kiểm tra ngắn.
Method	createMiniTest	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới bài kiểm tra ngắn.
Method	updateMiniTest	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật bài kiểm tra ngắn.
Method	deleteMiniTest	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa bài kiểm tra ngắn.
Method	setMiniTestStatus	Hàm hỗ trợ xử lý set mini test status trong module backend.
Method	getDashboardStats	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy thống kê dashboard.
Method	getStudents	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách người dùng/học viên.
Method	createUser	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới user.
Method	bulkInsertQuestions	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để ghi hàng loạt danh sách câu hỏi.
Method	updateUser	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật user.
Method	deleteUser	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa user.
Method	toggleUserStatus	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để bật/tắt user status.
Method	updateUserRole	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật vai trò người dùng.
Method	updateContentStatus	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật trạng thái nội dung.

Method	getPendingContent	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách nội dung chờ duyệt.
Method	getContentReviewLogs	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy content review logs.
Method	getAnalyticsData	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy analytics data.
Method	getContentManagementData	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy content management data.
Method	getNotifications	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách thông báo.
Method	getAuditLogs	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy nhật ký audit quản trị.
Method	sendAnnouncement	Hàm hỗ trợ xử lý send announcement trong module backend.
Method	createDailyReminders	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới daily reminders.
Method	normalizePagination	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để chuẩn hóa pagination.
Method	paginate	Hàm hỗ trợ xử lý paginate trong module backend.
Method	buildTopicCode	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xây dựng topic code.
Method	assertContentStatus	Hàm hỗ trợ xử lý assert content status trong module backend.
Method	normalizeImportKey	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để chuẩn hóa import key.
Method	parseDelimitedImport	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để phân tích và chuẩn hóa delimited import.
Method	getImportValue	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy import value.
Method	splitImportList	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tách import list.
Method	parseQuestionImport	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để phân tích và chuẩn hóa question import.

Method	parseWordImport	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để phân tích và chuẩn hóa dữ liệu import từ vựng.
Method	assertValidUserRole	Hàm hỗ trợ xử lý assert valid user role trong module backend.

AIService

Thực hiện toàn bộ nghiệp vụ chính của phân hệ AI hỗ trợ nội dung, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Service
File	services/ai.service.js
Trách nhiệm	Thực hiện toàn bộ nghiệp vụ chính của phân hệ AI hỗ trợ nội dung, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	suggestWordContent	Điều phối tra từ điển, dịch nghĩa và gọi OpenAI để gợi ý nội dung từ vựng TOEIC hoàn chỉnh.
Method	generateToeicExamples	Hàm hỗ trợ xử lý generate toeic examples trong module backend.
Method	translateToVietnamese	Hàm hỗ trợ xử lý translate to vietnamese trong module backend.
Method	lookupDictionary	Hàm hỗ trợ xử lý lookup dictionary trong module backend.
Method	firstMeaning	Hàm hỗ trợ xử lý first meaning trong module backend.
Method	firstDefinition	Hàm hỗ trợ xử lý first definition trong module backend.
Method	withTimeout	Hàm hỗ trợ xử lý with timeout trong module backend.
Method	cleanText	Hàm hỗ trợ xử lý clean text trong module backend.
Method	extractResponseText	Hàm hỗ trợ xử lý extract response text trong module backend.

Method	parseJsonText	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để phân tích và chuẩn hóa json text.
Method	normalizeExamples	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để chuẩn hóa examples.

AuthService

Thực hiện toàn bộ nghiệp vụ chính của phân hệ xác thực, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Service
File	services/auth.service.js
Trách nhiệm	Thực hiện toàn bộ nghiệp vụ chính của phân hệ xác thực, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	register	Kiểm tra trùng email, băm mật khẩu, gán role Learner mặc định và tạo tài khoản mới.
Method	login	Xác thực email/mật khẩu, tải quyền từ RolePermissions, sinh JWT và cộng điểm đăng nhập hằng ngày cho learner.

CategoriesService

Thực hiện toàn bộ nghiệp vụ chính của phân hệ danh mục nền tảng, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Service
File	services/categories.service.js
Trách nhiệm	Thực hiện toàn bộ nghiệp vụ chính của phân hệ danh mục nền tảng, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	getPartOfSpeeches	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh mục từ loại.

Method	getTopics	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách chủ đề.
--------	-----------	--

CreatorService

Thực hiện toàn bộ nghiệp vụ chính của phân hệ biên tập nội dung, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Service
File	services/creator.service.js
Trách nhiệm	Thực hiện toàn bộ nghiệp vụ chính của phân hệ biên tập nội dung, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	validateMiniTestReferences	Hàm hỗ trợ xử lý validate mini test references trong module backend.
Method	getDashboardStats	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy thống kê dashboard.
Method	getContentSummary	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy tổng hợp trạng thái nội dung.
Method	getTopicAnalytics	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy thống kê theo chủ đề.
Method	getMiniTestAnalytics	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy thống kê bài kiểm tra ngắn.
Method	getTopicCategories	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh mục chủ đề.
Method	getMyTopics	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy my topics.
Method	createTopic	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới chủ đề.
Method	updateTopic	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật chủ đề.
Method	deleteTopic	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa chủ đề.
Method	submitForReview	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để gửi for review.
Method	getMyWords	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy my words.

Method	createWord	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới từ vựng.
Method	updateWord	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật từ vựng.
Method	deleteWord	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa từ vựng.
Method	getMyQuestions	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy my questions.
Method	createQuestion	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới câu hỏi.
Method	updateQuestion	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật câu hỏi.
Method	deleteQuestion	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa câu hỏi.
Method	getMyMiniTests	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy my mini tests.
Method	createMiniTest	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới bài kiểm tra ngắn.
Method	updateMiniTest	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật bài kiểm tra ngắn.
Method	deleteMiniTest	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa bài kiểm tra ngắn.
Method	addMiniTestItem	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để thêm một phần tử câu hỏi của bài kiểm tra.
Method	removeMiniTestItem	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để gỡ bỏ một phần tử câu hỏi của bài kiểm tra.
Method	getMedia	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách media metadata.
Method	createMedia	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới danh sách media metadata.
Method	deleteMedia	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa danh sách media metadata.
Method	submitQuestionForReview	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để gửi question for review.

Method	submitWordForReview	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để gửi word for review.
Method	submitMiniTestForReview	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để gửi mini test for review.
Method	parseQuestionOptions	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để phân tích và chuẩn hóa question options.
Method	offset	Hàm hỗ trợ xử lý offset trong module backend.

GamificationService

Thực hiện toàn bộ nghiệp vụ chính của phân hệ gamification và phần thưởng, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Service
File	services/gamification.service.js
Trách nhiệm	Thực hiện toàn bộ nghiệp vụ chính của phân hệ gamification và phần thưởng, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	ensureSchema	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để bảo đảm sẵn sàng schema.
Method	createSchema	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới schema.
Method	awardXP	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cộng thưởng cho xp.
Method	awardDailyLogin	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cộng thưởng cho daily login.
Method	checkAchievements	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để kiểm tra achievements.
Method	getMetrics	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy metrics.
Method	getAchievements	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy achievements.
Method	getProfile	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy hồ sơ người dùng.

Method	markAchievementsSeen	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để đánh dấu trạng thái đã xem huy hiệu.
Method	getDateKey	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy date key.
Method	getLevelState	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy level state.
Method	getAchievementProgress	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy achievement progress.

LearningPathService

Thực hiện toàn bộ nghiệp vụ chính của phân hệ lộ trình học tập, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Service
File	services/learning-path.service.js
Trách nhiệm	Thực hiện toàn bộ nghiệp vụ chính của phân hệ lộ trình học tập, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	ensureSchema	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để bảo đảm sẵn sàng schema.
Method	createSchema	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới schema.
Method	syncPublishedTopics	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để đồng bộ published topics.
Method	getRoadmap	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy lộ trình học tập.
Method	buildRoadmap	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xây dựng lộ trình học tập.

ProgressService

Thực hiện toàn bộ nghiệp vụ chính của phân hệ tiến độ học tập, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Service

File	services/progress.service.js
Trách nhiệm	Thực hiện toàn bộ nghiệp vụ chính của phân hệ tiến độ học tập, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	getProgress	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy progress.
Method	getStats	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy thống kê.

ReportService

Thực hiện toàn bộ nghiệp vụ chính của phân hệ báo lỗi nội dung, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Service
File	services/report.service.js
Trách nhiệm	Thực hiện toàn bộ nghiệp vụ chính của phân hệ báo lỗi nội dung, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	ensureSchema	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để bảo đảm sẵn sàng schema.
Method	createReport	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới báo lỗi/nội dung.
Method	getReports	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách báo lỗi/nội dung.
Method	updateReport	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật báo lỗi/nội dung.
Method	normalizePagination	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để chuẩn hóa pagination.
Method	paginate	Hàm hỗ trợ xử lý paginate trong module backend.
Method	assertOneOf	Hàm hỗ trợ xử lý assert one of trong module backend.
Method	inferEntityType	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để suy luận entity type.

ReviewService

Thực hiện toàn bộ nghiệp vụ chính của phân hệ duyệt nội dung, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Service
File	services/review.service.js
Trách nhiệm	Thực hiện toàn bộ nghiệp vụ chính của phân hệ duyệt nội dung, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
Method	getPendingContent	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách nội dung chờ duyệt.
Method	approve	Phê duyệt nội dung chờ duyệt, cập nhật trạng thái Published, ghi mốc reviewed/published và lưu review log.
Method	reject	Từ chối nội dung PendingReview, lưu lý do từ chối và ghi nhận lịch sử duyệt.
Method	archive	Chuyển nội dung sang Archived và ghi nhận thao tác lưu trữ vào bảng log duyệt.
Method	getReviewLogs	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy nhật ký duyệt nội dung.
Private	_resolveEntity	Hàm nội bộ dùng để hỗ trợ lớp/module trong việc resolve entity.

UserService

Thực hiện toàn bộ nghiệp vụ chính của phân hệ người học, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Thuộc tính module

Thuộc tính	Mô tả
Tầng	Service
File	services/user.service.js
Trách nhiệm	Thực hiện toàn bộ nghiệp vụ chính của phân hệ người học, làm việc trực tiếp với SQL Server và stored procedure khi cần.

Danh sách hàm

Loại	Hàm / thành phần	Mô tả
------	------------------	-------

Method	getFlashcards	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy bộ flashcard.
Method	getDueFlashcards	Lấy danh sách flashcard đến hạn ôn theo SRS, có thể lọc theo chủ đề hoặc chế độ new/learned.
Method	getTopicWords	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách từ vựng của chủ đề.
Method	submitAnswer	Ghi nhận câu trả lời, gọi stored procedure lưu attempt, cập nhật lịch SRS theo reviewRating và cộng XP nếu cần.
Method	submitWordReview	Cập nhật trực tiếp UserWordProgress theo kết quả ôn từ, tính mastery/ease factor/next review date và gamification.
Method	getUserStats	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy user stats.
Method	getMasteryTimeline	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy dòng thời gian mức độ ghi nhớ.
Method	getMiniTests	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách bài kiểm tra ngắn.
Method	getMiniTestDetails	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy chi tiết bài kiểm tra ngắn.
Method	updateProfile	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật hồ sơ người dùng.
Method	getTestHistory	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy test history.
Method	getTestSessionDetails	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy test session details.
Method	getActivityHeatmap	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy activity heatmap.
Method	getProgressAnalytics	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy phân tích tiến độ học tập.
Method	getDailyProgress	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy tiến độ mục tiêu hằng ngày.
Method	getSmartReviewQueue	Xây dựng hàng đợi ôn thông minh dựa trên độ quá hạn, số lần sai liên tiếp và mức độ thành thạo.

Method	submitMiniTestBatch	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để gửi mini test batch.
Method	getDailyGoal	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy mục tiêu hằng ngày.
Method	updateDailyGoal	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật mục tiêu hằng ngày.
Method	updateSRSReviewLimit	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật srs review limit.
Method	getUserNotifications	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy user notifications.
Method	markNotificationRead	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để đánh dấu notification read.
Method	markAllNotificationsRead	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để đánh dấu all notifications read.
Method	getNotebook	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy sổ tay từ vựng.
Method	addNotebookEntry	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để thêm một mục trong sổ tay từ vựng.
Method	updateNotebookEntry	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật một mục trong sổ tay từ vựng.
Method	deleteNotebookEntry	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa một mục trong sổ tay từ vựng.
Method	getSessionSummary	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy tóm tắt phiên ôn tập.
Method	getMistakeReviewQueue	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy hàng đợi ôn lại câu sai.
Method	checkNotebookEntry	Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để kiểm tra một mục trong sổ tay từ vựng.
Method	offset	Hàm hỗ trợ xử lý offset trong module backend.
Method	percentage	Hàm hỗ trợ xử lý percentage trong module backend.

5. GIẢI MÃ CHI TIẾT TỪNG HÀM BACKEND

Chương này mô tả giả mã cho toàn bộ 305 hàm có tên được phát hiện trong backend/src. Giả mã được sinh từ cấu trúc câu lệnh thực tế của mã nguồn và giữ lại đầy đủ các nhánh TRY/CATCH, IF/ELSE, vòng lặp, lời gọi service, transaction, truy vấn SQL, stored procedure, RETURN và HTTP response. Cách trình bày thống nhất theo mẫu FUNCTION ... END FUNCTION mà nhóm sử dụng trong báo cáo.

Thông kê phạm vi giả mã

Thuộc tính	Mô tả
config/health-check.js	1 hàm
config/integration-test.js	1 hàm
controllers/admin.controller.js	53 hàm
controllers/ai.controller.js	1 hàm
controllers/auth.controller.js	2 hàm
controllers/categories.controller.js	2 hàm
controllers/creator.controller.js	31 hàm
controllers/gamification.controller.js	3 hàm
controllers/learning-path.controller.js	1 hàm
controllers/progress.controller.js	2 hàm
controllers/review.controller.js	5 hàm
controllers/user.controller.js	29 hàm
index.js	1 hàm
middlewares/auth.js	6 hàm
middlewares/errorHandler.js	1 hàm
middlewares/validate.js	1 hàm
services/admin.service.js	56 hàm
services/ai.service.js	9 hàm
services/auth.service.js	2 hàm
services/categories.service.js	2 hàm
services/creator.service.js	32 hàm
services/gamification.service.js	12 hàm
services/learning-path.service.js	5 hàm
services/progress.service.js	2 hàm
services/report.service.js	8 hàm
services/review.service.js	6 hàm
services/user.service.js	31 hàm

5.1. Config và kiểm tra vận hành

config/health-check.js

File này có 1 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 1: async runHealthCheck()

Dòng nguồn: 4 | Đầu vào: Không có tham số | Đầu ra: Kết quả kiểm tra/cấu hình phục vụ vận hành backend.

Mục đích: Hàm hỗ trợ xử lý run health check trong module backend.

```
FUNCTION runHealthCheck()
  THỰC HIỆN console.log("--- VOCABOOST SYSTEM DIAGNOSTIC ---")
  requiredEnv = ["DB_SERVER", "DB_NAME", "JWT_SECRET"]
  missing = requiredEnv.filter(HÀM CALLBACK(k))
  IF missing.length > 0 THEN
    THỰC HIỆN console.error("✗ MISSING ENV VARS: ", missing.join(", "))
  ELSE
    THỰC HIỆN console.log("☑ Environment Variables: OK")
  END IF
  TRY
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    THỰC HIỆN console.log("☑ Database Connection: OK")
    tables = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT name FROM sys.tables"
    THỰC HIỆN console.log(`☑ Database Schema: ${ tables.recordset.length }
      tables found`)
  CATCH err
    THỰC HIỆN console.error("✗ Database Connection: FAILED")
    THỰC HIỆN console.error(" Error: ", err.message)
  END TRY
  THỰC HIỆN console.log("-----")
  THỰC HIỆN process.exit(0)
END FUNCTION
```

config/integration-test.js

File này có 1 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 2: async runIntegrationTest()

Dòng nguồn: 6 | Đầu vào: Không có tham số | Đầu ra: Kết quả kiểm tra/cấu hình phục vụ vận hành backend.

Mục đích: Hàm hỗ trợ xử lý run integration test trong module backend.

```
FUNCTION runIntegrationTest()
  THỰC HIỆN console.log("🌀 STARTING INTEGRATION TEST...")
  TRY
    THỰC HIỆN console.log("1. Testing Registration...")
    email = `test_${ Date.now() }@example.com`
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ axios.post(`${ API_URL }/auth/register`, {
      fullName: "Test User", email: email, password: "password123" })
    THỰC HIỆN console.log("☑ Registration successful")
    THỰC HIỆN console.log("2. Testing Login...")
    loginRes = GỌI BẮT ĐỒNG BỘ axios.post(`${ API_URL }/auth/login`, { email:
      email, password: "password123" })
    token = loginRes.data.token
    THỰC HIỆN console.log("☑ Login successful")
    THỰC HIỆN console.log("2.3 Fetching real Flashcard for testing...")
    cardRes = GỌI BẮT ĐỒNG BỘ axios.get(`${ API_URL }/user/flashcards`, {
      headers: { Authorization: `Bearer ${ token }` } })
    IF cardRes.data.length = 0 THEN
      THROW ERROR TẠO Error("No flashcards found to test submission")
    END IF
    testQuestionId = cardRes.data[0].questionId
    THỰC HIỆN console.log(`☑ Found Question ID: ${ testQuestionId }`)
    THỰC HIỆN console.log("2.5 Testing Answer Submission...")
    submitRes = GỌI BẮT ĐỒNG BỘ axios.post(`${ API_URL }/user/submit-answer`, {
      questionId: testQuestionId, submittedAnswer: "test answer" }, { headers:
      { Authorization: `Bearer ${ token }` } })
    THỰC HIỆN console.log("☑ Answer submitted successfully")
    THỰC HIỆN console.log("3. Testing Data Retrieval (Auth required)...")
```

```

statsRes = GỌI BẮT ĐỒNG BỘ axios.get(`${ API_URL }/user/stats`, { headers: {
  Authorization: `Bearer ${ token }` } })
THỰC HIỆN console.log(` ✅ Stats fetched (Words Learned: ${
  statsRes.data.totalLearned })`)
THỰC HIỆN console.log("4. Testing Learning Engine Persistence...")
cardRes2 = GỌI BẮT ĐỒNG BỘ axios.get(`${ API_URL }/user/flashcards`, {
  headers: { Authorization: `Bearer ${ token }` } })
THỰC HIỆN console.log(` ✅ Flashcards received (${ cardRes2.data.length }
  cards)`)
THỰC HIỆN console.log("\n🚀 ALL BACKEND TESTS PASSED!")
THỰC HIỆN process.exit(0)
CATCH error
THỰC HIỆN console.error("\n❌ TEST FAILED!")
THỰC HIỆN console.error(" Status: ", error.response.status)
THỰC HIỆN console.error(" Message: ", error.response.data.message)
THỰC HIỆN console.error(" Error Details: ", error.response.data.error)
THỰC HIỆN process.exit(1)
END TRY
END FUNCTION

```

5.2. Middleware

middlewares/auth.js

File này có 6 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 3: normalizePermission(permissionCode)

Dòng nguồn: 3 | Đầu vào: permissionCode | Đầu ra: Quyết định gọi next(), trả lỗi HTTP hoặc tạo middleware kiểm tra request.

Mục đích: Thực thi middleware để chuẩn hóa permission.

```

FUNCTION normalizePermission(permissionCode)
  RETURN String(permissionCode OR "").trim().replace(/-/g, "_").toUpperCase()
END FUNCTION

```

Hàm 4: getUserPermissions(req)

Dòng nguồn: 5 | Đầu vào: req | Đầu ra: Quyết định gọi next(), trả lỗi HTTP hoặc tạo middleware kiểm tra request.

Mục đích: Thực thi middleware để lấy user permissions.

```

FUNCTION getUserPermissions(req)
  RETURN req.user.permissions OR [].map(normalizePermission)
END FUNCTION

```

Hàm 5: verifyToken(req, res, next)

Dòng nguồn: 7 | Đầu vào: req, res, next | Đầu ra: Quyết định gọi next(), trả lỗi HTTP hoặc tạo middleware kiểm tra request.

Mục đích: Kiểm tra Bearer token JWT, giải mã payload và gắn thông tin người dùng vào req.

```

FUNCTION verifyToken(req, res, next)
  authHeader = req.headers.authorization
  IF NOT authHeader OR NOT authHeader.startsWith("Bearer ") THEN
    TRẢ VỀ HTTP 401 CÙNG DỮ LIỆU { message: "Missing or invalid token" }
  END IF
  token = authHeader.split(" ")[1]
  TRY
    req.user = jwt.verify(token, process.env.JWT_SECRET)
    THỰC HIỆN next()
  CATCH error
    TRẢ VỀ HTTP 401 CÙNG DỮ LIỆU { message: "Token is invalid or expired" }
  END TRY
END FUNCTION

```


Hàm 6: checkPermission(permissionCode)

Dòng nguồn: 22 | Đầu vào: *permissionCode* | Đầu ra: Quyết định gọi *next()*, trả lỗi HTTP hoặc tạo middleware kiểm tra request.

Mục đích: Tạo middleware kiểm tra một quyền cụ thể trước khi cho phép truy cập endpoint.

```
FUNCTION checkPermission(permissionCode)
    RETURN HÀM CALLBACK(req, res, next)
END FUNCTION
```

Hàm 7: checkAnyPermission(permissionCodes)

Dòng nguồn: 32 | Đầu vào: *permissionCodes* | Đầu ra: Quyết định gọi *next()*, trả lỗi HTTP hoặc tạo middleware kiểm tra request.

Mục đích: Tạo middleware cho phép truy cập nếu người dùng có ít nhất một quyền trong danh sách.

```
FUNCTION checkAnyPermission(permissionCodes)
    RETURN HÀM CALLBACK(req, res, next)
END FUNCTION
```

Hàm 8: checkOwnership(tableName, idColumn, paramName = 'id', ownerColumn = 'CreatedByUserID')

Dòng nguồn: 46 | Đầu vào: *tableName*, *idColumn*, *paramName = 'id'*, *ownerColumn = 'CreatedByUserID'* | Đầu ra: Quyết định gọi *next()*, trả lỗi HTTP hoặc tạo middleware kiểm tra request.

Mục đích: Kiểm tra quyền sở hữu bản ghi khi người dùng không có đặc quyền quản trị hệ thống.

```
FUNCTION checkOwnership(tableName, idColumn, paramName = 'id', ownerColumn =
    'CreatedByUserID')
    RETURN HÀM CALLBACK(req, res, next)
END FUNCTION
```

middlewares/errorHandler.js

File này có 1 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 9: errorHandler(err, req, res, next)

Dòng nguồn: 2 | Đầu vào: *err*, *req*, *res*, *next* | Đầu ra: Quyết định gọi *next()*, trả lỗi HTTP hoặc tạo middleware kiểm tra request.

Mục đích: Hàm hỗ trợ xử lý error handler trong module backend.

```
FUNCTION errorHandler(err, req, res, next)
    THỰC HIỆN console.error(err.stack)
    TRẢ VỀ HTTP 500 CÙNG DỮ LIỆU { message: err.message OR "Lỗi server nội bộ" }
END FUNCTION
```

middlewares/validate.js

File này có 1 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 10: validate(schema)

Dòng nguồn: 3 | Đầu vào: *schema* | Đầu ra: Quyết định gọi *next()*, trả lỗi HTTP hoặc tạo middleware kiểm tra request.

Mục đích: Hàm hỗ trợ xử lý validate trong module backend.

```
FUNCTION validate(schema)
    RETURN MIDDLEWARE FUNCTION(req, res, next)
    TRY
        parsed = schema.parse({ body: req.body, query: req.query, params:
            req.params })
        IF Object.prototype.hasOwnProperty.call(parsed, "body") THEN
            req.body = parsed.body
        END IF
        IF Object.prototype.hasOwnProperty.call(parsed, "query") THEN
```

```

        req.query = parsed.query
    END IF
    IF Object.prototype.hasOwnProperty.call(parsed, "params") THEN
        req.params = parsed.params
    END IF
    THỰC HIỆN next()
CATCH error
    TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Dữ liệu không hợp lệ",
        errors: error.issues OR error.errors OR [] }
END TRY
END MIDDLEWARE FUNCTION
END FUNCTION

```

5.3. Controller

controllers/admin.controller.js

File này có 53 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 11: async getTopics(req, res, next)

Dòng nguồn: 7 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách chủ đề.

```

FUNCTION getTopics(req, res, next)
    TRY
        page = req.query.page
        IF page không tồn tại hoặc không chuyển được sang số nguyên THEN
            page = 1
        ELSE
            page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10)
        END IF
        limit = req.query.limit
        IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN
            limit = 50
        ELSE
            limit = CHUYỂN SANG SỐ NGUYÊN(limit, cơ số 10)
        END IF
        topics = GỌI BẮT ĐỒNG BỘ AdminService.getTopics(page, limit, { search:
            req.query.search, status: req.query.status, categoryId:
            req.query.categoryId })
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU topics
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 12: async getTopicCategories(req, res, next)

Dòng nguồn: 22 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh mục chủ đề.

```

FUNCTION getTopicCategories(req, res, next)
    TRY
        categories = GỌI BẮT ĐỒNG BỘ AdminService.getTopicCategories()
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU categories
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 13: async createTopic(req, res, next)

Dòng nguồn: 31 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới chủ đề.

```
FUNCTION createTopic(req, res, next)
  TRY
    adminId = req.user.id
    result = GỌI BẮT ĐỒNG BỘ AdminService.createTopic(req.body, adminId)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo chủ đề thành công", data:
      result }
  CATCH error
    IF error.message = "Invalid topic data" OR error.message = "Topic already
      exists" OR error.message = "Topic code already exists" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 14: async updateTopic(req, res, next)

Dòng nguồn: 48 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật chủ đề.

```
FUNCTION updateTopic(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.updateTopic(req.params.id, req.body,
      req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy chủ đề" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật chủ đề thành công" }
  CATCH error
    IF ["Invalid topic data", "Topic already exists", "Topic code already
      exists"].includes(error.message) THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 15: async deleteTopic(req, res, next)

Dòng nguồn: 63 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa chủ đề.

```
FUNCTION deleteTopic(req, res, next)
  TRY
    result = GỌI BẮT ĐỒNG BỘ AdminService.deleteTopic(req.params.id,
      req.user.id)
    IF NOT result.success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy chủ đề" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: NẾU result.archived THÌ "Đã lưu trữ
      chủ đề vì đang có nội dung liên quan" NGƯỢC LẠI "Xóa chủ đề thành công",
      archived: result.archived }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 16: async createTopicCategory(req, res, next)

Dòng nguồn: 78 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới danh mục chủ đề.

```
FUNCTION createTopicCategory(req, res, next)
  TRY
    result = GỌI BẮT ĐỒNG BỘ AdminService.createTopicCategory(req.body,
      req.user.id)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo danh mục chủ đề thành công",
      data: result }
  CATCH error
    IF ["Invalid topic category data", "Topic category code already
      exists"].includes(error.message) THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 17: async updateTopicCategory(req, res, next)

Dòng nguồn: 90 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật danh mục chủ đề.

```
FUNCTION updateTopicCategory(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.updateTopicCategory(req.params.id,
      req.body, req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy danh mục chủ
        đề" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật danh mục chủ đề thành
      công" }
  CATCH error
    IF ["Invalid topic category data", "Topic category code already
      exists"].includes(error.message) THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 18: async deleteTopicCategory(req, res, next)

Dòng nguồn: 105 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa danh mục chủ đề.

```
FUNCTION deleteTopicCategory(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.deleteTopicCategory(req.params.id,
      req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy danh mục chủ
        đề" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã tắt danh mục chủ đề" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 19: async getWords(req, res, next)

Dòng nguồn: 118 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách từ vựng.

```
FUNCTION getWords(req, res, next)
  TRY
    page = req.query.page
    IF page không tồn tại hoặc không chuyển được sang số nguyên THEN
      page = 1
    ELSE
      page = CHUYỂN SANG SỐ NGUYÊN(page)
    END IF
    limit = req.query.limit
    IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN
      limit = 20
    ELSE
      limit = CHUYỂN SANG SỐ NGUYÊN(limit)
    END IF
    words = GỌI BẮT ĐỒNG BỘ AdminService.getWords(page, limit, { topicId:
      req.query.topicId, partOfSpeechId: req.query.partOfSpeechId, status:
      req.query.status, missingExamples: req.query.missingExamples,
      missingQuestions: req.query.missingQuestions, sortBy: req.query.sortBy,
      sortDirection: req.query.sortDirection, search: req.query.search })
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU words
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 20: async createWord(req, res, next)

Dòng nguồn: 138 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới từ vựng.

```
FUNCTION createWord(req, res, next)
  TRY
    adminId = req.user.id
    wordData = req.body
    result = GỌI BẮT ĐỒNG BỘ AdminService.createWord(wordData, adminId)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo từ vựng thành công", data:
      result }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 21: async getWordDetail(req, res, next)

Dòng nguồn: 149 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy chi tiết từ vựng.

```
FUNCTION getWordDetail(req, res, next)
  TRY
    word = GỌI BẮT ĐỒNG BỘ AdminService.getWordDetail(req.params.id)
    IF NOT word THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy từ vựng" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU word
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 22: async bulkImportWords(req, res, next)

Dòng nguồn: 161 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để nhập hàng loạt danh sách từ vựng.

```
FUNCTION bulkImportWords(req, res, next)
  TRY
    adminId = req.user.id
    result = GỌI BẮT ĐỒNG BỘ AdminService.bulkInsertWords(req.body, adminId)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Import từ vựng hoàn tất", ...result }
  CATCH error
    IF error.message = "Invalid import payload" OR error.message.startsWith("CSV
      must include") THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 23: async previewWordImport(req, res, next)

Dòng nguồn: 180 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xem trước dữ liệu import từ vựng.

```
FUNCTION previewWordImport(req, res, next)
  TRY
    result = GỌI BẮT ĐỒNG BỘ AdminService.previewWordImport(req.body)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU result
  CATCH error
    IF error.message = "Invalid import payload" OR error.message.startsWith("CSV
      must include") THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 24: async updateWord(req, res, next)

Dòng nguồn: 195 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật từ vựng.

```
FUNCTION updateWord(req, res, next)
  TRY
    { id } = req.params
    wordData = req.body
    success = GỌI BẮT ĐỒNG BỘ AdminService.updateWord(id, wordData, req.user.id)
    IF success THEN
      TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật thành công" }
    ELSE
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy từ vựng" }
    END IF
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 25: async deleteWord(req, res, next)

Dòng nguồn: 210 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa từ vựng.

```
FUNCTION deleteWord(req, res, next)
  TRY
    { id } = req.params
    success = GỌI BẮT ĐỒNG BỘ AdminService.archiveWord(id, req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy từ vựng" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa từ vựng thành công" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 26: async hardDeleteWord(req, res, next)

Dòng nguồn: 223 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Hàm hỗ trợ xử lý hard delete word trong module backend.

```
FUNCTION hardDeleteWord(req, res, next)
  TRY
    { id } = req.params
    success = GỌI BẮT ĐỒNG BỘ AdminService.deleteWord(id, req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy từ vựng" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa vĩnh viễn từ vựng thành công" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 27: async getQuestionsByWord(req, res, next)

Dòng nguồn: 237 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy các câu hỏi của một từ vựng.

```
FUNCTION getQuestionsByWord(req, res, next)
  TRY
    { wordId } = req.params
    page = req.query.page
    IF page không tồn tại hoặc không chuyển được sang số nguyên THEN
      page = 1
    ELSE
      page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10)
    END IF
    limit = req.query.limit
    IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN
      limit = 20
    ELSE
      limit = CHUYỂN SANG SỐ NGUYÊN(limit, cơ số 10)
    END IF
    questions = GỌI BẮT ĐỒNG BỘ AdminService.getQuestionsByWord(wordId, page,
      limit, { search: req.query.search, type: req.query.type, status:
        req.query.status })
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU questions
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 28: async createQuestion(req, res, next)

Dòng nguồn: 253 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới câu hỏi.

```
FUNCTION createQuestion(req, res, next)
  TRY
    adminId = req.user.id
    result = GỌI BẮT ĐỒNG BỘ AdminService.createQuestion(req.body, adminId)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo câu hỏi thành công", data:
      result }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 29: async updateQuestion(req, res, next)

Dòng nguồn: 263 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật câu hỏi.

```
FUNCTION updateQuestion(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.updateQuestion(req.params.id,
      req.body, req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy câu hỏi" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật câu hỏi thành công" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 30: async deleteQuestion(req, res, next)

Dòng nguồn: 275 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa câu hỏi.

```
FUNCTION deleteQuestion(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.deleteQuestion(req.params.id,
      req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy câu hỏi" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa câu hỏi thành công" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 31: async getOverviewStats(req, res, next)

Dòng nguồn: 287 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy thống kê tổng quan.

```
FUNCTION getOverviewStats(req, res, next)
  TRY
    stats = GỌI BẮT ĐỒNG BỘ AdminService.getOverviewStats()
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU stats
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```



```
END TRY
END FUNCTION
```

Hàm 32: async getWeeklyActivity(req, res, next)

Dòng nguồn: 296 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy hoạt động theo tuần.

```
FUNCTION getWeeklyActivity(req, res, next)
  TRY
    activity = GỌI BẮT ĐỒNG BỘ AdminService.getWeeklyActivity()
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU activity
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 33: async getTodayActivity(req, res, next)

Dòng nguồn: 305 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy hoạt động trong ngày.

```
FUNCTION getTodayActivity(req, res, next)
  TRY
    activity = GỌI BẮT ĐỒNG BỘ AdminService.getTodayActivity()
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU activity
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 34: async getRecentUsers(req, res, next)

Dòng nguồn: 314 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách người dùng gần đây.

```
FUNCTION getRecentUsers(req, res, next)
  TRY
    users = GỌI BẮT ĐỒNG BỘ AdminService.getRecentUsers()
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU users
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 35: async getTopCourses(req, res, next)

Dòng nguồn: 323 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách chủ đề/bộ học nổi bật.

```
FUNCTION getTopCourses(req, res, next)
  TRY
    courses = GỌI BẮT ĐỒNG BỘ AdminService.getTopCourses()
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU courses
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 36: async bulkImportQuestions(req, res, next)

Dòng nguồn: 332 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để nhập hàng loạt danh sách câu hỏi.

```
FUNCTION bulkImportQuestions(req, res, next)
  TRY
    adminId = req.user.id
    result = GỌI BẮT ĐỒNG BỘ AdminService.bulkInsertQuestions(req.body, adminId)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Import câu hỏi hoàn tất", ...result }
  CATCH error
    IF error.message = "Invalid import payload" OR error.message.startsWith("CSV must include") THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 37: async getMiniTests(req, res, next)

Dòng nguồn: 349 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách bài kiểm tra ngắn.

```
FUNCTION getMiniTests(req, res, next)
  TRY
    page = req.query.page
    IF page không tồn tại hoặc không chuyển được sang số nguyên THEN
      page = 1
    ELSE
      page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10)
    END IF
    limit = req.query.limit
    IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN
      limit = 20
    ELSE
      limit = CHUYỂN SANG SỐ NGUYÊN(limit, cơ số 10)
    END IF
    tests = GỌI BẮT ĐỒNG BỘ AdminService.getMiniTests(page, limit, { search: req.query.search, topicId: req.query.topicId, status: req.query.status })
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU tests
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 38: async createMiniTest(req, res, next)

Dòng nguồn: 364 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới bài kiểm tra ngắn.

```
FUNCTION createMiniTest(req, res, next)
  TRY
    adminId = req.user.id
    result = GỌI BẮT ĐỒNG BỘ AdminService.createMiniTest(req.body, adminId)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo Mini Test thành công", data: result }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

```
END TRY
END FUNCTION
```

Hàm 39: async updateMiniTest(req, res, next)

Dòng nguồn: 374 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật bài kiểm tra ngắn.

```
FUNCTION updateMiniTest(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.updateMiniTest(req.params.id,
      req.body, req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy mini test" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật mini test thành công" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 40: async deleteMiniTest(req, res, next)

Dòng nguồn: 386 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa bài kiểm tra ngắn.

```
FUNCTION deleteMiniTest(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.deleteMiniTest(req.params.id,
      req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy mini test" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa mini test thành công" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 41: async publishMiniTest(req, res, next)

Dòng nguồn: 398 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xuất bản bài kiểm tra ngắn.

```
FUNCTION publishMiniTest(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.setMiniTestStatus(req.params.id,
      "Published", req.user.id, "Published from mini test manager")
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy mini test" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xuất bản mini test thành công" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 42: async archiveMiniTest(req, res, next)

Dòng nguồn: 410 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lưu trữ bài kiểm tra ngắn.

```
FUNCTION archiveMiniTest(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.setMiniTestStatus(req.params.id,
      "Archived", req.user.id, "Archived from mini test manager")
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy mini test" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Lưu trữ mini test thành công" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 43: async getStats(req, res, next)

Dòng nguồn: 422 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy thống kê.

```
FUNCTION getStats(req, res, next)
  TRY
    stats = GỌI BẮT ĐỒNG BỘ AdminService.getDashboardStats()
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU stats
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 44: async getStudents(req, res, next)

Dòng nguồn: 431 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách người dùng/học viên.

```
FUNCTION getStudents(req, res, next)
  TRY
    page = req.query.page
    IF page không tồn tại hoặc không chuyển được sang số nguyên THEN
      page = 1
    ELSE
      page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10)
    END IF
    limit = req.query.limit
    IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN
      limit = 20
    ELSE
      limit = CHUYỂN SANG SỐ NGUYÊN(limit, cơ số 10)
    END IF
    students = GỌI BẮT ĐỒNG BỘ AdminService.getStudents(page, limit, { search:
      req.query.search, status: req.query.status, role: req.query.role })
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU students
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 45: async createUser(req, res, next)

Dòng nguồn: 446 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới user.

```
FUNCTION createUser(req, res, next)
  TRY
    result = GỌI BẮT ĐỒNG BỘ AdminService.createUser(req.body)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo user thành công", data: result }
  CATCH error
    IF error.message = "Email already exists" OR error.message = "Invalid role"
      OR error.message = "Invalid user data" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 46: async updateUser(req, res, next)

Dòng nguồn: 458 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật user.

```
FUNCTION updateUser(req, res, next)
  TRY
    { id } = req.params
    success = GỌI BẮT ĐỒNG BỘ AdminService.updateUser(id, req.body)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy user" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật user thành công" }
  CATCH error
    IF error.message = "Email already exists" OR error.message = "Invalid role"
      OR error.message = "Invalid user data" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 47: async deleteUser(req, res, next)

Dòng nguồn: 476 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa user.

```
FUNCTION deleteUser(req, res, next)
  TRY
    { id } = req.params
    success = GỌI BẮT ĐỒNG BỘ AdminService.deleteUser(id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy user" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa user thành công" }
  CATCH error
    IF error.message = "User owns content" THEN
      TRẢ VỀ HTTP 409 CÙNG DỮ LIỆU { message: "Không thể xóa user đã tạo
        nội dung. Hãy khóa tài khoản thay vì xóa." }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 48: async toggleStudentStatus(req, res, next)

Dòng nguồn: 496 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để bật/tắt trạng thái tài khoản học viên.

```
FUNCTION toggleStudentStatus(req, res, next)
  TRY
    { id } = req.params
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ AdminService.toggleUserStatus(id)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật trạng thái thành công" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 49: async updateUserRole(req, res, next)

Dòng nguồn: 506 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật vai trò người dùng.

```
FUNCTION updateUserRole(req, res, next)
  TRY
    { id } = req.params
    { role } = req.body
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ AdminService.updateUserRole(id, role)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật vai trò thành công" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 50: async getAnalytics(req, res, next)

Dòng nguồn: 517 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy dữ liệu phân tích.

```
FUNCTION getAnalytics(req, res, next)
  TRY
    data = GỌI BẮT ĐỒNG BỘ AdminService.getAnalyticsData()
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 51: async getContentManagement(req, res, next)

Dòng nguồn: 526 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy content management.

```
FUNCTION getContentManagement(req, res, next)
  TRY
    data = GỌI BẮT ĐỒNG BỘ AdminService.getContentManagementData()
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 52: async updateContentStatus(req, res, next)

Dòng nguồn: 535 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật trạng thái nội dung.

```
FUNCTION updateContentStatus(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.updateContentStatus(req.body,
      req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy nội dung" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật trạng thái nội dung thành
      công" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 53: async getPendingContent(req, res, next)

Dòng nguồn: 547 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách nội dung chờ duyệt.

```
FUNCTION getPendingContent(req, res, next)
  TRY
    data = GỌI BẮT ĐỒNG BỘ AdminService.getPendingContent()
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 54: async approveContent(req, res, next)

Dòng nguồn: 556 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để phê duyệt content.

```
FUNCTION approveContent(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.updateContentStatus({ entityType:
      req.params.entityType, entityId: req.params.entityId, status:
      "Published", comment: req.body.comment OR "Approved from content review"
    }, req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy nội dung" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Duyệt nội dung thành công" }
  CATCH error
    IF error.message = "Invalid entity type" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 55: async rejectContent(req, res, next)

Dòng nguồn: 574 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để từ chối content.

```
FUNCTION rejectContent(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.updateContentStatus({ entityType:
      req.params.entityType, entityId: req.params.entityId, status:
      "Rejected", comment: req.body.reason OR "Rejected from content review"
    }, req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy nội dung" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Từ chối nội dung thành công" }
  CATCH error
    IF error.message = "Invalid entity type" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 56: async archiveContent(req, res, next)

Dòng nguồn: 592 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lưu trữ content.

```
FUNCTION archiveContent(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ AdminService.updateContentStatus({ entityType:
      req.params.entityType, entityId: req.params.entityId, status:
      "Archived", comment: req.body.comment OR "Archived from content review"
    }, req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy nội dung" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Lưu trữ nội dung thành công" }
  CATCH error
    IF error.message = "Invalid entity type" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 57: async getContentReviewLogs(req, res, next)

Dòng nguồn: 610 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy content review logs.

```
FUNCTION getContentReviewLogs(req, res, next)
  TRY
    data = GỌI BẮT ĐỒNG BỘ
      AdminService.getContentReviewLogs(req.params.entityType,
      req.params.entityId)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH error
    IF error.message = "Invalid entity type" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```


Hàm 58: async getNotifications(req, res, next)

Dòng nguồn: 622 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách thông báo.

```
FUNCTION getNotifications(req, res, next)
  TRY
    page = req.query.page
    IF page không tồn tại hoặc không chuyển được sang số nguyên THEN
      page = 1
    ELSE
      page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10)
    END IF
    limit = req.query.limit
    IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN
      limit = 50
    ELSE
      limit = CHUYỂN SANG SỐ NGUYÊN(limit, cơ số 10)
    END IF
    data = GỌI BẮT ĐỒNG BỘ AdminService.getNotifications(page, limit, { search:
      req.query.search, type: req.query.type, deliveryChannel:
      req.query.deliveryChannel, isRead: req.query.isRead })
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 59: async getAuditLogs(req, res, next)

Dòng nguồn: 638 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy nhật ký audit quản trị.

```
FUNCTION getAuditLogs(req, res, next)
  TRY
    page = req.query.page
    IF page không tồn tại hoặc không chuyển được sang số nguyên THEN
      page = 1
    ELSE
      page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10)
    END IF
    limit = req.query.limit
    IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN
      limit = 50
    ELSE
      limit = CHUYỂN SANG SỐ NGUYÊN(limit, cơ số 10)
    END IF
    data = GỌI BẮT ĐỒNG BỘ AdminService.getAuditLogs(page, limit, { search:
      req.query.search, action: req.query.action, entityType:
      req.query.entityType, adminId: req.query.adminId })
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 60: async getReports(req, res, next)

Dòng nguồn: 654 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách báo lỗi/nội dung.

```
FUNCTION getReports(req, res, next)
  TRY
    page = req.query.page
    IF page không tồn tại hoặc không chuyển được sang số nguyên THEN
      page = 1
    ELSE
      page = CHUYỂN SANG SỐ NGUYÊN(page, cơ số 10)
    END IF
    limit = req.query.limit
    IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN
      limit = 20
    ELSE
      limit = CHUYỂN SANG SỐ NGUYÊN(limit, cơ số 10)
    END IF
    data = GỌI BẮT ĐỒNG BỘ ReportService.getReports(page, limit, { search:
      req.query.search, status: req.query.status, reportType:
      req.query.reportType, entityType: req.query.entityType, priority:
      req.query.priority })
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 61: async updateReport(req, res, next)

Dòng nguồn: 671 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật báo lỗi/nội dung.

```
FUNCTION updateReport(req, res, next)
  TRY
    success = GỌI BẮT ĐỒNG BỘ ReportService.updateReport(req.params.id,
      req.body, req.user.id)
    IF NOT success THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Report not found" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Report updated" }
  CATCH error
    IF error.message.startsWith("Invalid ") THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 62: async sendAnnouncement(req, res, next)

Dòng nguồn: 686 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Hàm hỗ trợ xử lý send announcement trong module backend.

```
FUNCTION sendAnnouncement(req, res, next)
  TRY
    result = GỌI BẮT ĐỒNG BỘ AdminService.sendAnnouncement(req.body)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Announcement queued", data: result
    }
  CATCH error
    IF error.message = "Missing title or message" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
  END TRY
END FUNCTION
```

```

END IF
CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
END TRY
END FUNCTION

```

Hàm 63: async createDailyReminders(req, res, next)

Dòng nguồn: 698 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới daily reminders.

```

FUNCTION createDailyReminders(req, res, next)
  TRY
    result = GỌI BẮT ĐỒNG BỘ AdminService.createDailyReminders()
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Daily reminders queued", data:
      result }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION

```

controllers/ai.controller.js

File này có 1 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 64: async suggestWordContent(req, res, next)

Dòng nguồn: 4 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Hàm hỗ trợ xử lý suggest word content trong module backend.

```

FUNCTION suggestWordContent(req, res, next)
  TRY
    suggestion = GỌI BẮT ĐỒNG BỘ AiService.suggestWordContent(req.body)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU suggestion
  CATCH error
    IF error.statusCode THEN
      TRẢ VỀ HTTP error.statusCode CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION

```

controllers/auth.controller.js

File này có 2 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 65: async register(req, res, next)

Dòng nguồn: 5 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để đăng ký register.

```

FUNCTION register(req, res, next)
  TRY
    { fullName, email, password } = req.body
    IF NOT fullName OR NOT email OR NOT password THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Vui lòng cung cấp đủ thông
        tin" }
    END IF
    user = GỌI BẮT ĐỒNG BỘ AuthService.register(fullName, email, password)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Đăng ký thành công", user: user }
  CATCH error
    IF error.message = "Email đã tồn tại" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION

```

```
END TRY
END FUNCTION
```

Hàm 66: async login(req, res, next)

Dòng nguồn: 24 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xác thực login.

```
FUNCTION login(req, res, next)
  TRY
    { email, password } = req.body
    IF NOT email OR NOT password THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Vui lòng cung cấp email và mật khẩu" }
    END IF
    data = GỌI BẮT ĐỒNG BỘ AuthService.login(email, password)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đăng nhập thành công", ...data }
  CATCH error
    IF error.message = "Email hoặc mật khẩu không chính xác" THEN
      TRẢ VỀ HTTP 401 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

controllers/categories.controller.js

File này có 2 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 67: async getPartOfSpeeches(req, res, next)

Dòng nguồn: 5 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh mục từ loại.

```
FUNCTION getPartOfSpeeches(req, res, next)
  TRY
    pos = GỌI BẮT ĐỒNG BỘ CategoriesService.getPartOfSpeeches()
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU pos
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 68: async getTopics(req, res, next)

Dòng nguồn: 14 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách chủ đề.

```
FUNCTION getTopics(req, res, next)
  TRY
    topics = GỌI BẮT ĐỒNG BỘ CategoriesService.getTopics(req.user.id)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU topics
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

controllers/creator.controller.js

File này có 31 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 69: getCreatorErrorMessage(error)

Dòng nguồn: 20 | Đầu vào: error | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy creator error message.

```
FUNCTION getCreatorErrorMessage(error)
    RETURN CREATOR_ERROR_MESSAGES[error.message] OR error.message
END FUNCTION
```

Hàm 70: async getDashboard(req, res, next)

Dòng nguồn: 24 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy dashboard.

```
FUNCTION getDashboard(req, res, next)
    TRY
        stats = GỌI BẮT ĐỒNG BỘ CreatorService.getDashboardStats(req.user.id)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU stats
    CATCH err
        THỰC HIỆN next(err)
    END TRY
END FUNCTION
```

Hàm 71: async getContentSummary(req, res, next)

Dòng nguồn: 31 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy tổng hợp trạng thái nội dung.

```
FUNCTION getContentSummary(req, res, next)
    TRY
        data = GỌI BẮT ĐỒNG BỘ CreatorService.getContentSummary(req.user.id)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
    CATCH err
        THỰC HIỆN next(err)
    END TRY
END FUNCTION
```

Hàm 72: async getTopicAnalytics(req, res, next)

Dòng nguồn: 38 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy thống kê theo chủ đề.

```
FUNCTION getTopicAnalytics(req, res, next)
    TRY
        data = GỌI BẮT ĐỒNG BỘ CreatorService.getTopicAnalytics(req.user.id,
            req.params.id)
        IF NOT data THEN
            TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy hoặc không
                có quyền" }
        END IF
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
    CATCH err
        THỰC HIỆN next(err)
    END TRY
END FUNCTION
```

Hàm 73: async getMiniTestAnalytics(req, res, next)

Dòng nguồn: 46 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy thống kê bài kiểm tra ngắn.

```
FUNCTION getMiniTestAnalytics(req, res, next)
  TRY
    data = GỌI BẮT ĐỒNG BỘ CreatorService.getMiniTestAnalytics(req.user.id,
      req.params.id)
    IF NOT data THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy hoặc không
        có quyền" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH err
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 74: async getTopicCategories(req, res, next)

Dòng nguồn: 55 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh mục chủ đề.

```
FUNCTION getTopicCategories(req, res, next)
  TRY
    data = GỌI BẮT ĐỒNG BỘ CreatorService.getTopicCategories()
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH err
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 75: async getTopics(req, res, next)

Dòng nguồn: 63 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách chủ đề.

```
FUNCTION getTopics(req, res, next)
  TRY
    data = GỌI BẮT ĐỒNG BỘ CreatorService.getMyTopics(req.user.id, req.query)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH err
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 76: async createTopic(req, res, next)

Dòng nguồn: 70 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới chủ đề.

```
FUNCTION createTopic(req, res, next)
  TRY
    result = GỌI BẮT ĐỒNG BỘ CreatorService.createTopic(req.body, req.user.id)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo chủ đề thành công", data:
      result }
  CATCH err
    IF ["Topic name or code already exists", "Topic category not
      found"].includes(err.message) THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: getCreatorErrorMessage(err)
    }
    END IF
  END TRY
END FUNCTION
```

```

        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```

Hàm 77: async updateTopic(req, res, next)

Dòng nguồn: 82 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật chủ đề.

```

FUNCTION updateTopic(req, res, next)
    TRY
        ok = GỌI BẮT ĐỒNG BỘ CreatorService.updateTopic(req.params.id, req.body,
            req.user.id)
        IF NOT ok THEN
            TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy hoặc không
                có quyền" }
        END IF
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật thành công" }
    CATCH err
        IF ["Topic name or code already exists", "Topic category not
            found"].includes(err.message) THEN
            TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: getCreatorErrorMessage(err)
                }
        END IF
        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```

Hàm 78: async deleteTopic(req, res, next)

Dòng nguồn: 95 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa chủ đề.

```

FUNCTION deleteTopic(req, res, next)
    TRY
        ok = GỌI BẮT ĐỒNG BỘ CreatorService.deleteTopic(req.params.id, req.user.id)
        IF NOT ok THEN
            TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Chỉ xóa được bản nháp do
                bạn tạo" }
        END IF
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa thành công" }
    CATCH err
        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```

Hàm 79: async submitTopicForReview(req, res, next)

Dòng nguồn: 103 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gửi topic for review.

```

FUNCTION submitTopicForReview(req, res, next)
    TRY
        ok = GỌI BẮT ĐỒNG BỘ CreatorService.submitForReview("Topics", "TopicID",
            req.params.id, req.user.id)
        IF NOT ok THEN
            TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Không thể gửi duyệt (sai
                trạng thái hoặc không có quyền)" }
        END IF
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã gửi duyệt" }
    CATCH err
        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```

Hàm 80: async getWords(req, res, next)

Dòng nguồn: 112 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách từ vựng.

```
FUNCTION getWords(req, res, next)
  TRY
    data = GỌI BẮT ĐỒNG BỘ CreatorService.getMyWords(req.user.id, req.query)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH err
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 81: async createWord(req, res, next)

Dòng nguồn: 119 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới từ vựng.

```
FUNCTION createWord(req, res, next)
  TRY
    result = GỌI BẮT ĐỒNG BỘ CreatorService.createWord(req.body, req.user.id)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo từ vựng thành công", data:
      result }
  CATCH err
    IF ["Invalid part of speech", "Word already exists for this part of speech",
      "Topic not found or unavailable"].includes(err.message) THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: getCreatorErrorMessage(err)
    }
    END IF
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 82: async updateWord(req, res, next)

Dòng nguồn: 135 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật từ vựng.

```
FUNCTION updateWord(req, res, next)
  TRY
    ok = GỌI BẮT ĐỒNG BỘ CreatorService.updateWord(req.params.id, req.body,
      req.user.id)
    IF NOT ok THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy hoặc không
        có quyền" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật thành công" }
  CATCH err
    IF ["Invalid part of speech", "Word already exists for this part of speech",
      "Topic not found or unavailable"].includes(err.message) THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: getCreatorErrorMessage(err)
    }
    END IF
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```


Hàm 83: async deleteWord(req, res, next)

Dòng nguồn: 152 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa từ vựng.

```
FUNCTION deleteWord(req, res, next)
  TRY
    ok = GỌI BẮT ĐỒNG BỘ CreatorService.deleteWord(req.params.id, req.user.id)
    IF NOT ok THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Chỉ xóa được bản nháp do
        bạn tạo" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa thành công" }
  CATCH err
    IF err.message = "Word is linked to questions" THEN
      TRẢ VỀ HTTP 409 CÙNG DỮ LIỆU { message: "Từ vựng đang được dùng bởi
        câu hỏi.Hãy xử lý câu hỏi liên quan trước." }
    END IF
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 84: async submitWordForReview(req, res, next)

Dòng nguồn: 165 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gửi word for review.

```
FUNCTION submitWordForReview(req, res, next)
  TRY
    ok = GỌI BẮT ĐỒNG BỘ CreatorService.submitWordForReview(req.params.id,
      req.user.id)
    IF NOT ok THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Không thể gửi duyệt" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã gửi duyệt" }
  CATCH err
    IF err.message = "Word topics must be published before review" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Các chủ đề gắn với từ vựng
        phải được xuất bản trước khi gửi duyệt." }
    END IF
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 85: async getQuestions(req, res, next)

Dòng nguồn: 179 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách câu hỏi.

```
FUNCTION getQuestions(req, res, next)
  TRY
    data = GỌI BẮT ĐỒNG BỘ CreatorService.getMyQuestions(req.user.id, req.query)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH err
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 86: async createQuestion(req, res, next)

Dòng nguồn: 186 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới câu hỏi.

```
FUNCTION createQuestion(req, res, next)
  TRY
    result = GỌI BẮT ĐỒNG BỘ CreatorService.createQuestion(req.body,
      req.user.id)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo câu hỏi thành công", data:
      result }
  CATCH err
    IF err instanceof SyntaxError OR ["Word not found or unavailable", "Question
      options must be an array", "MCQ requires at least two options", "Correct
      answer must match an MCQ option"].includes(err.message) THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: NẾU err instanceof
        SyntaxError THÌ "Options JSON không hợp lệ" NGƯỢC LẠI
        getCreatorErrorMessage(err) }
    END IF
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 87: async updateQuestion(req, res, next)

Dòng nguồn: 203 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật câu hỏi.

```
FUNCTION updateQuestion(req, res, next)
  TRY
    ok = GỌI BẮT ĐỒNG BỘ CreatorService.updateQuestion(req.params.id, req.body,
      req.user.id)
    IF NOT ok THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy hoặc không
        có quyền" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật thành công" }
  CATCH err
    IF err instanceof SyntaxError OR ["Word not found or unavailable", "Question
      options must be an array", "MCQ requires at least two options", "Correct
      answer must match an MCQ option"].includes(err.message) THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: NẾU err instanceof
        SyntaxError THÌ "Options JSON không hợp lệ" NGƯỢC LẠI
        getCreatorErrorMessage(err) }
    END IF
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 88: async deleteQuestion(req, res, next)

Dòng nguồn: 221 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa câu hỏi.

```
FUNCTION deleteQuestion(req, res, next)
  TRY
    ok = GỌI BẮT ĐỒNG BỘ CreatorService.deleteQuestion(req.params.id,
      req.user.id)
    IF NOT ok THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Chỉ xóa được bản nháp do
        bạn tạo" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa thành công" }
  CATCH err
    IF err.message = "Question is used in mini tests" THEN
      TRẢ VỀ HTTP 409 CÙNG DỮ LIỆU { message: "Câu hỏi đang được dùng
```

```

        trong mini test.Hãy gỡ câu hỏi khỏi bài test trước." }
    END IF
    THỰC HIỆN next(err)
END TRY
END FUNCTION

```

Hàm 89: async submitQuestionForReview(req, res, next)

Dòng nguồn: 234 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gửi question for review.

```

FUNCTION submitQuestionForReview(req, res, next)
    TRY
        ok = GỌI BẮT ĐỒNG BỘ CreatorService.submitQuestionForReview(req.params.id,
            req.user.id)
        IF NOT ok THEN
            TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Không thể gửi duyệt" }
        END IF
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã gửi duyệt" }
    CATCH err
        IF err.message = "Question word must be published before review" THEN
            TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Từ vựng của câu hỏi phải
                được xuất bản trước khi gửi duyệt." }
        END IF
        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```

Hàm 90: async getMiniTests(req, res, next)

Dòng nguồn: 248 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách bài kiểm tra ngắn.

```

FUNCTION getMiniTests(req, res, next)
    TRY
        data = GỌI BẮT ĐỒNG BỘ CreatorService.getMyMiniTests(req.user.id, req.query)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
    CATCH err
        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```

Hàm 91: async createMiniTest(req, res, next)

Dòng nguồn: 255 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới bài kiểm tra ngắn.

```

FUNCTION createMiniTest(req, res, next)
    TRY
        result = GỌI BẮT ĐỒNG BỘ CreatorService.createMiniTest(req.body,
            req.user.id)
        TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Tạo bài test thành công", data:
            result }
    CATCH err
        IF ["Mini test contains duplicate questions", "Mini test requires at least
            one question", "Topic not found or unavailable", "Mini test questions
            must be owned and published"].includes(err.message) THEN
            TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: getCreatorErrorMessage(err)
            }
        END IF
        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```

Hàm 92: async updateMiniTest(req, res, next)

Dòng nguồn: 272 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật bài kiểm tra ngắn.

```
FUNCTION updateMiniTest(req, res, next)
  TRY
    ok = GỌI BẮT ĐỒNG BỘ CreatorService.updateMiniTest(req.params.id, req.body,
      req.user.id)
    IF NOT ok THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy hoặc không
        có quyền" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật thành công" }
  CATCH err
    IF ["Mini test contains duplicate questions", "Mini test requires at least
      one question", "Topic not found or unavailable", "Mini test questions
      must be owned and published"].includes(err.message) THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: getCreatorErrorMessage(err)
    }
    END IF
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 93: async deleteMiniTest(req, res, next)

Dòng nguồn: 290 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa bài kiểm tra ngắn.

```
FUNCTION deleteMiniTest(req, res, next)
  TRY
    ok = GỌI BẮT ĐỒNG BỘ CreatorService.deleteMiniTest(req.params.id,
      req.user.id)
    IF NOT ok THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Chỉ xóa được bản nháp do
        bạn tạo" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa thành công" }
  CATCH err
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 94: async addMiniTestItem(req, res, next)

Dòng nguồn: 298 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để thêm một phần tử câu hỏi của bài kiểm tra.

```
FUNCTION addMiniTestItem(req, res, next)
  TRY
    ok = GỌI BẮT ĐỒNG BỘ CreatorService.addMiniTestItem(req.params.id,
      req.body.questionId, req.user.id)
    IF NOT ok THEN
      TRẢ VỀ HTTP 403 CÙNG DỮ LIỆU { message: "Không có quyền" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Thêm câu hỏi thành công" }
  CATCH err
    IF ["Mini test questions must be owned and published", "Question is already
      in this mini test"].includes(err.message) THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: getCreatorErrorMessage(err)
    }
    END IF
  END TRY
END FUNCTION
```

```

        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```

Hàm 95: async removeMiniTestItem(req, res, next)

Dòng nguồn: 314 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gỡ bỏ một phần tử câu hỏi của bài kiểm tra.

```

FUNCTION removeMiniTestItem(req, res, next)
    TRY
        ok = GỌI BẮT ĐỒNG BỘ CreatorService.removeMiniTestItem(req.params.id,
            req.params.questionId, req.user.id)
        IF NOT ok THEN
            TRẢ VỀ HTTP 403 CÙNG DỮ LIỆU { message: "Không có quyền" }
        END IF
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Xóa câu hỏi khỏi test thành công" }
    CATCH err
        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```

Hàm 96: async submitMiniTestForReview(req, res, next)

Dòng nguồn: 322 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gửi mini test for review.

```

FUNCTION submitMiniTestForReview(req, res, next)
    TRY
        ok = GỌI BẮT ĐỒNG BỘ CreatorService.submitMiniTestForReview(req.params.id,
            req.user.id)
        IF NOT ok THEN
            TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Không thể gửi duyệt" }
        END IF
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã gửi duyệt" }
    CATCH err
        IF err.message.includes("Published") OR err.message.includes("ít nhất một
            câu hỏi") THEN
            TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: err.message }
        END IF
        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```

Hàm 97: async getMedia(req, res, next)

Dòng nguồn: 338 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách media metadata.

```

FUNCTION getMedia(req, res, next)
    TRY
        data = GỌI BẮT ĐỒNG BỘ CreatorService.getMedia(req.user.id)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
    CATCH err
        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```

Hàm 98: async createMedia(req, res, next)

Dòng nguồn: 345 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới danh sách media metadata.

```
FUNCTION createMedia(req, res, next)
  TRY
    data = GỌI BẮT ĐỒNG BỘ CreatorService.createMedia(req.body, req.user.id)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Đã thêm media", data: data }
  CATCH err
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 99: async deleteMedia(req, res, next)

Dòng nguồn: 352 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa danh sách media metadata.

```
FUNCTION deleteMedia(req, res, next)
  TRY
    ok = GỌI BẮT ĐỒNG BỘ CreatorService.deleteMedia(req.params.id, req.user.id)
    IF NOT ok THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy media" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã xóa media" }
  CATCH err
    IF err.message = "Media asset is currently linked to content" THEN
      TRẢ VỀ HTTP 409 CÙNG DỮ LIỆU { message: getCreatorErrorMessage(err) }
    END IF
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

controllers/gamification.controller.js

File này có 3 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 100: async getProfile(req, res, next)

Dòng nguồn: 4 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy hồ sơ người dùng.

```
FUNCTION getProfile(req, res, next)
  TRY
    profile = GỌI BẮT ĐỒNG BỘ GamificationService.getProfile(req.user.id)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU profile
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 101: async completePractice(req, res, next)

Dòng nguồn: 13 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Hàm hỗ trợ xử lý complete practice trong module backend.

```
FUNCTION completePractice(req, res, next)
  TRY
    { sessionKey, topicId = null, correctCount = 0, totalAttempts = 0 } =
```

```

    req.body
  IF NOT sessionKey OR typeofsessionKey ≠ "string" THEN
    TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Missing practice session
      key" }
  END IF
  reward = GỌI BẮT ĐỒNG BỘ GamificationService.awardXP(req.user.id, {
    eventType: "PracticeComplete", sourceKey: `practice: ${
      sessionKey.slice(0, 120) }`, metadata: { topicId: Number(topicId) OR
      NULL, correctCount: Number(correctCount) OR 0, totalAttempts:
      Number(totalAttempts) OR 0 } })
  TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU reward
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION

```

Hàm 102: async markAchievementsSeen(req, res, next)

Dòng nguồn: 34 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để đánh dấu trạng thái đã xem huy hiệu.

```

FUNCTION markAchievementsSeen(req, res, next)
  TRY
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ
      GamificationService.markAchievementsSeen(req.user.id,
      req.body.achievementIds)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Achievements marked as seen" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION

```

controllers/learning-path.controller.js

File này có 1 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 103: async getRoadmap(req, res, next)

Dòng nguồn: 4 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy lộ trình học tập.

```

FUNCTION getRoadmap(req, res, next)
  TRY
    roadmap = GỌI BẮT ĐỒNG BỘ LearningPathService.getRoadmap(req.user.id)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU roadmap
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION

```

controllers/progress.controller.js

File này có 2 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 104: async getProgress(req, res, next)

Dòng nguồn: 5 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy progress.

```

FUNCTION getProgress(req, res, next)
  TRY
    userId = req.user.id
    progress = GỌI BẮT ĐỒNG BỘ ProgressService.getProgress(userId)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU progress
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION

```

```

CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
END TRY
END FUNCTION

```

Hàm 105: async getStats(req, res, next)

Dòng nguồn: 15 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy thông kê.

```

FUNCTION getStats(req, res, next)
    TRY
        userId = req.user.id
        stats = GỌI BẮT ĐỒNG BỘ ProgressService.getStats(userId)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU stats
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

controllers/review.controller.js

File này có 5 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 106: async getPending(req, res, next)

Dòng nguồn: 4 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách chờ xử lý.

```

FUNCTION getPending(req, res, next)
    TRY
        data = GỌI BẮT ĐỒNG BỘ ReviewService.getPendingContent()
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
    CATCH err
        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```

Hàm 107: async approve(req, res, next)

Dòng nguồn: 11 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để phê duyệt approve.

```

FUNCTION approve(req, res, next)
    TRY
        { entityType, entityId } = req.params
        ok = GỌI BẮT ĐỒNG BỘ ReviewService.approve(entityType, entityId, req.user.id)
        IF NOT ok THEN
            TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy nội dung chờ duyệt" }
        END IF
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã duyệt thành công" }
    CATCH err
        IF err.message = "EntityType không hợp lệ" THEN
            TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: err.message }
        END IF
        THỰC HIỆN next(err)
    END TRY
END FUNCTION

```


Hàm 108: async reject(req, res, next)

Dòng nguồn: 23 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để từ chối reject.

```
FUNCTION reject(req, res, next)
  TRY
    { entityType, entityId } = req.params
    { reason } = req.body
    ok = GỌI BẮT ĐỒNG BỘ ReviewService.reject(entityType, entityId, req.user.id,
      reason)
    IF NOT ok THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy nội dung chờ
        duyệt" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã từ chối" }
  CATCH err
    IF err.message = "EntityType không hợp lệ" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: err.message }
    END IF
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 109: async archive(req, res, next)

Dòng nguồn: 36 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lưu trữ archive.

```
FUNCTION archive(req, res, next)
  TRY
    { entityType, entityId } = req.params
    ok = GỌI BẮT ĐỒNG BỘ ReviewService.archive(entityType, entityId,
      req.user.id)
    IF NOT ok THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy nội dung" }
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã lưu trữ" }
  CATCH err
    IF err.message = "EntityType không hợp lệ" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: err.message }
    END IF
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

Hàm 110: async getReviewLogs(req, res, next)

Dòng nguồn: 48 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy nhật ký duyệt nội dung.

```
FUNCTION getReviewLogs(req, res, next)
  TRY
    { entityType, entityId } = req.params
    data = GỌI BẮT ĐỒNG BỘ ReviewService.getReviewLogs(entityType, entityId)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH err
    IF err.message = "EntityType không hợp lệ" THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: err.message }
    END IF
    THỰC HIỆN next(err)
  END TRY
END FUNCTION
```

controllers/user.controller.js

File này có 29 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 111: async submitAnswer(req, res, next)

Dòng nguồn: 5 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gửi answer.

```
FUNCTION submitAnswer(req, res, next)
  TRY
    userId = req.user.id
    { questionId, wordId, submittedAnswer, isCorrect, reviewRating, activityType
      } = req.body
    IF NOT questionId AND NOT wordId THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Thiếu questionId" }
    END IF
    feedback = NULL
    IF questionId THEN
      feedback = GỌI BẮT ĐỒNG BỘ UserService.submitAnswer({ userId:
        userId, questionId: questionId, wordId: wordId, submittedAnswer:
        submittedAnswer, isCorrect: isCorrect, reviewRating:
        reviewRating, activityType: activityType })
    ELSE
      feedback = GỌI BẮT ĐỒNG BỘ UserService.submitWordReview({ userId:
        userId, wordId: wordId, isCorrect: Boolean(isCorrect),
        reviewRating: reviewRating, activityType: activityType })
    END IF
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Lưu kết quả thành công",
      ...feedback }
  CATCH error
    THỰC HIỆN console.error("[UserController.submitAnswer] Full Error: ", error)
    TRẢ VỀ HTTP 500 CÙNG DỮ LIỆU { message: "Lỗi server khi nộp bài", error:
      error.message, stack: NẾU process.env.NODE_ENV = "development" THÌ
      error.stack NGƯỢC LẠI undefined }
  END TRY
END FUNCTION
```

Hàm 112: async getFlashcards(req, res, next)

Dòng nguồn: 49 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy bộ flashcard.

```
FUNCTION getFlashcards(req, res, next)
  TRY
    userId = req.user.id
    flashcards = GỌI BẮT ĐỒNG BỘ UserService.getDueFlashcards(userId, { topicId:
      req.query.topicId, mode: req.query.mode })
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU flashcards
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 113: async getTopicWords(req, res, next)

Dòng nguồn: 62 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách từ vựng của chủ đề.

```
FUNCTION getTopicWords(req, res, next)
  TRY
    userId = req.user.id
    { topicId } = req.params
    words = GỌI BẮT ĐỒNG BỘ UserService.getTopicWords(userId, topicId)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU words
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

```

CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
END TRY
END FUNCTION

```

Hàm 114: async getStats(req, res, next)

Dòng nguồn: 73 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy thống kê.

```

FUNCTION getStats(req, res, next)
    TRY
        userId = req.user.id
        stats = GỌI BẮT ĐỒNG BỘ UserService.getUserStats(userId)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU stats
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 115: async getMasteryTimeline(req, res, next)

Dòng nguồn: 83 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy dòng thời gian mức độ ghi nhớ.

```

FUNCTION getMasteryTimeline(req, res, next)
    TRY
        userId = req.user.id
        projection = GỌI BẮT ĐỒNG BỘ UserService.getMasteryTimeline(userId)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU projection
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 116: async getMiniTests(req, res, next)

Dòng nguồn: 93 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách bài kiểm tra ngắn.

```

FUNCTION getMiniTests(req, res, next)
    TRY
        page = req.query.page
        IF page không tồn tại hoặc không chuyển được sang số nguyên THEN
            page = 1
        ELSE
            page = CHUYỂN SANG SỐ NGUYÊN(page)
        END IF
        pageSize = req.query.pageSize
        IF pageSize không tồn tại hoặc không chuyển được sang số nguyên THEN
            pageSize = 20
        ELSE
            pageSize = CHUYỂN SANG SỐ NGUYÊN(pageSize)
        END IF
        tests = GỌI BẮT ĐỒNG BỘ UserService.getMiniTests(page, pageSize)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU tests
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 117: async getMiniTestDetails(req, res, next)

Dòng nguồn: 104 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy chi tiết bài kiểm tra ngắn.

```
FUNCTION getMiniTestDetails(req, res, next)
  TRY
    { id } = req.params
    questions = GỌI BẮT ĐỒNG BỘ UserService.getMiniTestDetails(id)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU questions
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 118: async updateProfile(req, res, next)

Dòng nguồn: 114 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật hồ sơ người dùng.

```
FUNCTION updateProfile(req, res, next)
  TRY
    userId = req.user.id
    { fullName } = req.body
    result = GỌI BẮT ĐỒNG BỘ UserService.updateProfile(userId, fullName)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật thành công", data: result }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 119: async getTestHistory(req, res, next)

Dòng nguồn: 125 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy test history.

```
FUNCTION getTestHistory(req, res, next)
  TRY
    userId = req.user.id
    page = req.query.page
    IF page không tồn tại hoặc không chuyển được sang số nguyên THEN
      page = 1
    ELSE
      page = CHUYỂN SANG SỐ NGUYÊN(page)
    END IF
    pageSize = req.query.pageSize
    IF pageSize không tồn tại hoặc không chuyển được sang số nguyên THEN
      pageSize = 20
    ELSE
      pageSize = CHUYỂN SANG SỐ NGUYÊN(pageSize)
    END IF
    history = GỌI BẮT ĐỒNG BỘ UserService.getTestHistory(userId, page, pageSize)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU history
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 120: async getSessionDetails(req, res, next)

Dòng nguồn: 137 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy test session details.

```
FUNCTION getSessionDetails(req, res, next)
  TRY
    userId = req.user.id
    { testId, date } = req.query
    details = GỌI BẮT ĐỒNG BỘ UserService.getSessionDetails(userId, testId,
      date)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU details
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 121: async createReport(req, res, next)

Dòng nguồn: 148 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để tạo mới báo lỗi/nội dung.

```
FUNCTION createReport(req, res, next)
  TRY
    result = GỌI BẮT ĐỒNG BỘ ReportService.createReport(req.user.id, req.body)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Report submitted", data: result }
  CATCH error
    IF ["Invalid report type", "Invalid entity type", "Report description is too
      short"].includes(error.message) THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: error.message }
    END IF
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 122: async getDailyGoal(req, res, next)

Dòng nguồn: 161 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy mục tiêu hằng ngày.

```
FUNCTION getDailyGoal(req, res, next)
  TRY
    userId = req.user.id
    data = GỌI BẮT ĐỒNG BỘ UserService.getDailyGoal(userId)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 123: async updateDailyGoal(req, res, next)

Dòng nguồn: 171 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật mục tiêu hằng ngày.

```
FUNCTION updateDailyGoal(req, res, next)
  TRY
    userId = req.user.id
    { dailyGoal } = req.body
    IF NOT dailyGoal OR dailyGoal < 5 OR dailyGoal > 100 THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Mục tiêu phải từ 5 đến 100
        từ" }
    END IF
    GỌI BẮT ĐỒNG BỘ UserService.updateDailyGoal(userId, dailyGoal)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Cập nhật mục tiêu thành công" }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

```

END IF
result = GỌI BẮT ĐỒNG BỘ UserService.updateDailyGoal(userId, dailyGoal)
TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU result
CATCH error
  CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
END TRY
END FUNCTION

```

Hàm 124: async updateSRSSConfig(req, res, next)

Dòng nguồn: 185 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật cấu hình giới hạn ôn SRS.

```

FUNCTION updateSRSSConfig(req, res, next)
  TRY
    userId = req.user.id
    { srsReviewLimit } = req.body
    IF NOT srsReviewLimit OR srsReviewLimit < 5 OR srsReviewLimit > 50 THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Số thẻ mỗi ngày phải từ 5
        đến 50" }
    END IF
    result = GỌI BẮT ĐỒNG BỘ UserService.updateSRSSReviewLimit(userId,
      srsReviewLimit)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU result
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION

```

Hàm 125: async submitMiniTest(req, res, next)

Dòng nguồn: 200 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để gửi bài kiểm tra ngắn.

```

FUNCTION submitMiniTest(req, res, next)
  TRY
    userId = req.user.id
    { id } = req.params
    { answers } = req.body
    IF NOT Array.isArray(answers) OR answers.length = 0 THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Thiếu danh sách câu trả
        lời" }
    END IF
    result = GỌI BẮT ĐỒNG BỘ UserService.submitMiniTestBatch(userId, id,
      answers)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU result
  CATCH error
    THỰC HIỆN console.error("[UserController.submitMiniTest] Error: ", error)
    TRẢ VỀ HTTP 500 CÙNG DỮ LIỆU { message: "Lỗi server khi nộp bài kiểm tra",
      error: error.message }
  END TRY
END FUNCTION

```

Hàm 126: async getActivityHeatmap(req, res, next)

Dòng nguồn: 222 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy activity heatmap.

```

FUNCTION getActivityHeatmap(req, res, next)
  TRY
    userId = req.user.id
    year = req.query.year
    IF year không tồn tại hoặc không chuyển được sang số nguyên THEN
      year = TẠO Date().getFullYear()
    ELSE

```

```

        year = CHUYỂN SANG SỐ NGUYÊN(year)
    END IF
    data = GỌI BẮT ĐỒNG BỘ UserService.getActivityHeatmap(userId, year)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
END TRY
END FUNCTION

```

Hàm 127: async getProgressAnalytics(req, res, next)

Dòng nguồn: 233 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy phân tích tiến độ học tập.

```

FUNCTION getProgressAnalytics(req, res, next)
    TRY
        data = GỌI BẮT ĐỒNG BỘ UserService.getProgressAnalytics(req.user.id)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 128: async getDailyProgress(req, res, next)

Dòng nguồn: 243 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy tiến độ mục tiêu hằng ngày.

```

FUNCTION getDailyProgress(req, res, next)
    TRY
        userId = req.user.id
        progress = GỌI BẮT ĐỒNG BỘ UserService.getDailyProgress(userId)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU progress
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 129: async getSmartReviewQueue(req, res, next)

Dòng nguồn: 254 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy hàng đợi ôn tập thông minh.

```

FUNCTION getSmartReviewQueue(req, res, next)
    TRY
        userId = req.user.id
        limit = req.query.limit
        IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN
            limit = 20
        ELSE
            limit = CHUYỂN SANG SỐ NGUYÊN(limit)
        END IF
        queue = GỌI BẮT ĐỒNG BỘ UserService.getSmartReviewQueue(userId, limit)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU queue
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 130: async getNotebook(req, res, next)

Dòng nguồn: 266 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy sổ tay từ vựng.

```
FUNCTION getNotebook(req, res, next)
  TRY
    userId = req.user.id
    page = req.query.page
    IF page không tồn tại hoặc không chuyển được sang số nguyên THEN
      page = 1
    ELSE
      page = CHUYỂN SANG SỐ NGUYÊN(page)
    END IF
    pageSize = req.query.pageSize
    IF pageSize không tồn tại hoặc không chuyển được sang số nguyên THEN
      pageSize = 20
    ELSE
      pageSize = CHUYỂN SANG SỐ NGUYÊN(pageSize)
    END IF
    notebook = GỌI BẮT ĐỒNG BỘ UserService.getNotebook(userId, page, pageSize)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU notebook
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 131: async addNotebookEntry(req, res, next)

Dòng nguồn: 278 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để thêm một mục trong sổ tay từ vựng.

```
FUNCTION addNotebookEntry(req, res, next)
  TRY
    userId = req.user.id
    { wordId, personalNote } = req.body
    IF NOT wordId THEN
      TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Thiếu wordId" }
    END IF
    entry = GỌI BẮT ĐỒNG BỘ UserService.addNotebookEntry(userId, wordId,
      personalNote)
    TRẢ VỀ HTTP 201 CÙNG DỮ LIỆU { message: "Đã thêm vào sổ tay", data: entry }
  CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
  END TRY
END FUNCTION
```

Hàm 132: async updateNotebookEntry(req, res, next)

Dòng nguồn: 290 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để cập nhật một mục trong sổ tay từ vựng.

```
FUNCTION updateNotebookEntry(req, res, next)
  TRY
    userId = req.user.id
    { id } = req.params
    { personalNote, isFavorite } = req.body
    entry = GỌI BẮT ĐỒNG BỘ UserService.updateNotebookEntry(id, userId, {
      personalNote: personalNote, isFavorite: isFavorite })
    IF NOT entry THEN
      TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy mục trong sổ
        tay" }
    END IF
  END TRY
END FUNCTION
```



```

        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã cập nhật sổ tay", data: entry }
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 133: async deleteNotebookEntry(req, res, next)

Dòng nguồn: 303 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để xóa một mục trong sổ tay từ vựng.

```

FUNCTION deleteNotebookEntry(req, res, next)
    TRY
        userId = req.user.id
        { id } = req.params
        entry = GỌI BẮT ĐỒNG BỘ UserService.deleteNotebookEntry(id, userId)
        IF NOT entry THEN
            TRẢ VỀ HTTP 404 CÙNG DỮ LIỆU { message: "Không tìm thấy mục trong sổ tay" }
        END IF
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã xóa khỏi sổ tay" }
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 134: async checkNotebookEntry(req, res, next)

Dòng nguồn: 315 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để kiểm tra một mục trong sổ tay từ vựng.

```

FUNCTION checkNotebookEntry(req, res, next)
    TRY
        userId = req.user.id
        wordId = CHUYỂN SANG SỐ NGUYÊN(req.query.wordId)
        IF NOT wordId THEN
            TRẢ VỀ HTTP 400 CÙNG DỮ LIỆU { message: "Thiếu wordId" }
        END IF
        entry = GỌI BẮT ĐỒNG BỘ UserService.checkNotebookEntry(userId, wordId)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU entry OR { }
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 135: async getNotifications(req, res, next)

Dòng nguồn: 328 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy danh sách thông báo.

```

FUNCTION getNotifications(req, res, next)
    TRY
        userId = req.user.id
        limit = req.query.limit
        IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN
            limit = 20
        ELSE
            limit = CHUYỂN SANG SỐ NGUYÊN(limit)
        END IF
        data = GỌI BẮT ĐỒNG BỘ UserService.getUserNotifications(userId, limit)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU data
    CATCH error

```

```

        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 136: async markNotificationRead(req, res, next)

Dòng nguồn: 339 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để đánh dấu notification read.

```

FUNCTION markNotificationRead(req, res, next)
    TRY
        userId = req.user.id
        { id } = req.params
        THỰC HIỆN GỌI BẮT ĐỒNG BỘ UserService.markNotificationRead(userId, id)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU { message: "Đã đánh dấu đã đọc" }
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 137: async markAllNotificationsRead(req, res, next)

Dòng nguồn: 350 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để đánh dấu all notifications read.

```

FUNCTION markAllNotificationsRead(req, res, next)
    TRY
        userId = req.user.id
        result = GỌI BẮT ĐỒNG BỘ UserService.markAllNotificationsRead(userId)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU result
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 138: async getSessionSummary(req, res, next)

Dòng nguồn: 361 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy tóm tắt phiên ôn tập.

```

FUNCTION getSessionSummary(req, res, next)
    TRY
        userId = req.user.id
        summary = GỌI BẮT ĐỒNG BỘ UserService.getSessionSummary(userId)
        TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU summary
    CATCH error
        CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
    END TRY
END FUNCTION

```

Hàm 139: async getMistakeReviewQueue(req, res, next)

Dòng nguồn: 372 | Đầu vào: req, res, next | Đầu ra: HTTP status và JSON response; lỗi được chuyển tới middleware errorHandler.

Mục đích: Nhận request HTTP, kiểm tra/chuẩn hóa tham số và gọi service để lấy hàng đợi ôn lại câu sai.

```

FUNCTION getMistakeReviewQueue(req, res, next)
    TRY
        userId = req.user.id
        limit = req.query.limit
        IF limit không tồn tại hoặc không chuyển được sang số nguyên THEN

```

```

        limit = 10
    ELSE
        limit = CHUYỂN SANG SỐ NGUYÊN(limit)
    END IF
    queue = GỌI BẮT ĐỒNG BỘ UserService.getMistakeReviewQueue(userId, limit)
    TRẢ VỀ HTTP 200 CÙNG DỮ LIỆU queue
CATCH error
    CHUYỂN LỖI CHO MIDDLEWARE XỬ LÝ
END TRY
END FUNCTION

```

5.4. Service

services/admin.service.js

File này có 56 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 140: assertValidUserRole(roleName)

Dòng nguồn: 6 | Đầu vào: roleName | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý assert valid user role trong module backend.

```

FUNCTION assertValidUserRole(roleName)
    IF NOT USER_ROLES.includes(roleName) THEN
        THROW ERROR TẠO Error("Invalid role")
    END IF
END FUNCTION

```

Hàm 141: normalizePagination(page = 1, limit = 20, maxLimit = 100)

Dòng nguồn: 15 | Đầu vào: page = 1, limit = 20, maxLimit = 100 | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để chuẩn hóa pagination.

```

FUNCTION normalizePagination(page = 1, limit = 20, maxLimit = 100)
    normalizedPage = Math.max(CHUYỂN SANG SỐ NGUYÊN(page, 10) OR 1, 1)
    normalizedLimit = Math.min(Math.max(CHUYỂN SANG SỐ NGUYÊN(limit, 10) OR 20, 1),
        maxLimit)
    RETURN { page: normalizedPage, limit: normalizedLimit, offset: normalizedPage - 1
*
        normalizedLimit }
END FUNCTION

```

Hàm 142: paginate(items, total, page, limit)

Dòng nguồn: 26 | Đầu vào: items, total, page, limit | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý paginate trong module backend.

```

FUNCTION paginate(items, total, page, limit)
    normalizedTotal = Number(total) OR 0
    RETURN { items: items, pagination: { page: page, limit: limit, total:
        normalizedTotal, totalPages: Math.max(1, Math.ceil(normalizedTotal / limit))
    } }
END FUNCTION

```

Hàm 143: buildTopicCode(name)

Dòng nguồn: 40 | Đầu vào: name | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xây dựng topic code.

```

FUNCTION buildTopicCode(name)
    code = String(name NẾU RỖNG THÌ "").normalize("NFD").replace(/[u0300-\u036f]/g,
        "").replace(/\u0111/g, "d").replace(/\u0110/g,
        "D").toUpperCase().replace(/^[A-Z0-9]+/g, "_").replace(/^\_|_+$/g,
        "").slice(0,
        50)

```

```
RETURN code OR `TOPIC_${ Date.now() }`
END FUNCTION
```

Hàm 144: assertContentStatus(status)

Dòng nguồn: 54 | Đầu vào: status | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý assert content status trong module backend.

```
FUNCTION assertContentStatus(status)
  IF NOT this.CONTENT_STATUSES.includes(status) THEN
    THROW ERROR TẠO Error("Invalid content status")
  END IF
END FUNCTION
```

Hàm 145: async logAdminAction(adminId, action, entityType, entityId, details = null)

Dòng nguồn: 60 | Đầu vào: adminId, action, entityType, entityId, details = null | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý log admin action trong module backend.

```
FUNCTION logAdminAction(adminId, action, entityType, entityId, details = null)
  TRY
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    THỰC THI TRUY VẤN [CÂU LỆNH CREATE]
    THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên AdminAuditLogs]
  CATCH error
    THỰC HIỆN console.warn("Failed to write admin audit log", error.message)
  END TRY
END FUNCTION
```

Hàm 146: async logContentReview(entityType, entityId, oldStatus, newStatus, adminId, comment = null)

Dòng nguồn: 94 | Đầu vào: entityType, entityId, oldStatus, newStatus, adminId, comment = null | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý log content review trong module backend.

```
FUNCTION logContentReview(entityType, entityId, oldStatus, newStatus, adminId,
comment =
  null)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên ContentReviewLogs]
END FUNCTION
```

Hàm 147: normalizeImportKey(value)

Dòng nguồn: 112 | Đầu vào: value | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để chuẩn hóa import key.

```
FUNCTION normalizeImportKey(value)
  RETURN String(value NẾU RỖNG THÌ "").replace(/\u0111/g, "d").replace(/\u0110/g,
"D").replace(/\^\\uFEFF/, "").normalize("NFD").replace(/[\\u0300-\\u036f]/g,
"" ).toLowerCase().replace(/[a-z0-9]/g, "")
END FUNCTION
```

Hàm 148: parseDelimitedImport(input)

Dòng nguồn: 123 | Đầu vào: input | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để phân tích và chuẩn hóa delimited import.

```
FUNCTION parseDelimitedImport(input)
  IF Array.isArray(input) THEN
    RETURN input
  END IF
  IF typeofinput = "object" AND input THEN
    IF Array.isArray(input.words) THEN
      RETURN input.words
    END IF
  END IF
```

```

        IF Array.isArray(input.questions) THEN
            RETURN input.questions
        END IF
        IF Array.isArray(input.rows) THEN
            RETURN input.rows
        END IF
    END IF
    IF typeofinput ≠ "string" THEN
        THROW ERROR TẠO Error("Invalid import payload")
    END IF
    trimmed = input.trim()
    IF NOT trimmed THEN
        THROW ERROR TẠO Error("CSV must include a header and at least one data row")
    END IF
    firstLine = trimmed.split(/\r?\n/)[0] OR ""
    delimiters = [",", ";", "\t"]
    delimiter = delimiters.reduce(HÀM CALLBACK(best, current), ",")
    records = []
    row = []
    cell = ""
    inQuotes = false
    FOR let i = 0; WHILE i < trimmed.length; i TĂNG 1
        char = trimmed[i]
        next = trimmed[i + 1]
        IF char = "\"" AND next = "\"" THEN
            cell = "\""
            THỰC HIỆN i TĂNG 1
        ELSE
            IF char = "\"" THEN
                inQuotes = NOT inQuotes
            ELSE
                IF char = delimiter AND NOT inQuotes THEN
                    THỰC HIỆN row.push(cell.trim())
                    cell = ""
                ELSE
                    IF char = "\n" OR char = "\r" AND NOT inQuotes THEN
                        IF char = "\r" AND next = "\n" THEN
                            THỰC HIỆN i TĂNG 1
                        END IF
                        THỰC HIỆN row.push(cell.trim())
                        IF row.some(HÀM CALLBACK(value)) THEN
                            THỰC HIỆN records.push(row)
                        END IF
                        row = []
                        cell = ""
                    ELSE
                        cell = char
                    END IF
                END IF
            END IF
        END IF
    END FOR
    THỰC HIỆN row.push(cell.trim())
    IF row.some(HÀM CALLBACK(value)) THEN
        THỰC HIỆN records.push(row)
    END IF
    IF records.length < 2 THEN
        THROW ERROR TẠO Error("CSV must include a header and at least one data row")
    END IF
    headers = records[0].map(HÀM CALLBACK(header))
    RETURN records.slice(1).map(HÀM CALLBACK(cells))
END FUNCTION

```

Hàm 149: getImportValue(row, aliases)

Dòng nguồn: 195 | Đầu vào: row, aliases | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy import value.

```

FUNCTION getImportValue(row, aliases)
    normalizedAliases = aliases.map(HÀM CALLBACK(alias))

```

```

entries = Object.entries(row NẾU RÕNG THÌ { })
FOR EACH [key, value] IN entries
  IF normalizedAliases.includes(this.normalizeImportKey(key)) THEN
    RETURN value
  END IF
END FOR
RETURN undefined
END FUNCTION

```

Hàm 150: splitImportList(value)

Dòng nguồn: 208 | Đầu vào: value | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tách import list.

```

FUNCTION splitImportList(value)
  IF Array.isArray(value) THEN
    RETURN value.map(HÀM CALLBACK(item)).filter(Boolean)
  END IF
  RETURN String(value NẾU RÕNG THÌ "").split(/[; , |]/).map(HÀM
    CALLBACK(item)).filter(Boolean)
END FUNCTION

```

Hàm 151: parseQuestionImport(input)

Dòng nguồn: 219 | Đầu vào: input | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để phân tích và chuẩn hóa question import.

```

FUNCTION parseQuestionImport(input)
  RETURN this.parseDelimitedImport(input)
END FUNCTION

```

Hàm 152: parseWordImport(input)

Dòng nguồn: 223 | Đầu vào: input | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để phân tích và chuẩn hóa dữ liệu import từ vựng.

```

FUNCTION parseWordImport(input)
  RETURN this.parseDelimitedImport(input)
END FUNCTION

```

Hàm 153: async getTopics(page = 1, limit = 50, filters = { })

Dòng nguồn: 228 | Đầu vào: page = 1, limit = 50, filters = { } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách chủ đề.

```

FUNCTION getTopics(page = 1, limit = 50, filters = { })
  pool = GỢI BẮT ĐỒNG BỘ poolPromise
  paging = this.normalizePagination(page, limit, 200)
  search = String(filters.search NẾU RÕNG THÌ "").trim()
  status = String(filters.status NẾU RÕNG THÌ "").trim()
  categoryId = Number(filters.categoryId) OR NULL
  conditions = []
  request = pool.request().input("Offset", sql.Int, paging.offset).input("Limit",
    sql.Int, paging.limit)
  IF search THEN
    THỰC HIỆN request.input("Search", sql.NVarChar(250), `${ search }`)
    THỰC HIỆN conditions.push("t.TopicName LIKE @Search OR t.TopicCode LIKE
      @Search OR t.Description LIKE @Search")
  END IF
  IF status THEN
    THỰC HIỆN this.assertContentStatus(status)
    THỰC HIỆN request.input("ContentStatus", sql.NVarChar(30), status)
    THỰC HIỆN conditions.push("t.ContentStatus = @ContentStatus")
  END IF
  IF categoryId THEN
    THỰC HIỆN request.input("TopicCategoryID", sql.BigInt, categoryId)
    THỰC HIỆN conditions.push("t.TopicCategoryID = @TopicCategoryID")
  END IF

```

```

END IF
whereClause = NẾU conditions.length > 0 THÌ `WHERE ${ conditions.join(' AND ') }`
NGƯỢC LẠI ""
result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics,
TopicCategories, WordTopics, MiniTests]
RETURN this.paginate(result.recordsets[1] OR [], result.recordsets[0][0].total OR
0,
    paging.page, paging.limit)
END FUNCTION

```

Hàm 154: async getTopicCategories()

Dòng nguồn: 288 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh mục chủ đề.

```

FUNCTION getTopicCategories()
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên TopicCategories,
    Topics, WordTopics]
    RETURN result.recordset
END FUNCTION

```

Hàm 155: async createTopic(topicData, adminId)

Dòng nguồn: 314 | Đầu vào: topicData, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới chủ đề.

```

FUNCTION createTopic(topicData, adminId)
    name = String(topicData.name NẾU RỎNG THÌ "").trim()
    description = String(topicData.description NẾU RỎNG THÌ "").trim()
    code = this.buildTopicCode(topicData.code OR name)
    topicCategoryId = NẾU topicData.topicCategoryId THÌ
        Number(topicData.topicCategoryId) NGƯỢC LẠI NULL
    status = topicData.status OR "Published"
    IF NOT name OR NOT adminId THEN
        THROW ERROR TẠO Error("Invalid topic data")
    END IF
    THỰC HIỆN this.assertContentStatus(status)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    duplicate = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics]
    existing = duplicate.recordset[0] OR { }
    IF existing.nameCount > 0 THEN
        THROW ERROR TẠO Error("Topic already exists")
    END IF
    IF existing.codeCount > 0 THEN
        THROW ERROR TẠO Error("Topic code already exists")
    END IF
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên TopicCategories,
    Topics]
    created = result.recordset[0]
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId, "CREATE_TOPIC", "Topic",
        created.id, created)
    RETURN created
END FUNCTION

```

Hàm 156: async updateTopic(topicId, topicData, adminId)

Dòng nguồn: 391 | Đầu vào: topicId, topicData, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật chủ đề.

```

FUNCTION updateTopic(topicId, topicData, adminId)
    name = NẾU topicData.name THÌ String(topicData.name).trim() NGƯỢC LẠI NULL
    description = NẾU topicData.description ≠ undefined THÌ
String(topicData.description
    NẾU RỎNG THÌ "").trim() NGƯỢC LẠI NULL
    code = NẾU topicData.code THÌ this.buildTopicCode(topicData.code) NGƯỢC LẠI NULL

```

```

        topicCategoryId = NẾU topicData.topicCategoryId = undefined THÌ undefined NGƯỢC
LẠI
        NẾU topicData.topicCategoryId THÌ Number(topicData.topicCategoryId) NGƯỢC LẠI
        NULL
        status = topicData.status OR NULL
        IF NOT name AND description = NULL AND NOT code AND topicCategoryId = undefined
AND
        NOT status THEN
            THROW ERROR TẠO Error("Invalid topic data")
        END IF
        IF status THEN
            THỰC HIỆN this.assertContentStatus(status)
        END IF
        pool = GỌI BẮT ĐỒNG BỘ poolPromise
        IF name OR code THEN
            duplicate = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics]
            existing = duplicate.recordset[0] OR { }
            IF existing.nameCount > 0 THEN
                THROW ERROR TẠO Error("Topic already exists")
            END IF
            IF existing.codeCount > 0 THEN
                THROW ERROR TẠO Error("Topic code already exists")
            END IF
        END IF
        oldStatusResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT ContentStatus FROM
        Topics WHERE TopicID = @TopicID"
        IF oldStatusResult.recordset.length = 0 THEN
            RETURN false
        END IF
        request = pool.request().input("TopicID", sql.BigInt, topicId).input("TopicName",
        sql.NVarChar(200), name).input("TopicCode", sql.NVarChar(50),
        code).input("Description", sql.NVarChar(1000),
        description).input("TopicCategoryId", sql.BigInt, NẾU topicCategoryId =
        undefined THÌ NULL NGƯỢC LẠI topicCategoryId).input("HasTopicCategoryId",
        sql.Bit, topicCategoryId ≠ undefined).input("ContentStatus",
        sql.NVarChar(30),
        status)
        result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên TopicCategories,
        Topics]
        IF result.rowsAffected[0] > 0 THEN
            THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId, "UPDATE_TOPIC",
            "Topic", topicId, topicData)
            IF status THEN
                THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logContentReview("Topic", topicId,
                oldStatusResult.recordset[0].ContentStatus, status, adminId,
                "Updated from topic manager")
            END IF
        END IF
        RETURN result.rowsAffected[0] > 0
    END FUNCTION

```

Hàm 157: async deleteTopic(topicId, adminId)

Dòng nguồn: 463 | Đầu vào: topicId, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa chủ đề.

```

FUNCTION deleteTopic(topicId, adminId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    dependencies = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics,
    WordTopics, MiniTests]
    row = dependencies.recordset[0]
    IF NOT row.existsCount THEN
        RETURN { success: false, archived: false }
    END IF
    IF row.wordCount > 0 OR row.miniTestCount > 0 THEN
        THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.updateTopic(topicId, { status: "Archived" },
        adminId)
        RETURN { success: true, archived: true }
    END IF
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "DELETE FROM Topics WHERE TopicID =

```



```

        @TopicID"
    IF result.rowsAffected[0] > 0 THEN
        THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId, "DELETE_TOPIC",
            "Topic", topicId)
    END IF
    RETURN { success: result.rowsAffected[0] > 0, archived: false }
END FUNCTION

```

Hàm 158: async createTopicCategory(categoryData, adminId)

Dòng nguồn: 493 | Đầu vào: categoryData, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới danh mục chủ đề.

```

FUNCTION createTopicCategory(categoryData, adminId)
    name = String(categoryData.name NẾU RỎNG THÌ "").trim()
    code = this.buildTopicCode(categoryData.code OR name)
    description = String(categoryData.description NẾU RỎNG THÌ "").trim()
    iconUrl = String(categoryData.iconUrl NẾU RỎNG THÌ "").trim()
    displayOrder = Number(categoryData.displayOrder) OR 1
    isActive = NẾU categoryData.isActive = undefined THÌ true NGƯỢC LẠI
        Boolean(categoryData.isActive)
    IF NOT name THEN
        THROW ERROR TẠO Error("Invalid topic category data")
    END IF
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên TopicCategories]
    created = result.recordset[0]
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId, "CREATE_TOPIC_CATEGORY",
        "TopicCategory", created.id, created)
    RETURN created
END FUNCTION

```

Hàm 159: async updateTopicCategory(categoryId, categoryData, adminId)

Dòng nguồn: 526 | Đầu vào: categoryId, categoryData, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật danh mục chủ đề.

```

FUNCTION updateTopicCategory(categoryId, categoryData, adminId)
    name = String(categoryData.name NẾU RỎNG THÌ "").trim()
    code = this.buildTopicCode(categoryData.code OR name)
    description = String(categoryData.description NẾU RỎNG THÌ "").trim()
    iconUrl = String(categoryData.iconUrl NẾU RỎNG THÌ "").trim()
    displayOrder = Number(categoryData.displayOrder) OR 1
    isActive = NẾU categoryData.isActive = undefined THÌ true NGƯỢC LẠI
        Boolean(categoryData.isActive)
    IF NOT name THEN
        THROW ERROR TẠO Error("Invalid topic category data")
    END IF
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên TopicCategories]
    IF result.rowsAffected[0] > 0 THEN
        THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId,
            "UPDATE_TOPIC_CATEGORY", "TopicCategory", categoryId, categoryData)
    END IF
    RETURN result.rowsAffected[0] > 0
END FUNCTION

```

Hàm 160: async deleteTopicCategory(categoryId, adminId)

Dòng nguồn: 567 | Đầu vào: categoryId, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa danh mục chủ đề.

```

FUNCTION deleteTopicCategory(categoryId, adminId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Topics,
        TopicCategories]

```

```

success = result.rowsAffected.some(HÀM CALLBACK(count))
IF success THEN
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId,
        "DISABLE_TOPIC_CATEGORY", "TopicCategory", categoryId)
END IF
RETURN success
END FUNCTION

```

Hàm 161: async getWords(page = 1, limit = 20, filters = { })

Dòng nguồn: 588 | Đầu vào: page = 1, limit = 20, filters = { } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách từ vựng.

```

FUNCTION getWords(page = 1, limit = 20, filters = { })
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    paging = this.normalizePagination(page, limit, 100)
    topicId = Number(filters.topicId) OR NULL
    partOfSpeechId = Number(filters.partOfSpeechId) OR NULL
    search = String(filters.search NẾU RỘNG THÌ "").trim()
    status = String(filters.status NẾU RỘNG THÌ "").trim()
    missingExamples = filters.missingExamples = true OR filters.missingExamples =
"true"
    missingQuestions = filters.missingQuestions = true OR filters.missingQuestions =
"true"
    sortBy = NẾU ["term", "createdAt", "updatedAt", "questionCount",
        "exampleCount"].includes(filters.sortBy) THÌ filters.sortBy NGƯỢC LẠI
        "createdAt"
    sortDirection = NẾU String(filters.sortDirection NẾU RỘNG THÌ
"desc").toLowerCase()
        = "asc" THÌ "ASC" NGƯỢC LẠI "DESC"
    sortMap = { term: "w.Term", createdAt: "w.CreatedAt", updatedAt: "w.UpdatedAt",
        questionCount: "QuestionCount", exampleCount: "ExampleCount" }
    conditions = []
    request = pool.request().input("Offset", sql.Int, paging.offset).input("Limit",
        sql.Int, paging.limit)
    IF topicId THEN
        THỰC HIỆN request.input("TopicID", sql.BigInt, topicId)
        THỰC HIỆN conditions.push([CÂU LỆNH SELECT trên WordTopics])
    END IF
    IF search THEN
        THỰC HIỆN request.input("Search", sql.NVarChar(250), `${search}%`)
        THỰC HIỆN conditions.push("(w.Term LIKE @Search OR w.Meaning LIKE @Search OR
            w.Phonetic LIKE @Search)")
    END IF
    IF partOfSpeechId THEN
        THỰC HIỆN request.input("PartOfSpeechID", sql.Int, partOfSpeechId)
        THỰC HIỆN conditions.push("w.PartOfSpeechID = @PartOfSpeechID")
    END IF
    IF status THEN
        THỰC HIỆN this.assertContentStatus(status)
        THỰC HIỆN request.input("ContentStatus", sql.NVarChar(30), status)
        THỰC HIỆN conditions.push("w.ContentStatus = @ContentStatus")
    END IF
    IF missingExamples THEN
        THỰC HIỆN conditions.push("NOT EXISTS (SELECT 1 FROM ExampleSentences
            exFilter WHERE exFilter.WordID = w.WordID)")
    END IF
    IF missingQuestions THEN
        THỰC HIỆN conditions.push("NOT EXISTS (SELECT 1 FROM Questions qFilter WHERE
            qFilter.WordID = w.WordID)")
    END IF
    whereClause = NẾU conditions.length > 0 THÌ `WHERE ${conditions.join(' AND ')} `
        NGƯỢC LẠI ""
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words,
        Questions,
        ExampleSentences, PartOfSpeeches]
    total = result.recordsets[0][0].total OR 0
    words = GỌI BẮT ĐỒNG BỘ this.attachWordRelations(pool, result.recordsets[1] OR
[])

```

```
RETURN this.paginate(words, total, paging.page, paging.limit)
END FUNCTION
```

Hàm 162: `async attachWordRelations(pool, words)`

Dòng nguồn: 684 | Đầu vào: pool, words | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý attach word relations trong module backend.

```
FUNCTION attachWordRelations(pool, words)
  IF NOT words.length THEN
    RETURN words
  END IF
  wordIds = words.map(HÀM CALLBACK(word)).filter(Boolean)
  idList = wordIds.join(", ")
  relatedResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
WordTopics,
  ExampleSentences, Topics]
  topicsByWord = TẠO Map()
  examplesByWord = TẠO Map()
  FOR EACH topic IN relatedResult.recordsets[0] OR []
    key = Number(topic.wordId)
    IF NOT topicsByWord.has(key) THEN
      THỰC HIỆN topicsByWord.set(key, [])
    END IF
    THỰC HIỆN topicsByWord.get(key).push({ id: topic.id, name: topic.name, code:
topic.code })
  END FOR
  FOR EACH example IN relatedResult.recordsets[1] OR []
    key = Number(example.wordId)
    IF NOT examplesByWord.has(key) THEN
      THỰC HIỆN examplesByWord.set(key, [])
    END IF
    THỰC HIỆN examplesByWord.get(key).push({ id: example.id, sentence:
example.sentence, meaning: example.meaning })
  END FOR
  RETURN words.map(HÀM CALLBACK(word))
END FUNCTION
```

Hàm 163: `async getWordDetail(wordId)`

Dòng nguồn: 724 | Đầu vào: wordId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy chi tiết từ vựng.

```
FUNCTION getWordDetail(wordId)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words,
Questions,
  ExampleSentences, WordTopics, AdminAuditLogs]
  word = result.recordsets[0][0]
  IF NOT word THEN
    RETURN NULL
  END IF
  RETURN { ...word, topics: result.recordsets[1] OR [], examples:
result.recordsets[2]
OR [], questions: result.recordsets[3] OR [], auditLogs: result.recordsets[4]
OR
  [] }
END FUNCTION
```

Hàm 164: `async createWord(wordData, adminId)`

Dòng nguồn: 806 | Đầu vào: wordData, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Tạo mới từ vựng trong transaction, gán chủ đề, thêm ví dụ minh họa và ghi nhật ký quản trị.

```
FUNCTION createWord(wordData, adminId)
  { term, meaning, phonetic = '', partOfSpeechId, topicIds, examples, status =
```

```

        'Published' } = wordData
    THỰC HIỆN this.assertContentStatus(status)
    normalizedTopicIds = NẾU Array.isArray(topicIds) THÌ [...new
        Set(topicIds.map((topicId) => Number(topicId)).filter(Boolean))] NGƯỢC LẠI []
    validExamples = NẾU Array.isArray(examples) THÌ examples.filter(HÀM CALLBACK(ex))
        NGƯỢC LẠI []
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        request = TẠO sql.Request(transaction)
        wordResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Words]
        wordId = wordResult.recordset[0].id
        IF normalizedTopicIds.length > 0 THEN
            FOR EACH topicId IN normalizedTopicIds
                topicReq = TẠO sql.Request(transaction)
                THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên WordTopics]
            END FOR
        END IF
        IF validExamples.length > 0 THEN
            FOR EACH ex IN validExamples
                exReq = TẠO sql.Request(transaction)
                THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên ExampleSentences]
            END FOR
        END IF
        COMMIT TRANSACTION
        THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId, "CREATE_WORD",
            "Word", wordId, { term: term, topicIds: normalizedTopicIds })
        RETURN { id: wordId, term: term, meaning: meaning }
    CATCH error
        ROLLBACK TRANSACTION
        THROW ERROR error
    END TRY
END FUNCTION

```

Hàm 165: async updateWord(wordId, wordData, adminId = null)

Dòng nguồn: 876 | Đầu vào: wordId, wordData, adminId = null | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Cập nhật từ vựng, làm mới liên kết chủ đề/câu ví dụ trong transaction và lưu audit log.

```

FUNCTION updateWord(wordId, wordData, adminId = null)
    { term, meaning, phonetic = '', partOfSpeechId, topicIds, examples, status =
        'Published' } = wordData
    THỰC HIỆN this.assertContentStatus(status)
    normalizedTopicIds = NẾU Array.isArray(topicIds) THÌ [...new
        Set(topicIds.map((topicId) => Number(topicId)).filter(Boolean))] NGƯỢC LẠI
    NULL
    validExamples = NẾU Array.isArray(examples) THÌ examples.filter(HÀM CALLBACK(ex))
        NGƯỢC LẠI NULL
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        updateReq = TẠO sql.Request(transaction)
        result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Words]
        IF result.rowsAffected[0] = 0 THEN
            ROLLBACK TRANSACTION
            RETURN false
        END IF
        IF normalizedTopicIds THEN
            deleteTopicsReq = TẠO sql.Request(transaction)
            THỰC THI TRUY VẤN "DELETE FROM WordTopics WHERE WordID = @WordID"
            FOR EACH topicId IN normalizedTopicIds
                topicReq = TẠO sql.Request(transaction)
                THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên WordTopics]
            END FOR
        END IF
        IF validExamples THEN
            deleteExamplesReq = TẠO sql.Request(transaction)
            THỰC THI TRUY VẤN "DELETE FROM ExampleSentences WHERE WordID =

```

```

        @WordID"
    FOR EACH ex IN validExamples
        exReq = TẠO sql.Request(transaction)
        THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên ExampleSentences]
    END FOR
END IF
COMMIT TRANSACTION
IF adminId THEN
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId,
        "UPDATE_WORD", "Word", wordId, { term: term, topicIds:
            normalizedTopicIds })
END IF
RETURN true
CATCH error
    ROLLBACK TRANSACTION
    THROW ERROR error
END TRY
END FUNCTION

```

Hàm 166: async archiveWord(wordId, adminId = null)

Dòng nguồn: 961 | Đầu vào: wordId, adminId = null | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lưu trữ từ vựng.

```

FUNCTION archiveWord(wordId, adminId = null)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    oldStatusResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT ContentStatus FROM
Words
    WHERE WordID = @WordID"
    IF oldStatusResult.recordset.length = 0 THEN
        RETURN false
    END IF
    oldStatus = oldStatusResult.recordset[0].ContentStatus
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Words]
    IF result.rowsAffected[0] > 0 AND adminId THEN
        THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId, "ARCHIVE_WORD",
            "Word", wordId)
        THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logContentReview("Word", wordId, oldStatus,
            "Archived", adminId, "Archived from word manager")
    END IF
    RETURN result.rowsAffected[0] > 0
END FUNCTION

```

Hàm 167: async deleteWord(wordId, adminId = null)

Dòng nguồn: 988 | Đầu vào: wordId, adminId = null | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xóa mềm hoặc xóa logic từ vựng theo ràng buộc nghiệp vụ và cập nhật nhật ký thao tác.

```

FUNCTION deleteWord(wordId, adminId = null)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        request = TẠO sql.Request(transaction)
        result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words,
            MiniTestItems, Questions, MiniTests, ExerciseAttempts]
        COMMIT TRANSACTION
        success = result.recordset[0].deleted > 0
        IF success AND adminId THEN
            THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId,
                "DELETE_WORD", "Word", wordId)
        END IF
        RETURN success
    CATCH error
        ROLLBACK TRANSACTION
        THROW ERROR error
    END TRY
END FUNCTION

```

Hàm 168: async bulkInsertWords(input, adminId)

Dòng nguồn: 1055 | Đầu vào: input, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Đọc dữ liệu import hàng loạt, ánh xạ topic/từ loại và tạo từng từ vựng vào CSDL.

```

FUNCTION bulkInsertWords(input, adminId)
  rows = this.parseWordImport(input)
  results = { success: 0, failed: 0, errors: [] }
  pool = GỢI BẮT ĐỒNG BỘ poolPromise
  referenceData = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    PartOfSpeeches, Topics]
  partsOfSpeech = referenceData.recordsets[0] OR []
  topics = referenceData.recordsets[1] OR []
  partOfSpeechById = TẠO Map(partsOfSpeech.map(HÀM CALLBACK(item)))
  partOfSpeechByName = TẠO Map()
  topicById = TẠO Map(topics.map(HÀM CALLBACK(item)))
  topicByName = TẠO Map()
  FOR EACH item IN partsOfSpeech
    THỰC HIỆN partOfSpeechByName.set(this.normalizeImportKey(item.name),
      Number(item.id))
    THỰC HIỆN partOfSpeechByName.set(this.normalizeImportKey(item.code),
      Number(item.id))
  END FOR
  FOR EACH item IN topics
    THỰC HIỆN topicByName.set(this.normalizeImportKey(item.name),
      Number(item.id))
    THỰC HIỆN topicByName.set(this.normalizeImportKey(item.code),
      Number(item.id))
  END FOR
  FOR let index = 0; WHILE index < rows.length; index TĂNG 1
    row = rows[index]
    TRY
      term = String(this.getImportValue(row, ["term", "word",
        "vocabulary", "tu vung", "tu"])) NẾU RỎNG THÌ "").trim()
      meaning = String(this.getImportValue(row, ["meaning", "definition",
        "dinh nghĩa", "nghĩa"])) NẾU RỎNG THÌ "").trim()
      phonetic = String(this.getImportValue(row, ["phonetic",
        "pronunciation", "phien am"])) NẾU RỎNG THÌ "").trim()
      rawPartOfSpeechId = this.getImportValue(row, ["partOfSpeechId",
        "posId", "part of speech id", "loai tu id"])
      rawPartOfSpeechName = this.getImportValue(row, ["partOfSpeech",
        "partOfSpeechName", "pos", "part of speech", "loai tu", "tu
        loai"])
      rawTopicIds = this.getImportValue(row, ["topicIds", "topicId",
        "topic ids", "chu de ids", "chu de id"])
      rawTopics = this.getImportValue(row, ["topics", "topic",
        "topicNames", "topicName", "chu de", "ten chu de"])
      rawExampleSentence = this.getImportValue(row, ["exampleSentence",
        "sentence", "example", "cau vi du", "vi du"])
      rawExampleMeaning = this.getImportValue(row, ["exampleMeaning",
        "sentenceTranslation", "translation", "nghĩa cau vi du",
        "dich"])
      IF NOT term OR NOT meaning THEN
        THROW ERROR TẠO Error("Missing required fields: term,
          meaning")
      END IF
      partOfSpeechId = Number(rawPartOfSpeechId)
      IF NOT partOfSpeechId AND rawPartOfSpeechName THEN
        partOfSpeechId =
          partOfSpeechByName.get(this.normalizeImportKey(rawPartOfSpeechName))
      END IF
      IF NOT partOfSpeechId OR NOT
        partOfSpeechById.has(Number(partOfSpeechId)) THEN
        THROW ERROR TẠO Error("Invalid or missing
          partOfSpeechId/partOfSpeech")
      END IF
      topicIds = TẠO Set()
      FOR EACH value IN this.splitImportList(rawTopicIds)
        id = Number(value)

```

```

        IF NOT id OR NOT topicById.has(id) THEN
            THROW ERROR TẠO Error(`Invalid topicId: ${ value }`)
        END IF
        THỰC HIỆN topicIds.add(id)
    END FOR
    FOR EACH value IN this.splitImportList(rawTopics)
        id = Number(value)
        IF id AND topicById.has(id) THEN
            THỰC HIỆN topicIds.add(id)
            CONTINUE
        END IF
        mappedTopicId =
            topicByName.get(this.normalizeImportKey(value))
        IF NOT mappedTopicId THEN
            THROW ERROR TẠO Error(`Invalid topic: ${ value }`)
        END IF
        THỰC HIỆN topicIds.add(mappedTopicId)
    END FOR
    examples = []
    IF String(rawExampleSentence NẾU RỖNG THÌ "").trim() THEN
        THỰC HIỆN examples.push({ sentence:
            String(rawExampleSentence).trim(), meaning:
            String(rawExampleMeaning NẾU RỖNG THÌ "").trim() })
    END IF
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.createWord({ term: term, meaning:
        meaning, phonetic: phonetic, partOfSpeechId:
        Number(partOfSpeechId), topicIds: [...topicIds], examples:
        examples }, adminId)
    results.success = 1
CATCH error
    results.failed = 1
    THỰC HIỆN results.errors.push({ row: index + 2, message:
        error.message })
END TRY
END FOR
RETURN results
END FUNCTION

```

Hàm 169: async previewWordImport(input)

Dòng nguồn: 1165 | Đầu vào: input | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Tiền kiểm dữ liệu CSV import từ vựng, ánh xạ reference data và trả danh sách lỗi theo từng dòng.

```

FUNCTION previewWordImport(input)
    rows = this.parseWordImport(input)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    referenceData = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        PartOfSpeeches, Topics]
    partsOfSpeech = referenceData.recordsets[0] OR []
    topics = referenceData.recordsets[1] OR []
    partOfSpeechById = TẠO Map(partsOfSpeech.map(HÀM CALLBACK(item)))
    partOfSpeechByName = TẠO Map()
    topicById = TẠO Map(topics.map(HÀM CALLBACK(item)))
    topicByName = TẠO Map()
    previewRows = []
    valid = 0
    invalid = 0
    FOR EACH item IN partsOfSpeech
        THỰC HIỆN partOfSpeechByName.set(this.normalizeImportKey(item.name), item)
        THỰC HIỆN partOfSpeechByName.set(this.normalizeImportKey(item.code), item)
    END FOR
    FOR EACH item IN topics
        THỰC HIỆN topicByName.set(this.normalizeImportKey(item.name), item)
        THỰC HIỆN topicByName.set(this.normalizeImportKey(item.code), item)
    END FOR
    FOR let index = 0; WHILE index < rows.length; index TĂNG 1
        row = rows[index]
        errors = []
        term = String(this.getImportValue(row, ["term", "word", "vocabulary", "tu
            vung", "tu"])) NẾU RỖNG THÌ "").trim()

```



```

meaning = String(this.getImportValue(row, ["meaning", "definition", "định nghĩa", "nghĩa"])) NẾU RÕNG THÌ "").trim()
phonetic = String(this.getImportValue(row, ["phonetic", "pronunciation", "phien am"])) NẾU RÕNG THÌ "").trim()
rawPartOfSpeechId = this.getImportValue(row, ["partOfSpeechId", "posId", "part of speech id", "loai tu id"])
rawPartOfSpeechName = this.getImportValue(row, ["partOfSpeech", "partOfSpeechName", "pos", "part of speech", "loai tu", "tu loai"])
rawTopicIds = this.getImportValue(row, ["topicIds", "topicId", "topic ids", "chu de ids", "chu de id"])
rawTopics = this.getImportValue(row, ["topics", "topic", "topicNames", "topicName", "chu de", "ten chu de"])
rawExampleSentence = this.getImportValue(row, ["exampleSentence", "sentence", "example", "cau vi du", "vi du"])
rawExampleMeaning = this.getImportValue(row, ["exampleMeaning", "sentenceTranslation", "translation", "nghia cau vi du", "dich"])
IF NOT term THEN
    THỰC HIỆN errors.push("Missing term")
END IF
IF NOT meaning THEN
    THỰC HIỆN errors.push("Missing meaning")
END IF
partOfSpeech = partOfSpeechById.get(Number(rawPartOfSpeechId))
IF NOT partOfSpeech AND rawPartOfSpeechName THEN
    partOfSpeech =
        partOfSpeechByName.get(this.normalizeImportKey(rawPartOfSpeechName))
END IF
IF NOT partOfSpeech THEN
    THỰC HIỆN errors.push("Invalid or missing partOfSpeech")
END IF
resolvedTopics = []
FOR EACH value IN this.splitImportList(rawTopicIds)
    topic = topicById.get(Number(value))
    IF NOT topic THEN
        THỰC HIỆN errors.push(`Invalid topicId: ${ value }`)
    ELSE
        IF NOT resolvedTopics.some(HÀM CALLBACK(item)) THEN
            THỰC HIỆN resolvedTopics.push(topic)
        END IF
    END IF
END FOR
FOR EACH value IN this.splitImportList(rawTopics)
    topic = topicById.get(Number(value)) OR
        topicByName.get(this.normalizeImportKey(value))
    IF NOT topic THEN
        THỰC HIỆN errors.push(`Invalid topic: ${ value }`)
    ELSE
        IF NOT resolvedTopics.some(HÀM CALLBACK(item)) THEN
            THỰC HIỆN resolvedTopics.push(topic)
        END IF
    END IF
END FOR
IF errors.length > 0 THEN
    invalid = 1
ELSE
    valid = 1
END IF
THỰC HIỆN previewRows.push({ row: index + 2, valid: errors.length = 0,
    errors: errors, term: term, meaning: meaning, phonetic: phonetic,
    partOfSpeech: NẾU partOfSpeech THÌ { id: partOfSpeech.id, name:
    partOfSpeech.name, code: partOfSpeech.code } NGƯỢC LẠI NULL, topics:
    resolvedTopics.map(HÀM CALLBACK(topic)), examples: NẾU
    String(rawExampleSentence NẾU RÕNG THÌ "").trim() THÌ [{ sentence:
    String(rawExampleSentence).trim(), meaning: String(rawExampleMeaning NẾU
    RÕNG THÌ "").trim() }] NGƯỢC LẠI [] })
END FOR
RETURN { total: rows.length, valid: valid, invalid: invalid, rows: previewRows }
END FUNCTION

```


Hàm 170: async getQuestionsByWord(wordId, page = 1, limit = 20, filters = { })

Dòng nguồn: 1264 | Đầu vào: wordId, page = 1, limit = 20, filters = { } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy các câu hỏi của một từ vựng.

```
FUNCTION getQuestionsByWord(wordId, page = 1, limit = 20, filters = { })
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    paging = this.normalizePagination(page, limit, 100)
    search = String(filters.search NẾU RỖNG THÌ "").trim()
    type = String(filters.type NẾU RỖNG THÌ "").trim()
    status = String(filters.status NẾU RỖNG THÌ "").trim()
    conditions = ["WordID = @WordID"]
    request = pool.request().input("WordID", sql.BigInt, wordId).input("Offset",
        sql.Int, paging.offset).input("Limit", sql.Int, paging.limit)
    IF search THEN
        THỰC HIỆN request.input("Search", sql.NVarChar(250), `%${ search }%`)
        THỰC HIỆN conditions.push("(QuestionText LIKE @Search OR CorrectAnswer LIKE
            @Search OR Explanation LIKE @Search)")
    END IF
    IF type THEN
        THỰC HIỆN request.input("QuestionType", sql.NVarChar(30), type)
        THỰC HIỆN conditions.push("QuestionType = @QuestionType")
    END IF
    IF status THEN
        THỰC HIỆN request.input("ContentStatus", sql.NVarChar(30), status)
        THỰC HIỆN conditions.push("ContentStatus = @ContentStatus")
    END IF
    whereClause = `WHERE ${ conditions.join(' AND ') }`
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Questions]
    RETURN this.paginate(result.recordsets[1], result.recordsets[0][0].total OR 0,
        paging.page, paging.limit)
END FUNCTION
```

Hàm 171: async createQuestion(questionData, adminId)

Dòng nguồn: 1309 | Đầu vào: questionData, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới câu hỏi.

```
FUNCTION createQuestion(questionData, adminId)
    { wordId, questionType, questionText, optionsJson = '[]', correctAnswer,
        explanation, status = 'Published' } = questionData
    THỰC HIỆN this.assertContentStatus(status)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Questions]
    created = result.recordset[0]
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId, "CREATE_QUESTION",
        "Question", created.id, { wordId: wordId, questionType: questionType, status:
            status })
    RETURN created
END FUNCTION
```

Hàm 172: async updateQuestion(questionId, questionData, adminId)

Dòng nguồn: 1332 | Đầu vào: questionId, questionData, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật câu hỏi.

```
FUNCTION updateQuestion(questionId, questionData, adminId)
    { wordId, questionType, questionText, optionsJson = '[]', correctAnswer,
        explanation, status = 'Published' } = questionData
    THỰC HIỆN this.assertContentStatus(status)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    oldStatusResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT ContentStatus FROM
        Questions WHERE QuestionID = @QuestionID"
    IF oldStatusResult.recordset.length = 0 THEN
        RETURN false
    END IF
```

```

oldStatus = oldStatusResult.recordset[0].ContentStatus
result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Questions]
IF result.rowsAffected[0] > 0 THEN
    IF oldStatus ≠ status THEN
        THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logContentReview("Question",
            questionId, oldStatus, status, adminId, "Updated from question
            manager")
    END IF
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId, "UPDATE_QUESTION",
        "Question", questionId, { wordId: wordId, questionType: questionType,
        oldStatus: oldStatus, status: status })
END IF
RETURN result.rowsAffected[0] > 0
END FUNCTION

```

Hàm 173: async deleteQuestion(questionId, adminId)

Dòng nguồn: 1380 | Đầu vào: questionId, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa câu hỏi.

```

FUNCTION deleteQuestion(questionId, adminId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        request = TẠO sql.Request(transaction)
        result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Questions,
            MiniTestItems, ExerciseAttempts, MiniTests, mt]
        COMMIT TRANSACTION
        success = result.recordset[0].deleted > 0
        IF success THEN
            THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId,
                "DELETE_QUESTION", "Question", questionId)
        END IF
        RETURN success
    CATCH error
        ROLLBACK TRANSACTION
        THROW ERROR error
    END TRY
END FUNCTION

```

Hàm 174: async getMiniTests(page = 1, limit = 20, filters = { })

Dòng nguồn: 1428 | Đầu vào: page = 1, limit = 20, filters = { } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách bài kiểm tra ngắn.

```

FUNCTION getMiniTests(page = 1, limit = 20, filters = { })
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    paging = this.normalizePagination(page, limit, 100)
    search = String(filters.search NẾU RỖNG THÌ "").trim()
    status = String(filters.status NẾU RỖNG THÌ "").trim()
    topicId = Number(filters.topicId) OR NULL
    conditions = []
    request = pool.request().input("Offset", sql.Int, paging.offset).input("Limit",
        sql.Int, paging.limit)
    IF search THEN
        THỰC HIỆN request.input("Search", sql.NVarChar(250), `%${ search }%`)
        THỰC HIỆN conditions.push("(mt.TestTitle LIKE @Search OR mt.Description LIKE
            @Search OR t.TopicName LIKE @Search)")
    END IF
    IF status THEN
        THỰC HIỆN request.input("ContentStatus", sql.NVarChar(30), status)
        THỰC HIỆN conditions.push("mt.ContentStatus = @ContentStatus")
    END IF
    IF topicId THEN
        THỰC HIỆN request.input("TopicID", sql.BigInt, topicId)
        THỰC HIỆN conditions.push("mt.TopicID = @TopicID")
    END IF

```

```

whereClause = NẾU conditions.length > 0 THÌ `WHERE ${ conditions.join(' AND ' ) }`
NGƯỢC LẠI ""
result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTests,
MiniTestItems, Topics]
items = result.recordsets[1].map(HÀM CALLBACK(test))
RETURN this.paginate(items, result.recordsets[0][0].total OR 0, paging.page,
paging.limit)
END FUNCTION

```

Hàm 175: async createMiniTest(testData, adminId)

Dòng nguồn: 1484 | Đầu vào: testData, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới bài kiểm tra ngắn.

```

FUNCTION createMiniTest(testData, adminId)
{ title, description, topicId, questionIds } = testData
pool = GỌI BẮT ĐỒNG BỘ poolPromise
transaction = TẠO sql.Transaction(pool)
TRY
BẮT ĐẦU TRANSACTION
request = TẠO sql.Request(transaction)
testResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên
MiniTests]
testId = testResult.recordset[0].id
FOR let i = 0; WHILE i < questionIds.length; i TĂNG 1
itemReq = TẠO sql.Request(transaction)
THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên MiniTestItems]
END FOR
COMMIT TRANSACTION
THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId, "CREATE_MINI_TEST",
"MiniTest", testId, { title: title, topicId: topicId, questionCount:
questionIds.length })
RETURN { id: testId, title: title }
CATCH error
ROLLBACK TRANSACTION
THROW ERROR error
END TRY
END FUNCTION

```

Hàm 176: async updateMiniTest(testId, testData, adminId)

Dòng nguồn: 1528 | Đầu vào: testId, testData, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật bài kiểm tra ngắn.

```

FUNCTION updateMiniTest(testId, testData, adminId)
{ title, description, topicId, questionIds } = testData
pool = GỌI BẮT ĐỒNG BỘ poolPromise
transaction = TẠO sql.Transaction(pool)
TRY
BẮT ĐẦU TRANSACTION
updateReq = TẠO sql.Request(transaction)
result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên MiniTests]
IF result.rowsAffected[0] = 0 THEN
ROLLBACK TRANSACTION
RETURN false
END IF
deleteReq = TẠO sql.Request(transaction)
THỰC THI TRUY VẤN "DELETE FROM MiniTestItems WHERE MiniTestID = @MiniTestID"
FOR let i = 0; WHILE i < questionIds.length; i TĂNG 1
itemReq = TẠO sql.Request(transaction)
THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên MiniTestItems]
END FOR
COMMIT TRANSACTION
THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId, "UPDATE_MINI_TEST",
"MiniTest", testId, { title: title, topicId: topicId, questionCount:
questionIds.length })
RETURN true
CATCH error

```

```

ROLLBACK TRANSACTION
THROW ERROR error
END TRY
END FUNCTION

```

Hàm 177: async deleteMiniTest(testId, adminId)

Dòng nguồn: 1583 | Đầu vào: testId, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa bài kiểm tra ngắn.

```

FUNCTION deleteMiniTest(testId, adminId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        request = TẠO sql.Request(transaction)
        result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTests,
            MiniTestAttempts, MiniTestItems]
        COMMIT TRANSACTION
        success = result.recordset[0].deleted > 0
        IF success THEN
            THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId,
                "DELETE_MINI_TEST", "MiniTest", testId)
        END IF
        RETURN success
    CATCH error
        ROLLBACK TRANSACTION
        THROW ERROR error
    END TRY
END FUNCTION

```

Hàm 178: async setMiniTestStatus(testId, status, adminId, comment = null)

Dòng nguồn: 1619 | Đầu vào: testId, status, adminId, comment = null | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý set mini test status trong module backend.

```

FUNCTION setMiniTestStatus(testId, status, adminId, comment = null)
    THỰC HIỆN this.assertContentStatus(status)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    oldStatusResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT ContentStatus FROM
        MiniTests WHERE MiniTestID = @MiniTestID"
    IF oldStatusResult.recordset.length = 0 THEN
        RETURN false
    END IF
    oldStatus = oldStatusResult.recordset[0].ContentStatus
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên MiniTests]
    IF result.rowsAffected[0] > 0 THEN
        THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logContentReview("MiniTest", testId,
            oldStatus, status, adminId, comment)
        THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId,
            "UPDATE_MINI_TEST_STATUS", "MiniTest", testId, { oldStatus: oldStatus,
            status: status, comment: comment })
    END IF
    RETURN result.rowsAffected[0] > 0
END FUNCTION

```

Hàm 179: async getDashboardStats()

Dòng nguồn: 1651 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy thông kê dashboard.

```

FUNCTION getDashboardStats()
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Users, Words,
        Topics, Questions, ExerciseAttempts]
    RETURN { ...result.recordsets[0][0], systemHealth: { apiStatus: "OK",

```

```

        databaseStatus: "Connected", environment: process.env.NODE_ENV OR
        "development",
        uptimeSeconds: Math.round(process.uptime()) }, userGrowth:
        result.recordsets[1],
        weeklyActivity: result.recordsets[2], userTypes: result.recordsets[3],
        recentActivity: result.recordsets[4] }
    END FUNCTION

```

Hàm 180: async getStudents(page = 1, limit = 20, filters = { })

Dòng nguồn: 1723 | Đầu vào: page = 1, limit = 20, filters = { } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách người dùng/học viên.

```

FUNCTION getStudents(page = 1, limit = 20, filters = { })
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    paging = this.normalizePagination(page, limit, 100)
    search = String(filters.search NẾU RỖNG THÌ "").trim()
    status = String(filters.status NẾU RỖNG THÌ "").trim()
    role = String(filters.role NẾU RỖNG THÌ "").trim()
    conditions = []
    request = pool.request().input("Offset", sql.Int, paging.offset).input("Limit",
        sql.Int, paging.limit)
    IF search THEN
        THỰC HIỆN request.input("Search", sql.NVarChar(250), `%${ search }%`)
        THỰC HIỆN conditions.push("(u.FullName LIKE @Search OR u.Email LIKE @Search
            OR u.UserRole LIKE @Search OR r.RoleName LIKE @Search)")
    END IF
    IF status = "active" OR status = "banned" THEN
        THỰC HIỆN request.input("IsActive", sql.Bit, status = "active")
        THỰC HIỆN conditions.push("u.IsActive = @IsActive")
    END IF
    IF role THEN
        THỰC HIỆN request.input("RoleName", sql.NVarChar(50), role)
        THỰC HIỆN conditions.push("u.UserRole = @RoleName")
    END IF
    whereClause = NẾU conditions.length > 0 THÌ `WHERE ${ conditions.join(' AND ' ) }`
    NGƯỢC LẠI ""
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Users,
        UserWordProgress, ExerciseAttempts, Roles]
    RETURN this.paginate(result.recordsets[1], result.recordsets[0][0].total OR 0,
        paging.page, paging.limit)
END FUNCTION

```

Hàm 181: async createUser(userData)

Dòng nguồn: 1771 | Đầu vào: userData | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới user.

```

FUNCTION createUser(userData)
    { fullName, email, password, role = 'Learner', isActive = true } = userData
    THỰC HIỆN assertValidUserRole(role)
    IF NOT fullName OR NOT email OR NOT password OR password.length < 6 THEN
        THROW ERROR TẠO Error("Invalid user data")
    END IF
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    existing = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT UserID FROM Users WHERE
    Email =
        @Email"
    IF existing.recordset.length > 0 THEN
        THROW ERROR TẠO Error("Email already exists")
    END IF
    passwordHash = GỌI BẮT ĐỒNG BỘ bcrypt.hash(password, 10)
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Roles, Users]
    RETURN result.recordset[0]
END FUNCTION

```

Hàm 182: async bulkInsertQuestions(input, adminId)

Dòng nguồn: 1817 | Đầu vào: input, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để ghi hàng loạt danh sách câu hỏi.

```
FUNCTION bulkInsertQuestions(input, adminId)
    rows = this.parseQuestionImport(input)
    results = { success: 0, failed: 0, errors: [] }
    FOR let index = 0; WHILE index < rows.length; index TĂNG 1
        row = rows[index]
        wordId = Number(row.wordId NẾU RỖNG THÌ row.WordID NẾU RỖNG THÌ row.wordID)
        questionType = row.questionType NẾU RỖNG THÌ row.QuestionType
        questionText = row.questionText NẾU RỖNG THÌ row.QuestionText
        correctAnswer = row.correctAnswer NẾU RỖNG THÌ row.CorrectAnswer
        optionsJson = row.optionsJson NẾU RỖNG THÌ row.OptionsJson NẾU RỖNG THÌ "[]"
        explanation = row.explanation NẾU RỖNG THÌ row.Explanation NẾU RỖNG THÌ NULL
        status = row.status NẾU RỖNG THÌ row.ContentStatus NẾU RỖNG THÌ "Published"
        TRY
            IF NOT wordId OR NOT questionType OR NOT questionText OR NOT
                correctAnswer THEN
                THROW ERROR TẠO Error("Missing required fields: wordId,
                    questionType, questionText, correctAnswer")
            END IF
            THỰC HIỆN JSON.parse(optionsJson OR "[]")
            THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.createQuestion({ wordId: wordId,
                questionType: questionType, questionText: questionText,
                optionsJson: optionsJson OR "[]", correctAnswer: correctAnswer,
                explanation: explanation, status: status }, adminId)
            results.success = 1
        CATCH error
            results.failed = 1
            THỰC HIỆN results.errors.push({ row: index + 2, message:
                error.message })
        END TRY
    END FOR
    RETURN results
END FUNCTION
```

Hàm 183: async updateUser(userId, userData)

Dòng nguồn: 1861 | Đầu vào: userId, userData | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật user.

```
FUNCTION updateUser(userId, userData)
    { fullName, email, password, role = 'Learner', isActive = true } = userData
    THỰC HIỆN assertValidUserRole(role)
    IF NOT fullName OR NOT email THEN
        THROW ERROR TẠO Error("Invalid user data")
    END IF
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    existing = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT UserID FROM Users WHERE
    Email =
        @Email AND UserID <> @UserID"
    IF existing.recordset.length > 0 THEN
        THROW ERROR TẠO Error("Email already exists")
    END IF
    request = pool.request().input("UserID", sql.BigInt, userId).input("FullName",
        sql.NVarChar(200), fullName).input("Email", sql.NVarChar(255),
        email).input("RoleName", sql.NVarChar(50), role).input("IsActive", sql.Bit,
        Boolean(isActive))
    passwordUpdate = ""
    IF password THEN
        IF password.length < 6 THEN
            THROW ERROR TẠO Error("Invalid user data")
        END IF
        passwordHash = GỌI BẮT ĐỒNG BỘ bcrypt.hash(password, 10)
        THỰC HIỆN request.input("PasswordHash", sql.NVarChar(500), passwordHash)
        passwordUpdate = ", PasswordHash = @PasswordHash"
```

```

END IF
result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Roles, Users]
RETURN result.rowsAffected.some(HÀM CALLBACK(count))
END FUNCTION

```

Hàm 184: async deleteUser(userId)

Dòng nguồn: 1924 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa user.

```

FUNCTION deleteUser(userId)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  dependencies = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words,
    Questions, MiniTests, Topics]
  ownedContent = dependencies.recordset[0]
  IF ownedContent.words OR ownedContent.questions OR ownedContent.miniTests OR
    ownedContent.topics THEN
    THROW ERROR TẠO Error("User owns content")
  END IF
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "DELETE FROM Users WHERE UserID =
    @UserID"
  RETURN result.rowsAffected[0] > 0
END FUNCTION

```

Hàm 185: async toggleUserStatus(userId)

Dòng nguồn: 1948 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để bật/tắt user status.

```

FUNCTION toggleUserStatus(userId)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  THỰC THI TRUY VẤN "UPDATE Users SET IsActive = CASE WHEN IsActive = 1 THEN 0 ELSE
1
  END, UpdatedAt = SYSDATETIMEOFFSET() WHERE UserID = @UserID"
  RETURN true
END FUNCTION

```

Hàm 186: async updateUserRole(userId, roleName)

Dòng nguồn: 1956 | Đầu vào: userId, roleName | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật vai trò người dùng.

```

FUNCTION updateUserRole(userId, roleName)
  THỰC HIỆN assertValidUserRole(roleName)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Roles, Users]
  RETURN result.rowsAffected.some(HÀM CALLBACK(count))
END FUNCTION

```

Hàm 187: async updateContentStatus({ entityType, entityId, status, comment }, adminId)

Dòng nguồn: 1980 | Đầu vào: { entityType, entityId, status, comment }, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật trạng thái nội dung.

```

FUNCTION updateContentStatus({ entityType, entityId, status, comment }, adminId)
  THỰC HIỆN this.assertContentStatus(status)
  tableMap = { Topic: { table: "Topics", id: "TopicID" }, Word: { table: "Words",
id:
  "WordID" }, Question: { table: "Questions", id: "QuestionID" }, MiniTest: {
  table: "MiniTests", id: "MiniTestID", publishColumn: true } }
  target = tableMap[entityType]
  IF NOT target THEN
    THROW ERROR TẠO Error("Invalid entity type")
  END IF
  pool = GỌI BẮT ĐỒNG BỘ poolPromise

```



```

oldStatusResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT]
IF oldStatusResult.recordset.length = 0 THEN
    RETURN false
END IF
oldStatus = oldStatusResult.recordset[0].ContentStatus
publishFragment = NẾU target.publishColumn THÌ ", IsPublished = CASE WHEN
    @ContentStatus = N'Published' THEN 1 ELSE 0 END" NGƯỢC LẠI ""
result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE]
IF result.rowsAffected[0] > 0 THEN
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logContentReview(entityType, entityId,
        oldStatus, status, adminId, comment)
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.logAdminAction(adminId,
        "UPDATE_CONTENT_STATUS", entityType, entityId, { oldStatus: oldStatus,
        status: status, comment: comment })
END IF
RETURN result.rowsAffected[0] > 0
END FUNCTION

```

Hàm 188: async getPendingContent()

Dòng nguồn: 2027 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách nội dung chờ duyệt.

```

FUNCTION getPendingContent()
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, Words,
        Questions, MiniTests, Users]
    RETURN result.recordset
END FUNCTION

```

Hàm 189: async getContentReviewLogs(entityType, entityId)

Dòng nguồn: 2092 | Đầu vào: entityType, entityId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy content review logs.

```

FUNCTION getContentReviewLogs(entityType, entityId)
    validEntityTypes = ["Topic", "Word", "Question", "MiniTest"]
    IF NOT validEntityTypes.includes(entityType) THEN
        THROW ERROR TẠO Error("Invalid entity type")
    END IF
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        ContentReviewLogs,
        Users]
    RETURN result.recordset
END FUNCTION

```

Hàm 190: async getAnalyticsData()

Dòng nguồn: 2133 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy analytics data.

```

FUNCTION getAnalyticsData()
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        ExerciseAttempts,
        Questions, PartOfSpeeches, Words, WordTopics]
    RETURN { summary: result.recordsets[0][0], popularQuizzes: result.recordsets[1],
        questionAccuracy: result.recordsets[2], studyActivity: result.recordsets[3],
        wordDistribution: result.recordsets[4], difficultTopics: result.recordsets[5]
    }
END FUNCTION

```


Hàm 191: async getContentManagementData()

Dòng nguồn: 2202 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy content management data.

```
FUNCTION getContentManagementData()
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, Words,
        Questions, MiniTests, TopicCategories]
    RETURN { summary: result.recordsets[0][0], content: result.recordsets[1],
        categories: result.recordsets[2], reviewLogs: result.recordsets[3] }
END FUNCTION
```

Hàm 192: async getNotifications(page = 1, limit = 50, filters = { })

Dòng nguồn: 2319 | Đầu vào: page = 1, limit = 50, filters = { } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách thông báo.

```
FUNCTION getNotifications(page = 1, limit = 50, filters = { })
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    paging = this.normalizePagination(page, limit, 100)
    tableExists = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT]
    IF NOT tableExists.recordset[0].tableId THEN
        RETURN this.paginate([], 0, paging.page, paging.limit)
    END IF
    search = String(filters.search NẾU RỖNG THÌ "").trim()
    type = String(filters.type NẾU RỖNG THÌ "").trim()
    deliveryChannel = String(filters.deliveryChannel NẾU RỖNG THÌ "").trim()
    isRead = NẾU filters.isRead = undefined OR filters.isRead = "" THÌ NULL NGƯỢC LẠI
        filters.isRead = true OR filters.isRead = "true" OR filters.isRead = "1"
    conditions = []
    request = pool.request().input("Offset", sql.Int, paging.offset).input("Limit",
        sql.Int, paging.limit)
    IF search THEN
        THỰC HIỆN request.input("Search", sql.NVarChar(250), `${ search }`)
        THỰC HIỆN conditions.push("(n.Title LIKE @Search OR n.Message LIKE @Search
            OR u.FullName LIKE @Search OR u.Email LIKE @Search)")
    END IF
    IF type THEN
        THỰC HIỆN request.input("Type", sql.NVarChar(50), type)
        THỰC HIỆN conditions.push("n.Type = @Type")
    END IF
    IF deliveryChannel THEN
        THỰC HIỆN request.input("DeliveryChannel", sql.NVarChar(20),
            deliveryChannel)
        THỰC HIỆN conditions.push("n.DeliveryChannel = @DeliveryChannel")
    END IF
    IF isRead ≠ NULL THEN
        THỰC HIỆN request.input("IsRead", sql.Bit, isRead)
        THỰC HIỆN conditions.push("n.IsRead = @IsRead")
    END IF
    whereClause = NẾU conditions.length > 0 THÌ `WHERE ${ conditions.join(' AND ') }`
        NGƯỢC LẠI ""
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Notifications,
        Users]
    RETURN this.paginate(result.recordsets[1], result.recordsets[0][0].total OR 0,
        paging.page, paging.limit)
END FUNCTION
```

Hàm 193: async getAuditLogs(page = 1, limit = 50, filters = { })

Dòng nguồn: 2389 | Đầu vào: page = 1, limit = 50, filters = { } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy nhật ký audit quản trị.

```
FUNCTION getAuditLogs(page = 1, limit = 50, filters = { })
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
```

```

paging = this.normalizePagination(page, limit, 100)
tableExists = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT]
IF NOT tableExists.recordset[0].tableId THEN
    RETURN this.paginate([], 0, paging.page, paging.limit)
END IF
search = String(filters.search NẾU RỘNG THÌ "").trim()
action = String(filters.action NẾU RỘNG THÌ "").trim()
entityType = String(filters.entityType NẾU RỘNG THÌ "").trim()
adminId = Number(filters.adminId) OR NULL
conditions = []
request = pool.request().input("Offset", sql.Int, paging.offset).input("Limit",
    sql.Int, paging.limit)
IF search THEN
    THỰC HIỆN request.input("Search", sql.NVarChar(250), `%${ search }%`)
    THỰC HIỆN conditions.push("(1.Action LIKE @Search OR 1.EntityType LIKE
        @Search OR 1.Details LIKE @Search OR u.FullName LIKE @Search OR u.Email
        LIKE @Search)")
END IF
IF action THEN
    THỰC HIỆN request.input("Action", sql.NVarChar(100), action)
    THỰC HIỆN conditions.push("1.Action = @Action")
END IF
IF entityType THEN
    THỰC HIỆN request.input("EntityType", sql.NVarChar(50), entityType)
    THỰC HIỆN conditions.push("1.EntityType = @EntityType")
END IF
IF adminId THEN
    THỰC HIỆN request.input("AdminID", sql.BigInt, adminId)
    THỰC HIỆN conditions.push("1.ActionByUserID = @AdminID")
END IF
whereClause = NẾU conditions.length > 0 THÌ `WHERE ${ conditions.join(' AND ' ) }`
NGƯỢC LẠI ""
result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên AdminAuditLogs,
    Users]
RETURN this.paginate(result.recordsets[1], result.recordsets[0][0].total OR 0,
    paging.page, paging.limit)
END FUNCTION

```

Hàm 194: `async sendAnnouncement({ audience = 'All users', title, message, deliveryChannel = 'InApp', actionUrl = null })`

Dòng nguồn: 2456 | Đầu vào: { audience = 'All users', title, message, deliveryChannel = 'InApp', actionUrl = null } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý send announcement trong module backend.

```

FUNCTION sendAnnouncement({ audience = 'All users', title, message, deliveryChannel =
    'InApp', actionUrl = null })
IF NOT title OR NOT message THEN
    THROW ERROR TẠO Error("Missing title or message")
END IF
pool = GỌI BẮT ĐỒNG BỘ poolPromise
result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Users,
    Notifications]
RETURN { inserted: result.recordset[0].inserted }
END FUNCTION

```

Hàm 195: `async createDailyReminders()`

Dòng nguồn: 2494 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới daily reminders.

```

FUNCTION createDailyReminders()
pool = GỌI BẮT ĐỒNG BỘ poolPromise
result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Users,
    UserWordProgress, Notifications]
RETURN { inserted: result.recordset[0].inserted }
END FUNCTION

```

services/ai.service.js

File này có 9 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 196: withTimeout(ms = 12000)

Dòng nguồn: 5 | Đầu vào: ms = 12000 | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý with timeout trong module backend.

```
FUNCTION withTimeout(ms = 12000)
  controller = TẠO AbortController()
  timeout = setTimeout(HÀM CALLBACK(), ms)
  RETURN { signal: controller.signal, clear: HÀM CALLBACK() }
END FUNCTION
```

Hàm 197: cleanText(value)

Dòng nguồn: 11 | Đầu vào: value | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý clean text trong module backend.

```
FUNCTION cleanText(value)
  RETURN NẾU typeofvalue = "string" THÌ value.trim() NGƯỢC LẠI ""
END FUNCTION
```

Hàm 198: extractResponseText(response)

Dòng nguồn: 15 | Đầu vào: response | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý extract response text trong module backend.

```
FUNCTION extractResponseText(response)
  IF typeofresponse.output_text = "string" THEN
    RETURN response.output_text
  END IF
  chunks = []
  FOR EACH item IN response.output OR []
    IF item.type ≠ "message" THEN
      CONTINUE
    END IF
    FOR EACH content IN item.content OR []
      IF typeofcontent = "string" THEN
        THỰC HIỆN chunks.push(content)
      END IF
      IF typeofcontent.text = "string" THEN
        THỰC HIỆN chunks.push(content.text)
      END IF
    END FOR
  END FOR
  RETURN chunks.join("\n").trim()
END FUNCTION
```

Hàm 199: parseJsonText(text)

Dòng nguồn: 30 | Đầu vào: text | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để phân tích và chuẩn hóa json text.

```
FUNCTION parseJsonText(text)
  raw = cleanText(text)
  IF NOT raw THEN
    THROW ERROR TẠO Error("AI response is empty")
  END IF
  TRY
    RETURN JSON.parse(raw)
  CATCH error
    match = raw.match(/\{ [\s\S]* \}/)
    IF NOT match THEN
      THROW ERROR TẠO Error("AI response is not valid JSON")
    END IF
    RETURN JSON.parse(match[0])
  END TRY
```

```
END TRY
END FUNCTION
```

Hàm 200: normalizeExamples(examples)

Dòng nguồn: 43 | Đầu vào: *examples* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để chuẩn hóa examples.

```
FUNCTION normalizeExamples(examples)
  IF NOT Array.isArray(examples) THEN
    RETURN []
  END IF
  RETURN examples.map(HÀM CALLBACK(example)).filter(HÀM CALLBACK(example)).slice(0,
5)
END FUNCTION
```

Hàm 201: async suggestWordContent({ term, meaning, partOfSpeech, exampleCount = 3 })

Dòng nguồn: 56 | Đầu vào: { term, meaning, partOfSpeech, exampleCount = 3 } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Điều phối tra từ điển, dịch nghĩa và gọi OpenAI để gợi ý nội dung từ vựng TOEIC hoàn chỉnh.

```
FUNCTION suggestWordContent({ term, meaning, partOfSpeech, exampleCount = 3 })
  cleanTerm = cleanText(term)
  IF NOT cleanTerm THEN
    THROW ERROR TẠO Error("Missing term")
  END IF
  [dictionary, translatedTerm] = GỌI BẮT ĐỒNG BỘ
    Promise.all([this.lookupDictionary(cleanTerm).catch(HÀM CALLBACK()),
    this.translateToVietnamese(cleanTerm).catch(HÀM CALLBACK())])
  aiSuggestion = GỌI BẮT ĐỒNG BỘ this.generateToeicExamples({ term: cleanTerm,
  meaning: cleanText(meaning) OR translatedTerm OR dictionary.meaning OR "",
  partOfSpeech: cleanText(partOfSpeech) OR dictionary.partOfSpeech OR "",
  exampleCount: exampleCount })
  RETURN { term: cleanTerm, meaning: aiSuggestion.meaning OR translatedTerm OR
  dictionary.meaning OR "", phonetic: dictionary.phonetic OR
  aiSuggestion.phonetic
  OR "", partOfSpeech: aiSuggestion.partOfSpeech OR dictionary.partOfSpeech OR
  "",
  examples: NẾU aiSuggestion.examples.length > 0 THÌ aiSuggestion.examples
  NGƯỢC
  LẠI dictionary.examples OR [], sources: { ai: "openai", translate: NẾU
  translatedTerm THÌ "google-translate" NGƯỢC LẠI NULL, dictionary: NẾU
  dictionary
  THÌ "dictionaryapi.dev" NGƯỢC LẠI NULL } }
END FUNCTION
```

Hàm 202: async generateToeicExamples({ term, meaning, partOfSpeech, exampleCount })

Dòng nguồn: 86 | Đầu vào: { term, meaning, partOfSpeech, exampleCount } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý generate toeic examples trong module backend.

```
FUNCTION generateToeicExamples({ term, meaning, partOfSpeech, exampleCount })
  apiKey = process.env.OPENAI_API_KEY
  IF NOT apiKey THEN
    error = TẠO Error("OPENAI_API_KEY is not configured")
    error.statusCode = 503
    THROW ERROR error
  END IF
  model = process.env.OPENAI_MODEL OR process.env.AI_MODEL OR "gpt-5.4-mini"
  { signal, clear } = setTimeout(20000)
  TRY
    response = GỌI BẮT ĐỒNG BỘ fetch(OPENAI_RESPONSES_URL, { method: "POST",
    signal: signal, headers: { Authorization: `Bearer ${ apiKey }`,
```

```

Content-Type: "application/json" }, body: JSON.stringify({ model: model,
instructions: ["You create high-quality TOEIC vocabulary learning
content for Vietnamese learners.", "Return JSON only.Do not wrap the
JSON in markdown.", "Examples must be natural business, office, travel,
meeting, email, or customer-service TOEIC contexts.", "Each example
sentence must contain the target word exactly once when
natural."].join(" "), input: JSON.stringify({ term: term,
knownVietnameseMeaning: meaning, knownPartOfSpeech: partOfSpeech,
exampleCount: exampleCount, outputShape: { meaning: "Vietnamese meaning,
concise", phonetic: "IPA if confident, otherwise empty string",
partOfSpeech: "English part of speech if confident", examples: [{
sentence: "English TOEIC sentence", meaning: "Vietnamese translation" }]
} })), max_output_tokens: 900 }) })
data = GỌI BẮT ĐỒNG BỘ response.json().catch(HÀM CALLBACK())
IF NOT response.ok THEN
    message = data.error.message OR `OpenAI request failed with status
    ${ response.status }`
    error = TẠO Error(message)
    error.upstreamStatusCode = response.status
    error.statusCode = NẾU response.status = 401 OR response.status =
    403 THÌ 502 NGUỘC LẠI response.status
    THROW ERROR error
END IF
parsed = parseJsonText(extractResponseText(data))
RETURN { meaning: cleanText(parsed.meaning), phonetic:
    cleanText(parsed.phonetic), partOfSpeech:
    cleanText(parsed.partOfSpeech), examples:
    normalizeExamples(parsed.examples) }
FINALLY
    THỰC HIỆN clear()
END TRY
END FUNCTION

```

Hàm 203: async translateToVietnamese(text)

Dòng nguồn: 150 | Đầu vào: text | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý translate to vietnamese trong module backend.

```

FUNCTION translateToVietnamese(text)
    apiKey = process.env.GOOGLE_TRANSLATE_API_KEY
    IF NOT apiKey THEN
        RETURN NULL
    END IF
    { signal, clear } = setTimeout(8000)
    TRY
        response = GỌI BẮT ĐỒNG BỘ fetch(`${ GOOGLE_TRANSLATE_URL }?key=${
            encodeURIComponent(apiKey) }`, { method: "POST", signal: signal,
            headers: { Content-Type: "application/json" }, body: JSON.stringify({ q:
            text, source: "en", target: "vi", format: "text" }) })
        data = GỌI BẮT ĐỒNG BỘ response.json().catch(HÀM CALLBACK())
        IF NOT response.ok THEN
            RETURN NULL
        END IF
        RETURN cleanText(data.data.translations[0].translatedText) OR NULL
    FINALLY
        THỰC HIỆN clear()
    END TRY
END FUNCTION

```

Hàm 204: async lookupDictionary(term)

Dòng nguồn: 176 | Đầu vào: term | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý lookup dictionary trong module backend.

```

FUNCTION lookupDictionary(term)
    { signal, clear } = setTimeout(8000)
    TRY
        response = GỌI BẮT ĐỒNG BỘ fetch(`${ DICTIONARY_URL }/${
            encodeURIComponent(term) }`, { signal: signal })
        IF NOT response.ok THEN
            RETURN NULL
        END IF
    END TRY
END FUNCTION

```

```

END IF
entries = GỌI BẮT ĐỒNG BỘ response.json()
firstEntry = NẾU Array.isArray(entries) THÌ entries[0] NGƯỢC LẠI NULL
IF NOT firstEntry THEN
    RETURN NULL
END IF
phonetic = cleanText(firstEntry.phonetic) OR cleanText(firstEntry.phonetics
    OR []).find(HÀM CALLBACK(item)).text)
firstMeaning = firstEntry.meanings OR [][0]
firstDefinition = firstMeaning.definitions OR [][0]
examples = []
FOR EACH meaning IN firstEntry.meanings OR []
    FOR EACH definition IN meaning.definitions OR []
        IF definition.example THEN
            THỰC HIỆN examples.push({ sentence:
                definition.example, meaning: "" })
        END IF
    END FOR
END FOR
RETURN { phonetic: phonetic, partOfSpeech:
    cleanText(firstMeaning.partOfSpeech), meaning:
    cleanText(firstDefinition.definition), examples:
    normalizeExamples(examples) }
FINALLY
    THỰC HIỆN clear()
END TRY
END FUNCTION

```

services/auth.service.js

File này có 2 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 205: async register(fullName, email, password)

Dòng nguồn: 7 | Đầu vào: fullName, email, password | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Kiểm tra trùng email, băm mật khẩu, gán role Learner mặc định và tạo tài khoản mới.

```

FUNCTION register(fullName, email, password)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    checkUser = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Users]
    IF checkUser.recordset.length > 0 THEN
        THROW ERROR TẠO Error("Email đã tồn tại")
    END IF
    hashedPassword = GỌI BẮT ĐỒNG BỘ bcrypt.hash(password, 10)
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Roles, Users]
    user = result.recordset[0]
    RETURN { ...user, role: "Learner", permissions: ["VIEW_DASHBOARD", "LEARN_VOCAB"]
}
END FUNCTION

```

Hàm 206: async login(email, password)

Dòng nguồn: 40 | Đầu vào: email, password | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xác thực email/mật khẩu, tải quyền từ RolePermissions, sinh JWT và cộng điểm đăng nhập hằng ngày cho learner.

```

FUNCTION login(email, password)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Users, Roles]
    user = result.recordset[0]
    IF NOT user THEN
        THROW ERROR TẠO Error("Email hoặc mật khẩu không chính xác")
    END IF
    isMatch = GỌI BẮT ĐỒNG BỘ bcrypt.compare(password, user.passwordHash)
    IF NOT isMatch THEN
        THROW ERROR TẠO Error("Email hoặc mật khẩu không chính xác")
    END IF

```

```

permResult = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    RolePermissions, Permissions, Users]
permissions = permResult.recordset.map(HÀM CALLBACK(r))
payload = { id: user.id, fullName: user.fullName, role: user.role, permissions:
    permissions }
token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: "1d" })
gamification = NULL
IF user.role = "Learner" THEN
    TRY
        gamification = GỢI BẮT ĐỒNG BỘ
            GamificationService.awardDailyLogin(user.id)
    CATCH error
        THỰC HIỆN console.error("[AuthService.login] Failed to award daily
            login XP: ", error)
    END TRY
END IF
RETURN { token: token, gamification: gamification, user: { id: user.id, fullName:
    user.fullName, email: user.email, role: user.role, permissions: permissions }
}
END FUNCTION

```

services/categories.service.js

File này có 2 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 207: async getPartOfSpeeches()

Dòng nguồn: 5 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh mục từ loại.

```

FUNCTION getPartOfSpeeches()
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    result = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên PartOfSpeeches]
    RETURN result.recordset
END FUNCTION

```

Hàm 208: async getTopics(userId = null)

Dòng nguồn: 13 | Đầu vào: userId = null | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách chủ đề.

```

FUNCTION getTopics(userId = null)
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    result = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics,
        WordTopics,
        Words, UserWordProgress]
    RETURN result.recordset
END FUNCTION

```

services/creator.service.js

File này có 32 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 209: parseQuestionOptions(questionType, optionsJson, correctAnswer)

Dòng nguồn: 4 | Đầu vào: questionType, optionsJson, correctAnswer | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để phân tích và chuẩn hóa question options.

```

FUNCTION parseQuestionOptions(questionType, optionsJson, correctAnswer)
    options = JSON.parse(optionsJson OR "[]")
    IF NOT Array.isArray(options) THEN
        THROW ERROR TẠO Error("Question options must be an array")
    END IF
    IF questionType = "MCQ" THEN
        normalizedOptions = [...new Set(options.map((option) =>

```



```

        String(option).trim()).filter(Boolean))
    IF normalizedOptions.length < 2 THEN
        THROW ERROR TẠO Error("MCQ requires at least two options")
    END IF
    IF NOT normalizedOptions.includes(String(correctAnswer).trim()) THEN
        THROW ERROR TẠO Error("Correct answer must match an MCQ option")
    END IF
    RETURN normalizedOptions
END IF
RETURN options
END FUNCTION

```

Hàm 210: `async validateMiniTestReferences(transaction, topicId, questionIds, userId)`

Dòng nguồn: 22 | Đầu vào: transaction, topicId, questionIds, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý validate mini test references trong module backend.

```

FUNCTION validateMiniTestReferences(transaction, topicId, questionIds, userId)
    rawQuestionIds = NẾU Array.isArray(questionIds) THÌ questionIds NGƯỢC LẠI []
    normalizedQuestionIds = [...new Set(rawQuestionIds)]
    IF normalizedQuestionIds.length ≠ rawQuestionIds.length THEN
        THROW ERROR TẠO Error("Mini test contains duplicate questions")
    END IF
    IF NOT normalizedQuestionIds.length THEN
        THROW ERROR TẠO Error("Mini test requires at least one question")
    END IF
    IF topicId THEN
        topicRequest = TẠO sql.Request(transaction)
        topic = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics]
        IF NOT topic.recordset.length THEN
            THROW ERROR TẠO Error("Topic not found or unavailable")
        END IF
    END IF
    FOR EACH questionId IN normalizedQuestionIds
        questionRequest = TẠO sql.Request(transaction)
        question = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Questions]
        IF NOT question.recordset.length THEN
            THROW ERROR TẠO Error("Mini test questions must be owned and published")
        END IF
    END FOR
    RETURN normalizedQuestionIds
END FUNCTION

```

Hàm 211: `async getDashboardStats(userId)`

Dòng nguồn: 65 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy thông kê dashboard.

```

FUNCTION getDashboardStats(userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, Words, Questions, MiniTests]
    RETURN result.recordset[0] OR { }
END FUNCTION

```

Hàm 212: `async getContentSummary(userId)`

Dòng nguồn: 95 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy tổng hợp trạng thái nội dung.

```

FUNCTION getContentSummary(userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, Words, Questions, MiniTests]
    RETURN result.recordset
END FUNCTION

```


Hàm 213: async getTopicAnalytics(userId, topicId)

Dòng nguồn: 115 | Đầu vào: *userId, topicId* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy thống kê theo chủ đề.

```
FUNCTION getTopicAnalytics(userId, topicId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        vw_TopicLearningAnalytics, Topics]
    RETURN result.recordset[0] OR NULL
END FUNCTION
```

Hàm 214: async getMiniTestAnalytics(userId, miniTestId)

Dòng nguồn: 129 | Đầu vào: *userId, miniTestId* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy thống kê bài kiểm tra ngắn.

```
FUNCTION getMiniTestAnalytics(userId, miniTestId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        vw_MiniTestAnalytics, MiniTests]
    RETURN result.recordset[0] OR NULL
END FUNCTION
```

Hàm 215: async getTopicCategories()

Dòng nguồn: 144 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh mục chủ đề.

```
FUNCTION getTopicCategories()
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên TopicCategories]
    RETURN result.recordset
END FUNCTION
```

Hàm 216: async getMyTopics(userId, filters = { })

Dòng nguồn: 158 | Đầu vào: *userId, filters = { }* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy my topics.

```
FUNCTION getMyTopics(userId, filters = { })
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    page = Math.max(1, CHUYỂN SANG SỐ NGUYÊN(filters.page) OR 1)
    pageSize = Math.min(100, Math.max(1, CHUYỂN SANG SỐ NGUYÊN(filters.pageSize) OR
50))
    offset = page - 1 * pageSize
    req = pool.request().input("UserID", sql.BigInt, userId)
    where = "t.CreatedByUserID = @UserID"
    IF filters.status THEN
        THỰC HIỆN req.input("Status", sql.NVarChar(20), filters.status)
        where = " AND t.ContentStatus = @Status"
    END IF
    search = String(filters.search OR "").trim()
    IF search THEN
        THỰC HIỆN req.input("Search", sql.NVarChar(250), `>${search}<`)
        where = " AND (t.TopicName LIKE @Search OR t.TopicCode LIKE @Search OR
            t.Description LIKE @Search)"
    END IF
    countResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics]
    total = countResult.recordset[0].total
    req2 = pool.request().input("UserID", sql.BigInt, userId).input("Offset",
sql.Int,
    offset).input("PageSize", sql.Int, pageSize)
    where2 = "t.CreatedByUserID = @UserID"
```

```

IF filters.status THEN
    THỰC HIỆN req2.input("Status", sql.NVarChar(20), filters.status)
    where2 = " AND t.ContentStatus = @Status"
END IF
IF search THEN
    THỰC HIỆN req2.input("Search", sql.NVarChar(250), `>${ search }`)
    where2 = " AND (t.TopicName LIKE @Search OR t.TopicCode LIKE @Search OR
        t.Description LIKE @Search)"
END IF
result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
ContentReviewLogs,
Topics, TopicCategories]
RETURN { data: result.recordset, total: total, page: page, pageSize: pageSize,
totalPages: Math.ceil(total / pageSize) }
END FUNCTION

```

Hàm 217: async createTopic(data, userId)

Dòng nguồn: 214 | Đầu vào: data, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới chủ đề.

```

FUNCTION createTopic(data, userId)
{ topicName, topicCode, description, topicCategoryId } = data
pool = GỌI BẮT ĐỒNG BỘ poolPromise
IF topicCategoryId THEN
    category = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT TopicCategoryId FROM
        TopicCategories WHERE TopicCategoryId = @TopicCategoryId AND IsActive =
        1"
    IF NOT category.recordset.length THEN
        THROW ERROR TẠO Error("Topic category not found")
    END IF
END IF
duplicate = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT TopicID FROM Topics WHERE
    TopicName = @TopicName OR TopicCode = @TopicCode"
IF duplicate.recordset.length THEN
    THROW ERROR TẠO Error("Topic name or code already exists")
END IF
result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Topics]
RETURN { id: result.recordset[0].id, topicName: topicName }
END FUNCTION

```

Hàm 218: async updateTopic(id, data, userId)

Dòng nguồn: 245 | Đầu vào: id, data, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật chủ đề.

```

FUNCTION updateTopic(id, data, userId)
{ topicName, topicCode, description, topicCategoryId } = data
pool = GỌI BẮT ĐỒNG BỘ poolPromise
IF topicCategoryId THEN
    category = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT TopicCategoryId FROM
        TopicCategories WHERE TopicCategoryId = @TopicCategoryId AND IsActive =
        1"
    IF NOT category.recordset.length THEN
        THROW ERROR TẠO Error("Topic category not found")
    END IF
END IF
duplicate = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics]
IF duplicate.recordset.length THEN
    THROW ERROR TẠO Error("Topic name or code already exists")
END IF
result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Topics]
RETURN result.rowsAffected[0] > 0
END FUNCTION

```

Hàm 219: async deleteTopic(id, userId)

Dòng nguồn: 284 | Đầu vào: id, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa chủ đề.

```
FUNCTION deleteTopic(id, userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH DELETE trên Topics]
    RETURN result.rowsAffected[0] > 0
END FUNCTION
```

Hàm 220: async submitForReview(tableName, idColumn, id, userId)

Dòng nguồn: 296 | Đầu vào: tableName, idColumn, id, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để gửi for review.

```
FUNCTION submitForReview(tableName, idColumn, id, userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        req1 = TẠO sql.Request(transaction)
        upd = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE]
        IF upd.rowsAffected[0] = 0 THEN
            ROLLBACK TRANSACTION
            RETURN false
        END IF
        req2 = TẠO sql.Request(transaction)
        THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên ContentReviewLogs]
        COMMIT TRANSACTION
        RETURN true
    CATCH err
        ROLLBACK TRANSACTION
        THROW ERROR err
    END TRY
END FUNCTION
```

Hàm 221: async getMyWords(userId, filters = { })

Dòng nguồn: 333 | Đầu vào: userId, filters = { } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy my words.

```
FUNCTION getMyWords(userId, filters = { })
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    page = Math.max(1, CHUYỂN SANG SỐ NGUYÊN(filters.page) OR 1)
    pageSize = Math.min(100, Math.max(1, CHUYỂN SANG SỐ NGUYÊN(filters.pageSize) OR 50))
    offset = page - 1 * pageSize
    req = pool.request().input("UserID", sql.BigInt, userId)
    where = "w.CreatedByUserID = @UserID"
    IF filters.status THEN
        THỰC HIỆN req.input("Status", sql.NVarChar(20), filters.status)
        where = " AND w.ContentStatus = @Status"
    END IF
    search = String(filters.search OR "").trim()
    IF search THEN
        THỰC HIỆN req.input("Search", sql.NVarChar(250), `>${search}<`)
        where = " AND (w.Term LIKE @Search OR w.Meaning LIKE @Search OR w.Phonetic LIKE @Search)"
    END IF
    countResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words]
    total = countResult.recordset[0].total
    req2 = pool.request().input("UserID", sql.BigInt, userId).input("Offset", sql.Int, offset).input("PageSize", sql.Int, pageSize)
    where2 = "w.CreatedByUserID = @UserID"
    IF filters.status THEN
        THỰC HIỆN req2.input("Status", sql.NVarChar(20), filters.status)
```

```

        where2 = " AND w.ContentStatus = @Status"
    END IF
    IF search THEN
        THỰC HIỆN req2.input("Search", sql.NVarChar(250), `%${ search }%`)
        where2 = " AND (w.Term LIKE @Search OR w.Meaning LIKE @Search OR w.Phonetic
            LIKE @Search)"
    END IF
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên WordTopics,
        ExampleSentences, ContentReviewLogs, Words, Topics]
    RETURN { data: result.recordset.map(HÀM CALLBACK(word)), total: total, page:
page,
        pageSize: pageSize, totalPages: Math.ceil(total / pageSize) }
END FUNCTION

```

Hàm 222: async createWord(data, userId)

Dòng nguồn: 413 | Đầu vào: data, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới từ vựng.

```

FUNCTION createWord(data, userId)
    { term, meaning, phonetic, partOfSpeechId, topicIds, examples } = data
    normalizedTopicIds = [...new Set(Array.isArray(topicIds) ? topicIds: [])]
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        ownershipRequest = TẠO sql.Request(transaction)
        references = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT PartOfSpeechID FROM
            PartOfSpeeches WHERE PartOfSpeechID = @PartOfSpeechID"
        IF NOT references.recordset.length THEN
            THROW ERROR TẠO Error("Invalid part of speech")
        END IF
        duplicateRequest = TẠO sql.Request(transaction)
        duplicate = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT WordID FROM Words
            WHERE Term = @Term AND PartOfSpeechID = @PartOfSpeechID"
        IF duplicate.recordset.length THEN
            THROW ERROR TẠO Error("Word already exists for this part of speech")
        END IF
        FOR EACH topicId IN normalizedTopicIds
            topicRequest = TẠO sql.Request(transaction)
            topic = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
                Topics]
            IF NOT topic.recordset.length THEN
                THROW ERROR TẠO Error("Topic not found or unavailable")
            END IF
        END FOR
        req = TẠO sql.Request(transaction)
        wordResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Words]
        wordId = wordResult.recordset[0].id
        IF normalizedTopicIds.length THEN
            FOR EACH tid IN normalizedTopicIds
                r = TẠO sql.Request(transaction)
                THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên WordTopics]
            END FOR
        END IF
        validExamples = NẾU Array.isArray(examples) THÌ examples.filter(HÀM
            CALLBACK(e)) NGƯỢC LẠI []
        FOR EACH ex IN validExamples
            r = TẠO sql.Request(transaction)
            THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên ExampleSentences]
        END FOR
        COMMIT TRANSACTION
        RETURN { id: wordId, term: term }
    CATCH err
        ROLLBACK TRANSACTION
        THROW ERROR err
    END TRY
END FUNCTION

```

Hàm 223: async updateWord(id, data, userId)

Dòng nguồn: 489 | Đầu vào: id, data, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật từ vựng.

```

FUNCTION updateWord(id, data, userId)
  { term, meaning, phonetic, partOfSpeechId, topicIds = [], examples = [] } = data
  normalizedTopicIds = [...new Set(topicIds)]
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  transaction = TẠO sql.Transaction(pool)
  TRY
    BẮT ĐẦU TRANSACTION
    partOfSpeechRequest = TẠO sql.Request(transaction)
    partOfSpeech = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT PartOfSpeechID FROM
      PartOfSpeeches WHERE PartOfSpeechID = @PartOfSpeechID"
    IF NOT partOfSpeech.recordset.length THEN
      THROW ERROR TẠO Error("Invalid part of speech")
    END IF
    duplicateRequest = TẠO sql.Request(transaction)
    duplicate = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words]
    IF duplicate.recordset.length THEN
      THROW ERROR TẠO Error("Word already exists for this part of speech")
    END IF
    FOR EACH topicId IN normalizedTopicIds
      topicRequest = TẠO sql.Request(transaction)
      topic = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        Topics]
      IF NOT topic.recordset.length THEN
        THROW ERROR TẠO Error("Topic not found or unavailable")
      END IF
    END FOR
    request = TẠO sql.Request(transaction)
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Words]
    IF NOT result.rowsAffected[0] THEN
      ROLLBACK TRANSACTION
      RETURN false
    END IF
    clear = TẠO sql.Request(transaction)
    THỰC THI TRUY VẤN [CÂU LỆNH DELETE trên WordTopics, ExampleSentences]
    FOR EACH topicId IN normalizedTopicIds
      topicRequest = TẠO sql.Request(transaction)
      THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên WordTopics]
    END FOR
    FOR EACH example IN examples.filter(HÀM CALLBACK(item))
      exampleRequest = TẠO sql.Request(transaction)
      THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên ExampleSentences]
    END FOR
    COMMIT TRANSACTION
    RETURN true
  CATCH error
    ROLLBACK TRANSACTION
    THROW ERROR error
  END TRY
END FUNCTION

```

Hàm 224: async deleteWord(id, userId)

Dòng nguồn: 582 | Đầu vào: id, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa từ vựng.

```

FUNCTION deleteWord(id, userId)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  linked = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Questions,
Words]
  IF NOT linked.recordset.length THEN
    RETURN false
  END IF
  IF linked.recordset[0].QuestionCount > 0 THEN
    THROW ERROR TẠO Error("Word is linked to questions")
  END IF

```

```

result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH DELETE trên Words]
RETURN result.rowsAffected[0] > 0
END FUNCTION

```

Hàm 225: `async getMyQuestions(userId, filters = { })`

Dòng nguồn: 605 | Đầu vào: `userId, filters = { }` | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy my questions.

```

FUNCTION getMyQuestions(userId, filters = { })
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    page = Math.max(1, CHUYỂN SANG SỐ NGUYÊN(filters.page) OR 1)
    pageSize = Math.min(100, Math.max(1, CHUYỂN SANG SỐ NGUYÊN(filters.pageSize) OR 50))
    offset = page - 1 * pageSize
    req = pool.request().input("UserID", sql.BigInt, userId)
    where = "q.CreatedByUserID = @UserID"
    IF filters.status THEN
        THỰC HIỆN req.input("Status", sql.NVarChar(20), filters.status)
        where = " AND q.ContentStatus = @Status"
    END IF
    search = String(filters.search OR "").trim()
    IF search THEN
        THỰC HIỆN req.input("Search", sql.NVarChar(250), `>${search}`)
        where = " AND (q.QuestionText LIKE @Search OR q.CorrectAnswer LIKE @Search OR w.Term LIKE @Search)"
    END IF
    countResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Questions, Words]
    total = countResult.recordset[0].total
    req2 = pool.request().input("UserID", sql.BigInt, userId).input("Offset", sql.Int, offset).input("PageSize", sql.Int, pageSize)
    where2 = "q.CreatedByUserID = @UserID"
    IF filters.status THEN
        THỰC HIỆN req2.input("Status", sql.NVarChar(20), filters.status)
        where2 = " AND q.ContentStatus = @Status"
    END IF
    IF search THEN
        THỰC HIỆN req2.input("Search", sql.NVarChar(250), `>${search}`)
        where2 = " AND (q.QuestionText LIKE @Search OR q.CorrectAnswer LIKE @Search OR w.Term LIKE @Search)"
    END IF
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ContentReviewLogs, Questions, Words]
    RETURN { data: result.recordset, total: total, page: page, pageSize: pageSize, totalPages: Math.ceil(total / pageSize) }
END FUNCTION

```

Hàm 226: `async createQuestion(data, userId)`

Dòng nguồn: 663 | Đầu vào: `data, userId` | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới câu hỏi.

```

FUNCTION createQuestion(data, userId)
    { wordId, questionType, questionText, optionsJson, correctAnswer, explanation } = data
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    normalizedOptionsJson = JSON.stringify(this.parseQuestionOptions(questionType, optionsJson, correctAnswer))
    word = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words]
    IF NOT word.recordset.length THEN
        THROW ERROR TẠO Error("Word not found or unavailable")
    END IF
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Questions]
    RETURN result.recordset[0]
END FUNCTION

```

Hàm 227: async updateQuestion(id, data, userId)

Dòng nguồn: 697 | Đầu vào: id, data, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật câu hỏi.

```
FUNCTION updateQuestion(id, data, userId)
    { wordId, questionType, questionText, optionsJson, correctAnswer, explanation } =
        data
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    normalizedOptionsJson = JSON.stringify(this.parseQuestionOptions(questionType,
        optionsJson, correctAnswer))
    word = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words]
    IF NOT word.recordset.length THEN
        THROW ERROR TẠO Error("Word not found or unavailable")
    END IF
    result = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Questions]
    RETURN result.rowsAffected[0] > 0
END FUNCTION
```

Hàm 228: async deleteQuestion(id, userId)

Dòng nguồn: 732 | Đầu vào: id, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa câu hỏi.

```
FUNCTION deleteQuestion(id, userId)
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    linked = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTestItems,
        Questions]
    IF NOT linked.recordset.length THEN
        RETURN false
    END IF
    IF linked.recordset[0].MiniTestCount > 0 THEN
        THROW ERROR TẠO Error("Question is used in mini tests")
    END IF
    result = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH DELETE trên Questions]
    RETURN result.rowsAffected[0] > 0
END FUNCTION
```

Hàm 229: async getMyMiniTests(userId, filters = { })

Dòng nguồn: 757 | Đầu vào: userId, filters = { } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy my mini tests.

```
FUNCTION getMyMiniTests(userId, filters = { })
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    page = Math.max(1, CHUYỂN SANG SỐ NGUYÊN(filters.page) OR 1)
    pageSize = Math.min(100, Math.max(1, CHUYỂN SANG SỐ NGUYÊN(filters.pageSize) OR
50))
    offset = page - 1 * pageSize
    req = pool.request().input("UserID", sql.BigInt, userId)
    where = "mt.CreatedByUserID = @UserID"
    IF filters.status THEN
        THỰC HIỆN req.input("Status", sql.NVarChar(20), filters.status)
        where = " AND mt.ContentStatus = @Status"
    END IF
    search = String(filters.search OR "").trim()
    IF search THEN
        THỰC HIỆN req.input("Search", sql.NVarChar(250), `>${search}<`)
        where = " AND (mt.TestTitle LIKE @Search OR mt.Description LIKE @Search)"
    END IF
    countResult = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTests]
    total = countResult.recordset[0].total
    req2 = pool.request().input("UserID", sql.BigInt, userId).input("Offset",
sql.Int,
    offset).input("PageSize", sql.Int, pageSize)
    where2 = "mt.CreatedByUserID = @UserID"
    IF filters.status THEN
        THỰC HIỆN req2.input("Status", sql.NVarChar(20), filters.status)
```



```

        where2 = " AND mt.ContentStatus = @Status"
    END IF
    IF search THEN
        THỰC HIỆN req2.input("Search", sql.NVarChar(250), `%${ search }%`)
        where2 = " AND (mt.TestTitle LIKE @Search OR mt.Description LIKE @Search)"
    END IF
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTestItems,
        ContentReviewLogs, MiniTests, Topics]
    RETURN { data: result.recordset.map(HÀM CALLBACK(test)), total: total, page:
page,
        pageSize: pageSize, totalPages: Math.ceil(total / pageSize) }
END FUNCTION

```

Hàm 230: async createMiniTest(data, userId)

Dòng nguồn: 829 | Đầu vào: data, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới bài kiểm tra ngắn.

```

FUNCTION createMiniTest(data, userId)
{ title, description, topicId, questionIds } = data
pool = GỌI BẮT ĐỒNG BỘ poolPromise
transaction = TẠO sql.Transaction(pool)
TRY
    BẮT ĐẦU TRANSACTION
    normalizedQuestionIds = GỌI BẮT ĐỒNG BỘ
        this.validateMiniTestReferences(transaction, topicId, questionIds,
            userId)
    req = TẠO sql.Request(transaction)
    testResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên
        MiniTests]
    testId = testResult.recordset[0].id
    IF normalizedQuestionIds.length THEN
        FOR let i = 0; WHILE i < normalizedQuestionIds.length; i TĂNG 1
            r = TẠO sql.Request(transaction)
            THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên MiniTestItems]
        END FOR
    END IF
    COMMIT TRANSACTION
    RETURN { id: testId, title: title }
CATCH err
    ROLLBACK TRANSACTION
    THROW ERROR err
END TRY
END FUNCTION

```

Hàm 231: async updateMiniTest(id, data, userId)

Dòng nguồn: 874 | Đầu vào: id, data, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật bài kiểm tra ngắn.

```

FUNCTION updateMiniTest(id, data, userId)
{ title, description, topicId, questionIds = [] } = data
pool = GỌI BẮT ĐỒNG BỘ poolPromise
transaction = TẠO sql.Transaction(pool)
TRY
    BẮT ĐẦU TRANSACTION
    normalizedQuestionIds = GỌI BẮT ĐỒNG BỘ
        this.validateMiniTestReferences(transaction, topicId, questionIds,
            userId)
    request = TẠO sql.Request(transaction)
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên MiniTests]
    IF NOT result.rowsAffected[0] THEN
        ROLLBACK TRANSACTION
        RETURN false
    END IF
    clear = TẠO sql.Request(transaction)
    THỰC THI TRUY VẤN "DELETE FROM MiniTestItems WHERE MiniTestID = @MiniTestID"
    FOR let index = 0; WHILE index < normalizedQuestionIds.length; index TĂNG 1

```



```

        itemRequest = TẠO sql.Request(transaction)
        THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên MiniTestItems]
    END FOR
    COMMIT TRANSACTION
    RETURN true
CATCH error
    ROLLBACK TRANSACTION
    THROW ERROR error
END TRY
END FUNCTION

```

Hàm 232: async deleteMiniTest(id, userId)

Dòng nguồn: 927 | Đầu vào: id, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa bài kiểm tra ngắn.

```

FUNCTION deleteMiniTest(id, userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        r1 = TẠO sql.Request(transaction)
        THỰC THI TRUY VẤN [CÂU LỆNH DELETE trên MiniTestItems, MiniTests]
        r2 = TẠO sql.Request(transaction)
        del = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH DELETE trên MiniTests]
        COMMIT TRANSACTION
        RETURN del.rowsAffected[0] > 0
    CATCH err
        ROLLBACK TRANSACTION
        THROW ERROR err
    END TRY
END FUNCTION

```

Hàm 233: async addMiniTestItem(miniTestId, questionId, userId)

Dòng nguồn: 947 | Đầu vào: miniTestId, questionId, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để thêm một phần tử câu hỏi của bài kiểm tra.

```

FUNCTION addMiniTestItem(miniTestId, questionId, userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        checkRequest = TẠO sql.Request(transaction)
        check = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTests,
            Questions, MiniTestItems]
        IF NOT check.recordset.length THEN
            ROLLBACK TRANSACTION
            RETURN false
        END IF
        IF NOT check.recordset[0].QuestionAvailable THEN
            THROW ERROR TẠO Error("Mini test questions must be owned and
                published")
        END IF
        IF check.recordset[0].AlreadyAdded THEN
            THROW ERROR TẠO Error("Question is already in this mini test")
        END IF
        insertRequest = TẠO sql.Request(transaction)
        THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên MiniTestItems]
        updateRequest = TẠO sql.Request(transaction)
        THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên MiniTestItems, MiniTests]
        COMMIT TRANSACTION
        RETURN true
    CATCH error
        ROLLBACK TRANSACTION
        THROW ERROR error
    END TRY
END FUNCTION

```

Hàm 234: async removeMiniTestItem(miniTestId, questionId, userId)

Dòng nguồn: 1009 | Đầu vào: miniTestId, questionId, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để gỡ bỏ một phần tử câu hỏi của bài kiểm tra.

```
FUNCTION removeMiniTestItem(miniTestId, questionId, userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        checkRequest = TẠO sql.Request(transaction)
        check = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTests]
        IF NOT check.recordset.length THEN
            ROLLBACK TRANSACTION
            RETURN false
        END IF
        deleteRequest = TẠO sql.Request(transaction)
        THỰC THI TRUY VẤN "DELETE FROM MiniTestItems WHERE MiniTestID = @MiniTestID
            AND QuestionID = @QuestionID"
        updateRequest = TẠO sql.Request(transaction)
        THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên MiniTestItems, MiniTests]
        COMMIT TRANSACTION
        RETURN true
    CATCH error
        ROLLBACK TRANSACTION
        THROW ERROR error
    END TRY
END FUNCTION
```

Hàm 235: async getMedia(userId)

Dòng nguồn: 1050 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách media metadata.

```
FUNCTION getMedia(userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MediaAssets]
    RETURN result.recordset
END FUNCTION
```

Hàm 236: async createMedia(data, userId)

Dòng nguồn: 1071 | Đầu vào: data, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới danh sách media metadata.

```
FUNCTION createMedia(data, userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên MediaAssets]
    RETURN result.recordset[0]
END FUNCTION
```

Hàm 237: async deleteMedia(id, userId)

Dòng nguồn: 1099 | Đầu vào: id, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa danh sách media metadata.

```
FUNCTION deleteMedia(id, userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
ContentMediaLinks,
MediaAssets]
    RETURN result.rowsAffected[0] > 0
END FUNCTION
```

Hàm 238: async submitQuestionForReview(id, userId)

Dòng nguồn: 1114 | Đầu vào: id, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để gửi question for review.

```
FUNCTION submitQuestionForReview(id, userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    dependency = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Questions, Words]
    IF NOT dependency.recordset.length THEN
        RETURN false
    END IF
    IF dependency.recordset[0].WordStatus ≠ "Published" THEN
        THROW ERROR TẠO Error("Question word must be published before review")
    END IF
    RETURN this.submitForReview("Questions", "QuestionID", id, userId)
END FUNCTION
```

Hàm 239: async submitWordForReview(id, userId)

Dòng nguồn: 1133 | Đầu vào: id, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để gửi word for review.

```
FUNCTION submitWordForReview(id, userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    dependency = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words, WordTopics, Topics]
    IF NOT dependency.recordset.length THEN
        RETURN false
    END IF
    IF dependency.recordset[0].UnpublishedTopics OR 0 > 0 THEN
        THROW ERROR TẠO Error("Word topics must be published before review")
    END IF
    RETURN this.submitForReview("Words", "WordID", id, userId)
END FUNCTION
```

Hàm 240: async submitMiniTestForReview(id, userId)

Dòng nguồn: 1155 | Đầu vào: id, userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để gửi mini test for review.

```
FUNCTION submitMiniTestForReview(id, userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    check = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTests, MiniTestItems, Questions, Topics]
    IF NOT check.recordset.length THEN
        RETURN false
    END IF
    IF NOT check.recordset[0].total THEN
        THROW ERROR TẠO Error("Bài test phải có ít nhất một câu hỏi trước khi gửi duyệt")
    END IF
    IF check.recordset[0].unpublished > 0 THEN
        THROW ERROR TẠO Error("Tất cả câu hỏi trong bài test phải được Published trước khi gửi duyệt")
    END IF
    IF check.recordset[0].topicUnpublished > 0 THEN
        THROW ERROR TẠO Error("Chủ đề của bài test phải được Published trước khi gửi duyệt")
    END IF
    RETURN this.submitForReview("MiniTests", "MiniTestID", id, userId)
END FUNCTION
```

services/gamification.service.js

File này có 12 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 241: getDateKey(date = new Date())

Dòng nguồn: 22 | Đầu vào: `date = new Date()` | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy date key.

```
FUNCTION getDateKey(date = new Date())
    RETURN date.toISOString().slice(0, 10)
END FUNCTION
```

Hàm 242: getLevelState(totalXP = 0)

Dòng nguồn: 26 | Đầu vào: `totalXP = 0` | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy level state.

```
FUNCTION getLevelState(totalXP = 0)
    safeXP = Math.max(0, Number(totalXP) OR 0)
    currentLevel = 1
    levelStartXP = 0
    xpForNextLevel = 100
    WHILE safeXP >= levelStartXP + xpForNextLevel
        levelStartXP = xpForNextLevel
        currentLevel = 1
        xpForNextLevel = currentLevel * 100
    END WHILE
    currentLevelXP = safeXP - levelStartXP
    RETURN { totalXP: safeXP, currentLevel: currentLevel, currentLevelXP:
        currentLevelXP, xpForNextLevel: xpForNextLevel, xpToNextLevel: xpForNextLevel
    -
        currentLevelXP, nextLevelTotalXP: levelStartXP + xpForNextLevel,
    levelProgress:
        Math.round(currentLevelXP / xpForNextLevel * 100) }
END FUNCTION
```

Hàm 243: async ensureSchema()

Dòng nguồn: 50 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để bảo đảm sẵn sàng schema.

```
FUNCTION ensureSchema()
    IF NOT this.schemaReady THEN
        this.schemaReady = this.createSchema().catch(HÀM_CALLBACK(error))
    END IF
    RETURN this.schemaReady
END FUNCTION
```

Hàm 244: async createSchema()

Dòng nguồn: 60 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới schema.

```
FUNCTION createSchema()
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    THỰC THI TRUY VẤN [CÂU LỆNH ALTER trên sys]
    FOR EACH achievement IN ACHIEVEMENT_SEED
        [code, name, description, icon, criteriaType, criteriaValue, displayOrder] =
            achievement
        THỰC THI TRUY VẤN [CÂU LỆNH MERGE trên SET, Achievements]
    END FOR
END FUNCTION
```

Hàm 245: async awardXP(userId, { eventType, sourceKey = null, metadata = null } = { })

Dòng nguồn: 169 | Đầu vào: userId, { eventType, sourceKey = null, metadata = null } = { } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cộng thưởng cho xp.

```
FUNCTION awardXP(userId, { eventType, sourceKey = null, metadata = null } = { })
  THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema()
  amount = XP_REWARDS[eventType]
  IF NOT amount THEN
    THROW ERROR TẠO Error(`Unsupported XP event type: ${ eventType }`)
  END IF
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  transaction = TẠO sql.Transaction(pool)
  BẮT ĐẦU TRANSACTION
  TRY
    request = TẠO sql.Request(transaction)
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
      UserXPEvents, Users]
    row = result.recordset[0] OR { }
    level = this.getLevelState(row.totalXP)
    THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Users]
    COMMIT TRANSACTION
    unlockedAchievements = NẾU row.xpGained > 0 THÌ GỌI BẮT ĐỒNG BỘ
      this.checkAchievements(userId) NGƯỢC LẠI []
    RETURN { xpEventId: Number(row.xpEventId OR 0), xpGained:
      Number(row.xpGained OR 0), eventType: eventType, awarded:
      Number(row.xpGained OR 0) > 0, ...level, unlockedAchievements:
      unlockedAchievements }
  CATCH error
    ROLLBACK TRANSACTION
    THROW ERROR error
  END TRY
END FUNCTION
```

Hàm 246: async awardDailyLogin(userId)

Dòng nguồn: 234 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cộng thưởng cho daily login.

```
FUNCTION awardDailyLogin(userId)
  RETURN this.awardXP(userId, { eventType: "DailyLogin", sourceKey: `daily-login:
  ${
    this.getDateKey() }` })
END FUNCTION
```

Hàm 247: async checkAchievements(userId)

Dòng nguồn: 241 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để kiểm tra achievements.

```
FUNCTION checkAchievements(userId)
  THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema()
  profile = GỌI BẮT ĐỒNG BỘ this.getMetrics(userId)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên Achievements,
    UserAchievements]
  unlockedIds = result.recordset.map(HÀM CALLBACK(row))
  IF unlockedIds.length = 0 THEN
    RETURN []
  END IF
  allAchievements = GỌI BẮT ĐỒNG BỘ this.getAchievements(userId, profile)
  RETURN allAchievements.filter(HÀM CALLBACK(achievement))
END FUNCTION
```

Hàm 248: async getMetrics(userId)*Dòng nguồn: 276 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.***Mục đích:** Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy metrics.

```

FUNCTION getMetrics(userId)
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema()
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên UserXPEvents,
        ExerciseAttempts, DailyActivity, Users, UserWordProgress]
    row = result.recordset[0] OR { }
    RETURN { wordsLearned: Number(row.wordsLearned OR 0), bestTestScore:
        Number(row.bestTestScore OR 0), streak: Number(row.streak OR 0), todayXP:
        Number(row.todayXP OR 0), ...this.getLevelState(row.totalXP) }
END FUNCTION

```

Hàm 249: async getAchievements(userId, metrics = null)*Dòng nguồn: 324 | Đầu vào: userId, metrics = null | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.***Mục đích:** Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy achievements.

```

FUNCTION getAchievements(userId, metrics = null)
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema()
    profile = metrics OR GỌI BẮT ĐỒNG BỘ this.getMetrics(userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Achievements,
        UserAchievements]
    RETURN result.recordset.map(HÀM CALLBACK(achievement))
END FUNCTION

```

Hàm 250: getAchievementProgress(criteriaType, metrics)*Dòng nguồn: 360 | Đầu vào: criteriaType, metrics | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.***Mục đích:** Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy achievement progress.

```

FUNCTION getAchievementProgress(criteriaType, metrics)
    IF criteriaType = "WORDS_LEARNED" THEN
        RETURN metrics.wordsLearned
    END IF
    IF criteriaType = "STREAK_DAYS" THEN
        RETURN metrics.streak
    END IF
    IF criteriaType = "TEST_SCORE" THEN
        RETURN metrics.bestTestScore
    END IF
    IF criteriaType = "LEVEL" THEN
        RETURN metrics.currentLevel
    END IF
    RETURN 0
END FUNCTION

```

Hàm 251: async getProfile(userId)*Dòng nguồn: 368 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.***Mục đích:** Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy hồ sơ người dùng.

```

FUNCTION getProfile(userId)
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema()
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.checkAchievements(userId)
    metrics = GỌI BẮT ĐỒNG BỘ this.getMetrics(userId)
    achievements = GỌI BẮT ĐỒNG BỘ this.getAchievements(userId, metrics)
    RETURN { ...metrics, achievements: achievements, unseenAchievements:
        achievements.filter(HÀM CALLBACK(achievement)) }
END FUNCTION

```

Hàm 252: async markAchievementsSeen(userId, achievementIds = [])

Dòng nguồn: 380 | Đầu vào: *userId*, *achievementIds* = [] | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để đánh dấu trạng thái đã xem huy hiệu.

```
FUNCTION markAchievementsSeen(userId, achievementIds = [])
  THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema()
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  request = pool.request().input("UserID", sql.BigInt, userId)
  IF NOT Array.isArray(achievementIds) OR achievementIds.length = 0 THEN
    THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên UserAchievements]
    RETURN không có giá trị
  END IF
  safeIds = achievementIds.map(Number).filter(Number.isInteger)
  IF safeIds.length = 0 THEN
    RETURN không có giá trị
  END IF
  THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên UserAchievements]
END FUNCTION
```

services/learning-path.service.js

File này có 5 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 253: async ensureSchema()

Dòng nguồn: 14 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để bảo đảm sẵn sàng schema.

```
FUNCTION ensureSchema()
  IF NOT this.schemaReady THEN
    this.schemaReady = this.createSchema().catch(HÀM CALLBACK(error))
  END IF
  RETURN this.schemaReady
END FUNCTION
```

Hàm 254: async createSchema()

Dòng nguồn: 24 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới schema.

```
FUNCTION createSchema()
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  THỰC THI TRUY VẤN [CÂU LỆNH CREATE trên sys]
  FOR EACH level IN LEVEL_SEED
    [code, name, targetScore, description, displayOrder, accentKey] = level
    THỰC THI TRUY VẤN [CÂU LỆNH MERGE trên SET, LearningPathLevels]
  END FOR
END FUNCTION
```

Hàm 255: async syncPublishedTopics()

Dòng nguồn: 95 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để đồng bộ published topics.

```
FUNCTION syncPublishedTopics()
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, RankedTopics, LearningPathTopics,
    TopicsToMap, LearningPathLevels]
END FUNCTION
```


Hàm 256: async getRoadmap(userId)

Dòng nguồn: 120 | Đầu vào: *userId* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy lộ trình học tập.

```
FUNCTION getRoadmap(userId)
  THỰC HIỆN GỌI BẮT ĐỒNG BỘ Promise.all([this.ensureSchema(),
    GamificationService.ensureSchema()])
  THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.syncPublishedTopics()
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    LearningPathLevels,
    LearningPathTopics, WordTopics, UserXPEvents, MiniTests]
  RETURN this.buildRoadmap(result.recordsets[0], result.recordsets[1])
END FUNCTION
```

Hàm 257: buildRoadmap(levelRows, topicRows)

Dòng nguồn: 189 | Đầu vào: *levelRows*, *topicRows* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xây dựng lộ trình học tập.

```
FUNCTION buildRoadmap(levelRows, topicRows)
  allTopics = []
  levels = levelRows.map(HÀM CALLBACK(level))
  completedTopics = allTopics.filter(HÀM CALLBACK(topic)).length
  currentTopicIndex = allTopics.findIndex(HÀM CALLBACK(topic))
  currentTopic = NẾU currentTopicIndex >= 0 THÌ allTopics[currentTopicIndex] NGƯỢC
  LẠI
    allTopics.at(-1) OR NULL
  nextTopic = NẾU currentTopicIndex >= 0 THÌ allTopics[currentTopicIndex + 1] OR
  NULL
    NGƯỢC LẠI NULL
  currentActivity = currentTopic.activities.find(HÀM CALLBACK(activity)) OR NULL
  RETURN { completionPercentage: NẾU allTopics.length > 0 THÌ
    Math.round(allTopics.reduce(HÀM CALLBACK(sum, topic), 0) / allTopics.length)
    NGƯỢC LẠI 0, completedTopics: completedTopics, totalTopics: allTopics.length,
    currentPosition: NẾU currentTopic THÌ { levelTitle: levels.find(HÀM
    CALLBACK(level)).title, topicId: currentTopic.topicId, topicTitle:
    currentTopic.title, topicStatus: currentTopic.status, activityTitle:
    currentActivity.title OR "Roadmap completed", activityRoute:
    currentActivity.route OR "/user/courses", completionPercentage:
    currentTopic.completionPercentage } NGƯỢC LẠI NULL, currentLesson: NẾU
    currentTopic THÌ { topicId: currentTopic.topicId, title: currentTopic.title,
    route: `/user/learn/${ currentTopic.topicId }`, status: currentTopic.status,
    completionPercentage: currentTopic.completionPercentage } NGƯỢC LẠI NULL,
    nextLesson: NẾU nextTopic THÌ { topicId: nextTopic.topicId, title:
    nextTopic.title, route: `/user/learn/${ nextTopic.topicId }`, status:
    nextTopic.status, completionPercentage: nextTopic.completionPercentage }
  NGƯỢC
    LẠI NULL, levels: levels }
END FUNCTION
```

services/progress.service.js

File này có 2 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 258: async getProgress(userId)

Dòng nguồn: 5 | Đầu vào: *userId* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy progress.

```
FUNCTION getProgress(userId)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  totalResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    UserWordProgress]
  statsResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    ExerciseAttempts]
  weakResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    ExerciseAttempts, Words]
```



```

    streakResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        ExerciseAttempts, DailyActivity, RankedDates]
    totalLearned = totalResult.recordset[0].totalLearned OR 0
    correct = statsResult.recordset[0].correct OR 0
    wrong = statsResult.recordset[0].wrong OR 0
    streak = streakResult.recordset[0].streak OR 0
    accuracy = NẾU correct + wrong > 0 THÌ Math.round(correct / correct + wrong *
100)
    NGƯỢC LẠI 0
    RETURN { totalLearned: totalLearned, accuracy: accuracy, streak: streak, correct:
        correct, wrong: wrong, weakWords: weakResult.recordset.map(HÀM CALLBACK(row))
    }
END FUNCTION

```

Hàm 259: async getStats(userId)

Dòng nguồn: 83 | Đầu vào: *userId* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy thống kê.

```

FUNCTION getStats(userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    masteryResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        UserWordProgress]
    weeklyResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        ExerciseAttempts]
    RETURN { masteryLevels: masteryResult.recordset, weeklyActivity:
        weeklyResult.recordset }
END FUNCTION

```

services/report.service.js

File này có 8 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 260: normalizePagination(page = 1, limit = 20, maxLimit = 100)

Dòng nguồn: 9 | Đầu vào: *page = 1, limit = 20, maxLimit = 100* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để chuẩn hóa pagination.

```

FUNCTION normalizePagination(page = 1, limit = 20, maxLimit = 100)
    normalizedPage = Math.max(CHUYỂN SANG SỐ NGUYÊN(page, 10) OR 1, 1)
    normalizedLimit = Math.min(Math.max(CHUYỂN SANG SỐ NGUYÊN(limit, 10) OR 20, 1),
        maxLimit)
    RETURN { page: normalizedPage, limit: normalizedLimit, offset: normalizedPage - 1
*
        normalizedLimit }
END FUNCTION

```

Hàm 261: paginate(items, total, page, limit)

Dòng nguồn: 20 | Đầu vào: *items, total, page, limit* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý paginate trong module backend.

```

FUNCTION paginate(items, total, page, limit)
    normalizedTotal = Number(total) OR 0
    RETURN { items: items, pagination: { page: page, limit: limit, total:
        normalizedTotal, totalPages: Math.max(1, Math.ceil(normalizedTotal / limit))
    } }
END FUNCTION

```

Hàm 262: async ensureSchema()

Dòng nguồn: 33 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để bảo đảm sẵn sàng schema.

```
FUNCTION ensureSchema()
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    THỰC THI TRUY VẤN [CÂU LỆNH CREATE trên sys]
END FUNCTION
```

Hàm 263: assertOneOf(value, allowed, fieldName)

Dòng nguồn: 77 | Đầu vào: value, allowed, fieldName | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý assert one of trong module backend.

```
FUNCTION assertOneOf(value, allowed, fieldName)
    IF NOT allowed.includes(value) THEN
        THROW ERROR TẠO Error(`Invalid ${ fieldName }`)
    END IF
END FUNCTION
```

Hàm 264: inferEntityType({ entityType, questionId, wordId, reportType })

Dòng nguồn: 83 | Đầu vào: { entityType, questionId, wordId, reportType } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để suy luận entity type.

```
FUNCTION inferEntityType({ entityType, questionId, wordId, reportType })
    IF entityType THEN
        RETURN entityType
    END IF
    IF reportType = "AudioIssue" THEN
        RETURN "Audio"
    END IF
    IF questionId THEN
        RETURN "Question"
    END IF
    IF wordId THEN
        RETURN "Word"
    END IF
    RETURN "General"
END FUNCTION
```

Hàm 265: async createReport(userId, reportData)

Dòng nguồn: 91 | Đầu vào: userId, reportData | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để tạo mới báo lỗi/nội dung.

```
FUNCTION createReport(userId, reportData)
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema()
    reportType = String(reportData.reportType OR "").trim()
    entityType = this.inferEntityType({ entityType: String(reportData.entityType OR "").trim(), questionId: reportData.questionId, wordId: reportData.wordId, reportType: reportType })
    wordId = Number(reportData.wordId) OR NULL
    questionId = Number(reportData.questionId) OR NULL
    title = String(reportData.title OR reportType).trim().slice(0, 200)
    description = String(reportData.description OR "").trim().slice(0, 2000)
    THỰC HIỆN this.assertOneOf(reportType, REPORT_TYPES, "report type")
    THỰC HIỆN this.assertOneOf(entityType, ENTITY_TYPES, "entity type")
    IF NOT description OR description.length < 5 THEN
        THROW ERROR TẠO Error("Report description is too short")
    END IF
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên ContentReports]
```

```
RETURN result.recordset[0]
END FUNCTION
```

Hàm 266: async getReports(page = 1, limit = 20, filters = { })

Dòng nguồn: 131 | Đầu vào: page = 1, limit = 20, filters = { } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách báo lỗi/nội dung.

```
FUNCTION getReports(page = 1, limit = 20, filters = { })
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema()
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    paging = this.normalizePagination(page, limit, 100)
    search = String(filters.search NẾU RỖNG THÌ "").trim()
    status = String(filters.status NẾU RỖNG THÌ "").trim()
    reportType = String(filters.reportType NẾU RỖNG THÌ "").trim()
    entityType = String(filters.entityType NẾU RỖNG THÌ "").trim()
    priority = String(filters.priority NẾU RỖNG THÌ "").trim()
    conditions = []
    request = pool.request().input("Offset", sql.Int, paging.offset).input("Limit",
        sql.Int, paging.limit)
    IF search THEN
        THỰC HIỆN request.input("Search", sql.NVarChar(250), `%${ search }%`)
        THỰC HIỆN conditions.push(`(cr.Title LIKE @Search OR cr.Description LIKE
            @Search OR cr.AdminResponse LIKE @Search OR reporter.FullName LIKE
            @Se...)`)
    END IF
    IF status THEN
        THỰC HIỆN this.assertOneOf(status, REPORT_STATUSES, "report status")
        THỰC HIỆN request.input("Status", sql.NVarChar(30), status)
        THỰC HIỆN conditions.push("cr.Status = @Status")
    END IF
    IF reportType THEN
        THỰC HIỆN this.assertOneOf(reportType, REPORT_TYPES, "report type")
        THỰC HIỆN request.input("ReportType", sql.NVarChar(50), reportType)
        THỰC HIỆN conditions.push("cr.ReportType = @ReportType")
    END IF
    IF entityType THEN
        THỰC HIỆN this.assertOneOf(entityType, ENTITY_TYPES, "entity type")
        THỰC HIỆN request.input("EntityType", sql.NVarChar(30), entityType)
        THỰC HIỆN conditions.push("cr.EntityType = @EntityType")
    END IF
    IF priority THEN
        THỰC HIỆN this.assertOneOf(priority, PRIORITIES, "priority")
        THỰC HIỆN request.input("Priority", sql.NVarChar(20), priority)
        THỰC HIỆN conditions.push("cr.Priority = @Priority")
    END IF
    whereClause = NẾU conditions.length > 0 THÌ `WHERE ${ conditions.join(' AND ' ) }`
    NGƯỢC LẠI ""
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên ContentReports,
        Users, Words, Questions]
    RETURN this.paginate(result.recordsets[1] OR [], result.recordsets[0][0].total OR
0,
        paging.page, paging.limit)
END FUNCTION
```

Hàm 267: async updateReport(reportId, data, adminId)

Dòng nguồn: 226 | Đầu vào: reportId, data, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật báo lỗi/nội dung.

```
FUNCTION updateReport(reportId, data, adminId)
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ this.ensureSchema()
    status = String(data.status OR "").trim()
    priority = String(data.priority OR "").trim()
    adminResponse = NẾU data.adminResponse = undefined THÌ undefined NGƯỢC LẠI
        String(data.adminResponse OR "").trim().slice(0, 2000)
    IF status THEN
        THỰC HIỆN this.assertOneOf(status, REPORT_STATUSES, "report status")
    
```

```

END IF
IF priority THEN
  THỰC HIỆN this.assertOneOf(priority, PRIORITIES, "priority")
END IF
pool = GỢI BẮT ĐỒNG BỘ poolPromise
oldResult = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT Status, Priority FROM
  ContentReports WHERE ContentReportID = @ContentReportID"
IF oldResult.recordset.length = 0 THEN
  RETURN false
END IF
result = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên ContentReports]
IF result.rowsAffected[0] > 0 THEN
  AdminService = require("./admin.service")
  THỰC HIỆN GỢI BẮT ĐỒNG BỘ AdminService.logAdminAction(adminId,
    "UPDATE_CONTENT_REPORT", "ContentReport", reportId, { old:
      oldResult.recordset[0], status: status OR undefined, priority: priority
      OR undefined, hasResponse: adminResponse ≠ undefined })
END IF
RETURN result.rowsAffected[0] > 0
END FUNCTION

```

services/review.service.js

File này có 6 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 268: `async getPendingContent()`

Dòng nguồn: 7 | Đầu vào: Không có tham số | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách nội dung chờ duyệt.

```

FUNCTION getPendingContent()
  pool = GỢI BẮT ĐỒNG BỘ poolPromise
  result = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Topics, Words,
    Questions, MiniTests, Users]
  RETURN result.recordset
END FUNCTION

```

Hàm 269: `_resolveEntity(entityType)`

Dòng nguồn: 35 | Đầu vào: `entityType` | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm nội bộ dùng để hỗ trợ lớp/module trong việc resolve entity.

```

FUNCTION _resolveEntity(entityType)
  input = String(entityType OR "").toLowerCase()
  map = { topic: { type: "Topic", table: "Topics", idCol: "TopicID",
    publishedAtCol:
      "PublishedAt", reviewedByCol: "ReviewedByUserID", reviewedAtCol: "ReviewedAt"
  },
    word: { type: "Word", table: "Words", idCol: "WordID", publishedAtCol:
      "PublishedAt", reviewedByCol: "ReviewedByUserID", reviewedAtCol: "ReviewedAt"
  },
    question: { type: "Question", table: "Questions", idCol: "QuestionID",
      publishedAtCol: "PublishedAt", reviewedByCol: "ReviewedByUserID",
      reviewedAtCol:
        "ReviewedAt" }, minitest: { type: "MiniTest", table: "MiniTests", idCol:
        "MiniTestID", publishedAtCol: "PublishedAt", reviewedByCol:
        "ReviewedByUserID",
        reviewedAtCol: "ReviewedAt" } }
  entity = map[input]
  IF NOT entity THEN
    THROW ERROR TẠO Error("EntityType không hợp lệ")
  END IF
  RETURN entity
END FUNCTION

```

Hàm 270: async approve(entityType, entityId, adminId)

Dòng nguồn: 51 | Đầu vào: entityType, entityId, adminId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Phê duyệt nội dung chờ duyệt, cập nhật trạng thái Published, ghi mốc reviewed/published và lưu review log.

```

FUNCTION approve(entityType, entityId, adminId)
    e = this._resolveEntity(entityType)
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        extraSet = ""
        IF e.publishedAtCol THEN
            extraSet = `, ${ e.publishedAtCol } = SYSDATETIMEOFFSET() `
        END IF
        IF e.reviewedByCol THEN
            extraSet = `, ${ e.reviewedByCol } = @AdminID `
        END IF
        IF e.reviewedAtCol THEN
            extraSet = `, ${ e.reviewedAtCol } = SYSDATETIMEOFFSET() `
        END IF
        req1 = TẠO sql.Request(transaction)
        upd = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE]
        IF upd.rowsAffected[0] = 0 THEN
            ROLLBACK TRANSACTION
            RETURN false
        END IF
        IF e.type = "MiniTest" THEN
            req1b = TẠO sql.Request(transaction)
            THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên MiniTests]
        END IF
        req2 = TẠO sql.Request(transaction)
        THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên ContentReviewLogs]
        COMMIT TRANSACTION
        RETURN true
    CATCH err
        ROLLBACK TRANSACTION
        THROW ERROR err
    END TRY
END FUNCTION

```

Hàm 271: async reject(entityType, entityId, adminId, reason)

Dòng nguồn: 106 | Đầu vào: entityType, entityId, adminId, reason | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Từ chối nội dung PendingReview, lưu lý do từ chối và ghi nhận lịch sử duyệt.

```

FUNCTION reject(entityType, entityId, adminId, reason)
    e = this._resolveEntity(entityType)
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        extraSet = ""
        IF e.reviewedByCol THEN
            extraSet = `, ${ e.reviewedByCol } = @AdminID `
        END IF
        IF e.reviewedAtCol THEN
            extraSet = `, ${ e.reviewedAtCol } = SYSDATETIMEOFFSET() `
        END IF
        req1 = TẠO sql.Request(transaction)
        upd = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE]
        IF upd.rowsAffected[0] = 0 THEN
            ROLLBACK TRANSACTION
            RETURN false
        END IF
        req2 = TẠO sql.Request(transaction)
        THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên ContentReviewLogs]
    CATCH err
        ROLLBACK TRANSACTION
        THROW ERROR err
    END TRY
END FUNCTION

```

```

        COMMIT TRANSACTION
        RETURN true
    CATCH err
        ROLLBACK TRANSACTION
        THROW ERROR err
    END TRY
END FUNCTION

```

Hàm 272: `async archive(entityType, entityId, adminId)`

Dòng nguồn: 154 | Đầu vào: `entityType`, `entityId`, `adminId` | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Chuyển nội dung sang Archived và ghi nhận thao tác lưu trữ vào bảng log duyệt.

```

FUNCTION archive(entityType, entityId, adminId)
    e = this._resolveEntity(entityType)
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    TRY
        BẮT ĐẦU TRANSACTION
        req1 = TẠO sql.Request(transaction)
        upd = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE]
        IF upd.rowsAffected[0] = 0 THEN
            ROLLBACK TRANSACTION
            RETURN false
        END IF
        req2 = TẠO sql.Request(transaction)
        THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên ContentReviewLogs]
        COMMIT TRANSACTION
        RETURN true
    CATCH err
        ROLLBACK TRANSACTION
        THROW ERROR err
    END TRY
END FUNCTION

```

Hàm 273: `async getReviewLogs(entityType, entityId)`

Dòng nguồn: 196 | Đầu vào: `entityType`, `entityId` | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy nhật ký duyệt nội dung.

```

FUNCTION getReviewLogs(entityType, entityId)
    e = this._resolveEntity(entityType)
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    result = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    ContentReviewLogs,
    Users]
    RETURN result.recordset
END FUNCTION

```

services/user.service.js

File này có 31 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 274: `async getFlashcards(userId)`

Dòng nguồn: 5 | Đầu vào: `userId` | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy bộ flashcard.

```

FUNCTION getFlashcards(userId)
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    result = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Questions,
    Words,
    UserWordProgress]
    RETURN result.recordset
END FUNCTION

```

Hàm 275: async getDueFlashcards(userId, { topicId = null, mode = null } = { })

Dòng nguồn: 21 | Đầu vào: *userId*, { *topicId* = null, *mode* = null } = { } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Lấy danh sách flashcard đến hạn ôn theo SRS, có thể lọc theo chủ đề hoặc chế độ new/learned.

```
FUNCTION getDueFlashcards(userId, { topicId = null, mode = null } = { })
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Users,
        UserTopicEnrollments, Words, Questions, ExampleSentences]
    RETURN result.recordset
END FUNCTION
```

Hàm 276: async getTopicWords(userId, topicId)

Dòng nguồn: 120 | Đầu vào: *userId*, *topicId* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách từ vựng của chủ đề.

```
FUNCTION getTopicWords(userId, topicId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên WordTopics,
        ExampleSentences, Words, PartOfSpeeches, UserWordProgress]
    RETURN result.recordset
END FUNCTION
```

Hàm 277: async submitAnswer({ userId, questionId, wordId, submittedAnswer, isCorrect, reviewRating, activityType })

Dòng nguồn: 158 | Đầu vào: { *userId*, *questionId*, *wordId*, *submittedAnswer*, *isCorrect*, *reviewRating*, *activityType* } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Ghi nhận câu trả lời, gọi stored procedure lưu attempt, cập nhật lịch SRS theo *reviewRating* và cộng XP nếu cần.

```
FUNCTION submitAnswer({ userId, questionId, wordId, submittedAnswer, isCorrect,
    reviewRating, activityType })
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI STORED PROCEDURE "usp_SubmitQuestionAttempt"
    canonicalWordId = Number(result.recordset[0].WordID OR 0)
    IF NOT canonicalWordId THEN
        THROW ERROR TẠO Error("Question does not resolve to a vocabulary word")
    END IF
    reviewFeedback = { }
    IF ["Again", "Hard", "Good", "Easy"].includes(reviewRating) THEN
        feedbackResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên
            UserWordProgress]
        reviewFeedback = feedbackResult.recordset[0] OR { }
    END IF
    gamification = NẾU activityType = "LearnWord" THÌ GỌI BẮT ĐỒNG BỘ
        GamificationService.awardXP(userId, { eventType: "LearnWord", sourceKey:
            `learn-word: ${ canonicalWordId } : ${ GamificationService.getDateKey() }`,
            metadata: { wordId: canonicalWordId, reviewRating: reviewRating OR NULL } })
    NGƯỢC LẠI NULL
    RETURN { ...result.recordset[0] OR { }, ...reviewFeedback, xpGained:
        gamification.xpGained OR 0, reviewRating: reviewRating OR NULL, gamification:
        gamification }
END FUNCTION
```


Hàm 278: async submitWordReview({ userId, wordId, isCorrect, reviewRating, activityType })

Dòng nguồn: 238 | Đầu vào: { userId, wordId, isCorrect, reviewRating, activityType } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Cập nhật trực tiếp UserWordProgress theo kết quả ôn từ, tính mastery/ease factor/next review date và gamification.

```
FUNCTION submitWordReview({ userId, wordId, isCorrect, reviewRating, activityType })
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH MERGE trên SET,
    UserWordProgress]
  gamification = NẾU activityType = "LearnWord" THÌ GỌI BẮT ĐỒNG BỘ
    GamificationService.awardXP(userId, { eventType: "LearnWord", sourceKey:
      `learn-word: ${ wordId } : ${ GamificationService.getDateKey() }`, metadata: {
        wordId: wordId, reviewRating: reviewRating OR NULL } }) NGƯỢC LẠI NULL
  RETURN { ...result.recordset[0] OR { }, xpGained: gamification.xpGained OR 0,
    reviewRating: reviewRating OR NULL, gamification: gamification }
END FUNCTION
```

Hàm 279: async getUserStats(userId)

Dòng nguồn: 346 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy user stats.

```
FUNCTION getUserStats(userId)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  learnedResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN "SELECT COUNT(*) AS total FROM
    UserWordProgress WHERE UserID = @UserID AND MasteryLevel >= 3"
  accuracyResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    ExerciseAttempts]
  weakWordsResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    UserWordProgress, Words]
  recentAttemptsResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    ExerciseAttempts, Words]
  gamification = GỌI BẮT ĐỒNG BỘ GamificationService.getProfile(userId)
  stats = { totalLearned: learnedResult.recordset[0].total, accuracy:
    Math.round(accuracyResult.recordset[0].accuracy OR 0), correct:
    accuracyResult.recordset[0].correct OR 0, wrong:
    accuracyResult.recordset[0].wrong OR 0, weakWords: weakWordsResult.recordset,
    recentAttempts: recentAttemptsResult.recordset, streak: gamification.streak,
    totalXP: gamification.totalXP, currentLevel: gamification.currentLevel,
    currentLevelXP: gamification.currentLevelXP, xpForNextLevel:
    gamification.xpForNextLevel, xpToNextLevel: gamification.xpToNextLevel,
    levelProgress: gamification.levelProgress, todayXP: gamification.todayXP }
  stats.masteryTimeline = GỌI BẮT ĐỒNG BỘ this.getMasteryTimeline(userId)
  trendsResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    ExerciseAttempts]
  stats.dailyTrends = trendsResult.recordset.map(HÀM CALLBACK(r))
  stats.achievements = gamification.achievements
  RETURN stats
END FUNCTION
```

Hàm 280: async getMasteryTimeline(userId)

Dòng nguồn: 432 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy dòng thời gian mức độ ghi nhớ.

```
FUNCTION getMasteryTimeline(userId)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  viewExists = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT]
  IF viewExists.recordset[0].viewId THEN
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
      vw_MasteryTimelineProjection]
    IF result.recordset.length > 0 THEN
      RETURN result.recordset[0]
    END IF
  END IF
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
```



```
UserWordProgress]
  row = result.recordset[0] OR { }
  RETURN { totalWords: row.totalWords OR 0, masteredWords: row.masteredWords OR 0,
    completionPercentage: row.completionPercentage OR 0, estimatedDaysToMastery:
    NULL, projectedCompletionDate: NULL }
END FUNCTION
```

Hàm 281: async getMiniTests(page = 1, pageSize = 20)

Dòng nguồn: 476 | Đầu vào: page = 1, pageSize = 20 | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy danh sách bài kiểm tra ngắn.

```
FUNCTION getMiniTests(page = 1, pageSize = 20)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  page = Math.max(1, page)
  pageSize = Math.min(100, Math.max(1, pageSize))
  offset = page - 1 * pageSize
  countResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTests]
  total = countResult.recordset[0].total
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTests,
  Topics]
  RETURN { data: result.recordset, total: total, page: page, pageSize: pageSize,
    totalPages: Math.ceil(total / pageSize) }
END FUNCTION
```

Hàm 282: async getMiniTestDetails(testId)

Dòng nguồn: 509 | Đầu vào: testId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy chi tiết bài kiểm tra ngắn.

```
FUNCTION getMiniTestDetails(testId)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên MiniTestItems,
  Questions, Words]
  RETURN result.recordset
END FUNCTION
```

Hàm 283: async updateProfile(userId, fullName)

Dòng nguồn: 525 | Đầu vào: userId, fullName | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật hồ sơ người dùng.

```
FUNCTION updateProfile(userId, fullName)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  THỰC THI TRUY VẤN "UPDATE Users SET FullName = @FullName, UpdatedAt =
  SYSDATETIMEOFFSET() WHERE UserID = @UserID"
  RETURN { id: userId, fullName: fullName }
END FUNCTION
```

Hàm 284: async getTestHistory(userId, page = 1, pageSize = 20)

Dòng nguồn: 537 | Đầu vào: userId, page = 1, pageSize = 20 | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy test history.

```
FUNCTION getTestHistory(userId, page = 1, pageSize = 20)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  page = Math.max(1, page)
  pageSize = Math.min(100, Math.max(1, pageSize))
  offset = page - 1 * pageSize
  countResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
  ExerciseAttempts, Questions, MiniTestItems, MiniTests]
  total = countResult.recordset[0].total
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
  ExerciseAttempts,
  Questions, MiniTestItems, MiniTests]
```

```
RETURN { data: result.recordset, total: total, page: page, pageSize: pageSize,
        totalPages: Math.ceil(total / pageSize) }
END FUNCTION
```

Hàm 285: async getTestSessionDetails(userId, testId, date)

Dòng nguồn: 583 | Đầu vào: *userId, testId, date* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy test session details.

```
FUNCTION getTestSessionDetails(userId, testId, date)
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    result = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
ExerciseAttempts,
    Questions, MiniTestItems, Words]
    RETURN result.recordset
END FUNCTION
```

Hàm 286: async getActivityHeatmap(userId, year)

Dòng nguồn: 611 | Đầu vào: *userId, year* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy activity heatmap.

```
FUNCTION getActivityHeatmap(userId, year)
    THỰC HIỆN GỢI BẮT ĐỒNG BỘ GamificationService.ensureSchema()
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    result = GỢI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
ExerciseAttempts,
    UserXPEvents, DailyAttempts, DailyXP]
    RETURN result.recordset
END FUNCTION
```

Hàm 287: async getProgressAnalytics(userId)

Dòng nguồn: 641 | Đầu vào: *userId* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy phân tích tiến độ học tập.

```
FUNCTION getProgressAnalytics(userId)
    THỰC HIỆN GỢI BẮT ĐỒNG BỘ GamificationService.ensureSchema()
    pool = GỢI BẮT ĐỒNG BỘ poolPromise
    [activityResult, growthResult, topicResult, retentionResult, gamification] = GỢI
BẮT
    ĐỒNG BỘ Promise.all([THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên DateSeries,
VẤN
    ExerciseAttempts, UserXPEvents, DailyAttempts, DailyRewards], THỰC THI TRUY
    [CÂU LỆNH SELECT trên MonthOffsets, MonthSeries, UserWordProgress], THỰC THI
THI
    TRUY VẤN [CÂU LỆNH SELECT trên Topics, WordTopics, UserWordProgress], THỰC
    TRUY VẤN [CÂU LỆNH SELECT trên ExerciseAttempts, UserWordProgress],
    GamificationService.getMetrics(userId)])
    retentionRow = retentionResult.recordset[0] OR { }
    totalAnswers = Number(retentionRow.totalAnswers OR 0)
    correctAnswers = Number(retentionRow.correctAnswers OR 0)
    learnedWords = Number(retentionRow.learnedWords OR 0)
    forgottenWords = Number(retentionRow.forgottenWords OR 0)
    upToDateWords = Number(retentionRow.upToDateWords OR 0)
    masteredWords = Number(retentionRow.masteredWords OR 0)
    percentage = HÀM CALLBACK(value, total)
    activity = activityResult.recordset.map(HÀM CALLBACK(day))
    RETURN { summary: { activeDays: activity.filter(HÀM CALLBACK(day)).length,
totalXP:
    gamification.totalXP, currentStreak: gamification.streak, learnedWords:
    learnedWords, masteredWords: masteredWords }, activity: activity,
    vocabularyGrowth: growthResult.recordset.map(HÀM CALLBACK(point)),
topicMastery:
    topicResult.recordset.map(HÀM CALLBACK(topic)), retention: {
correctAnswerRate:
    percentage(correctAnswers, totalAnswers), forgottenWordRate:
```

```
percentage(forgottenWords, learnedWords), reviewCompletionRate:
percentage(upToDateWords, learnedWords), totalAnswers: totalAnswers,
correctAnswers: correctAnswers, learnedWords: learnedWords, forgottenWords:
forgottenWords, upToDateWords: upToDateWords } }
END FUNCTION
```

Hàm 288: percentage(value, total)

Dòng nguồn: 756 | Đầu vào: value, total | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Hàm hỗ trợ xử lý percentage trong module backend.

```
FUNCTION percentage(value, total)
    RETURN NẾU total > 0 THÌ Math.round(value / total * 100) NGƯỢC LẠI 0
END FUNCTION
```

Hàm 289: async getDailyProgress(userId)

Dòng nguồn: 804 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy tiến độ mục tiêu hằng ngày.

```
FUNCTION getDailyProgress(userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
ExerciseAttempts]
    RETURN { todayCount: result.recordset[0].count OR 0 }
END FUNCTION
```

Hàm 290: async getSmartReviewQueue(userId, limit = 20)

Dòng nguồn: 817 | Đầu vào: userId, limit = 20 | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xây dựng hàng đợi ôn thông minh dựa trên độ quá hạn, số lần sai liên tiếp và mức độ thành thạo.

```
FUNCTION getSmartReviewQueue(userId, limit = 20)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    limit = Math.min(50, Math.max(1, limit))
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Words,
PartOfSpeeches, UserWordProgress]
    RETURN result.recordset
END FUNCTION
```

Hàm 291: async submitMiniTestBatch(userId, testId, answers)

Dòng nguồn: 857 | Đầu vào: userId, testId, answers | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để gửi mini test batch.

```
FUNCTION submitMiniTestBatch(userId, testId, answers)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    transaction = TẠO sql.Transaction(pool)
    committed = false
    TRY
        BẮT ĐẦU TRANSACTION
        testQuestionsResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT
trên MiniTestItems, Questions]
        testQuestions = TẠO Map(testQuestionsResult.recordset.map(HÀM
CALLBACK(question)))
        IF testQuestions.size = 0 THEN
            THROW ERROR TẠO Error('Mini test ${ testId } does not contain
questions')
        END IF
        IF answers.length ≠ testQuestions.size THEN
            THROW ERROR TẠO Error('Mini test ${ testId } requires exactly ${
testQuestions.size } answers')
        END IF
        correctCount = 0
```

```

results = []
submittedQuestionIds = TẠO Set()
FOR EACH answer IN answers
  { questionId } = answer
  submittedAnswer = String(answer.submittedAnswer OR "").slice(0,
    1000)
  numericQuestionId = Number(questionId)
  question = testQuestions.get(numericQuestionId)
  IF NOT question THEN
    THROW ERROR TẠO Error(`Question ${ questionId } || "unknown" }
      does not belong to mini test ${ testId }`)
  END IF
  IF submittedQuestionIds.has(numericQuestionId) THEN
    THROW ERROR TẠO Error(`Question ${ numericQuestionId } was
      submitted more than once`)
  END IF
  THỰC HIỆN submittedQuestionIds.add(numericQuestionId)
  wordId = Number(question.WordID)
  isCorrect = submittedAnswer.trim().toLocaleLowerCase() =
    String(question.CorrectAnswer OR "").trim().toLocaleLowerCase()
  scoreAwarded = NẾU isCorrect THÌ 100 NGƯỢC LẠI 0
  req = TẠO sql.Request(transaction)
  THỰC HIỆN req.input("UserID", sql.BigInt, userId)
  THỰC HIỆN req.input("QuestionID", sql.BigInt, questionId OR NULL)
  THỰC HIỆN req.input("WordID", sql.BigInt, wordId OR NULL)
  THỰC HIỆN req.input("SubmittedAnswer", sql.NVarChar(1000),
    submittedAnswer)
  THỰC HIỆN req.input("IsCorrect", sql.Bit, isCorrect)
  THỰC HIỆN req.input("ScoreAwarded", sql.Decimal(5, 2), scoreAwarded)
  THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên ExerciseAttempts]
  IF wordId THEN
    wordReq = TẠO sql.Request(transaction)
    THỰC HIỆN wordReq.input("UserID", sql.BigInt, userId)
    THỰC HIỆN wordReq.input("WordID", sql.BigInt, wordId)
    THỰC HIỆN wordReq.input("IsCorrect", sql.Bit, isCorrect)
    THỰC THI TRUY VẤN [CÂU LỆNH MERGE trên SET,
      UserWordProgress]
  END IF
  IF isCorrect THEN
    THỰC HIỆN correctCount TĂNG 1
  END IF
  THỰC HIỆN results.push({ questionId: questionId, wordId: wordId,
    isCorrect: isCorrect })
END FOR
score = Math.round(correctCount / answers.length * 100)
attemptResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT]
COMMIT TRANSACTION
committed = true
miniTestAttemptId = attemptResult.recordset[0].id
gamification = GỌI BẮT ĐỒNG BỘ GamificationService.awardXP(userId, {
  eventType: "MiniTestComplete", sourceKey: `mini-test-attempt: ${
    miniTestAttemptId}`, metadata: { testId: Number(testId),
    miniTestAttemptId: miniTestAttemptId, score: score } })
RETURN { total: answers.length, correct: correctCount, score: score,
  xpEarned: gamification.xpGained, gamification: gamification, results:
  results }
CATCH err
  IF NOT committed THEN
    ROLLBACK TRANSACTION
  END IF
  THROW ERROR err
END TRY
END FUNCTION

```

Hàm 292: async getDailyGoal(userId)

Dòng nguồn: 1024 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy mục tiêu hằng ngày.

```

FUNCTION getDailyGoal(userId)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise

```

```

    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Users]
    RETURN result.recordset[0] OR { dailyGoal: 20, srsReviewLimit: 15 }
END FUNCTION

```

Hàm 293: async updateDailyGoal(userId, dailyGoal)

Dòng nguồn: 1031 | Đầu vào: *userId, dailyGoal* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật mục tiêu hằng ngày.

```

FUNCTION updateDailyGoal(userId, dailyGoal)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    goal = Math.min(100, Math.max(5, Number(dailyGoal) OR 20))
    THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Users]
    RETURN { dailyGoal: goal }
END FUNCTION

```

Hàm 294: async updateSRSReviewLimit(userId, limit)

Dòng nguồn: 1041 | Đầu vào: *userId, limit* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật srs review limit.

```

FUNCTION updateSRSReviewLimit(userId, limit)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    newLimit = Math.min(50, Math.max(5, Number(limit) OR 15))
    THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Users]
    RETURN { srsReviewLimit: newLimit }
END FUNCTION

```

Hàm 295: async getUserNotifications(userId, limit = 20)

Dòng nguồn: 1052 | Đầu vào: *userId, limit = 20* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy user notifications.

```

FUNCTION getUserNotifications(userId, limit = 20)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    limit = Math.min(50, Math.max(1, limit))
    tableCheck = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT]
    IF NOT tableCheck.recordset[0].tableId THEN
        RETURN { notifications: [], unreadCount: 0 }
    END IF
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên Notifications]
    countResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
Notifications]
    RETURN { notifications: result.recordset, unreadCount:
        countResult.recordset[0].unread OR 0, total: countResult.recordset[0].total
    OR 0
    }
END FUNCTION

```

Hàm 296: async markNotificationRead(userId, notificationId)

Dòng nguồn: 1097 | Đầu vào: *userId, notificationId* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để đánh dấu notification read.

```

FUNCTION markNotificationRead(userId, notificationId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    tableCheck = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT]
    IF NOT tableCheck.recordset[0].tableId THEN
        RETURN { success: false }
    END IF
    THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Notifications]
    RETURN { success: true }
END FUNCTION

```

Hàm 297: async markAllNotificationsRead(userId)

Dòng nguồn: 1114 | Đầu vào: userId | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để đánh dấu all notifications read.

```
FUNCTION markAllNotificationsRead(userId)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  tableCheck = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT]
  IF NOT tableCheck.recordset[0].tableId THEN
    RETURN { success: false, count: 0 }
  END IF
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên Notifications]
  RETURN { success: true, count: result.rowsAffected[0] OR 0 }
END FUNCTION
```

Hàm 298: async getNotebook(userId, page = 1, pageSize = 20)

Dòng nguồn: 1131 | Đầu vào: userId, page = 1, pageSize = 20 | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy sổ tay từ vựng.

```
FUNCTION getNotebook(userId, page = 1, pageSize = 20)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  page = Math.max(1, page)
  pageSize = Math.min(50, Math.max(1, pageSize))
  offset = page - 1 * pageSize
  countResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    UserVocabularyNotebook]
  total = countResult.recordset[0].total
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    UserVocabularyNotebook, Words, PartOfSpeeches, UserWordProgress]
  RETURN { data: result.recordset, total: total, page: page, pageSize: pageSize,
    totalPages: Math.ceil(total / pageSize) }
END FUNCTION
```

Hàm 299: async addNotebookEntry(userId, wordId, personalNote)

Dòng nguồn: 1179 | Đầu vào: userId, wordId, personalNote | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để thêm một mục trong sổ tay từ vựng.

```
FUNCTION addNotebookEntry(userId, wordId, personalNote)
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  existing = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
    UserVocabularyNotebook]
  IF existing.recordset.length > 0 THEN
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên
      UserVocabularyNotebook]
    RETURN result.recordset[0]
  END IF
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH INSERT trên
    UserVocabularyNotebook]
  RETURN result.recordset[0]
END FUNCTION
```

Hàm 300: async updateNotebookEntry(notebookId, userId, { personalNote, isFavorite })

Dòng nguồn: 1217 | Đầu vào: notebookId, userId, { personalNote, isFavorite } | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để cập nhật một mục trong sổ tay từ vựng.

```
FUNCTION updateNotebookEntry(notebookId, userId, { personalNote, isFavorite })
  pool = GỌI BẮT ĐỒNG BỘ poolPromise
  result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH UPDATE trên
    UserVocabularyNotebook]
  RETURN result.recordset[0] OR NULL
END FUNCTION
```

Hàm 301: async deleteNotebookEntry(notebookId, userId)

Dòng nguồn: 1248 | Đầu vào: *notebookId, userId* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để xóa một mục trong sổ tay từ vựng.

```
FUNCTION deleteNotebookEntry(notebookId, userId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH DELETE trên
        UserVocabularyNotebook]
    RETURN result.recordset[0] OR NULL
END FUNCTION
```

Hàm 302: async getSessionSummary(userId)

Dòng nguồn: 1262 | Đầu vào: *userId* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy tóm tắt phiên ôn tập.

```
FUNCTION getSessionSummary(userId)
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ GamificationService.ensureSchema()
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        ExerciseAttempts,
        Users, UserXPEvents, SessionStats, XPInfo]
    row = result.recordset[0] OR { totalAttempts: 0, correctCount: 0, wrongCount: 0,
        accuracy: 0, xpEarned: 0, TotalXP: 0, CurrentLevel: 1 }
    weakResult = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        ExerciseAttempts, Words]
    RETURN { totalAttempts: row.totalAttempts, correctCount: row.correctCount,
        wrongCount: row.wrongCount, accuracy: Number(row.accuracy), xpEarned:
        Number(row.xpEarned), totalXP: Number(row.TotalXP), currentLevel:
        Number(row.CurrentLevel), weakWords: weakResult.recordset }
END FUNCTION
```

Hàm 303: async getMistakeReviewQueue(userId, limit = 10)

Dòng nguồn: 1341 | Đầu vào: *userId, limit = 10* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để lấy hàng đợi ôn lại câu sai.

```
FUNCTION getMistakeReviewQueue(userId, limit = 10)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    limit = Math.min(30, Math.max(1, limit))
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        ExerciseAttempts,
        Words, PartOfSpeeches, UserWordProgress]
    RETURN result.recordset
END FUNCTION
```

Hàm 304: async checkNotebookEntry(userId, wordId)

Dòng nguồn: 1375 | Đầu vào: *userId, wordId* | Đầu ra: Dữ liệu nghiệp vụ, kết quả cập nhật CSDL hoặc lỗi nghiệp vụ.

Mục đích: Xử lý nghiệp vụ và truy vấn/cập nhật CSDL để kiểm tra một mục trong sổ tay từ vựng.

```
FUNCTION checkNotebookEntry(userId, wordId)
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    result = GỌI BẮT ĐỒNG BỘ THỰC THI TRUY VẤN [CÂU LỆNH SELECT trên
        UserVocabularyNotebook]
    RETURN result.recordset[0] OR NULL
END FUNCTION
```

5.5. Application bootstrap**index.js**

File này có 1 hàm được mô tả. Thứ tự bên dưới bám theo vị trí xuất hiện trong mã nguồn.

Hàm 305: `async gracefulShutdown(signal)`

Dòng nguồn: 90 | Đầu vào: `signal` | Đầu ra: Kết quả xử lý tương ứng với từng nhánh RETURN của hàm.

Mục đích: Hàm hỗ trợ xử lý graceful shutdown trong module backend.

```
FUNCTION gracefulShutdown(signal)
  THỰC HIỆN console.log(`Received ${ signal }.Shutting down gracefully...`)
  TRY
    pool = GỌI BẮT ĐỒNG BỘ poolPromise
    THỰC HIỆN GỌI BẮT ĐỒNG BỘ pool.close()
    THỰC HIỆN console.log("Database pool closed.")
    THỰC HIỆN process.exit(0)
  CATCH err
    THỰC HIỆN console.error("Error during shutdown: ", err)
    THỰC HIỆN process.exit(1)
  END TRY
END FUNCTION
```


6. THUẬT TOÁN NGHIỆP VỤ TRỌNG YẾU

Ngoài danh sách hàm, phần này đi sâu vào những nghiệp vụ cốt lõi nhất của backend để làm rõ cách hệ thống thật sự hoạt động. Đây là phần quan trọng nếu báo cáo cần thể hiện ‘mô tả chi tiết cách hiện thực’ chứ không chỉ liệt kê tên hàm.

6.1. Đăng nhập người dùng (AuthService.login)

Mục tiêu: Xác thực tài khoản, tài quyền, phát hành JWT và kích hoạt phần thưởng đăng nhập hằng ngày.

Đầu vào / đầu ra

Thuộc tính	Mô tả
Đầu vào	email; password
Đầu ra	token JWT; thông tin người dùng; danh sách quyền; gamification nếu là learner

Luồng xử lý

Bước	Mô tả
1	Truy vấn người dùng theo email và trạng thái IsActive = 1.
2	So khớp mật khẩu nhập vào với PasswordHash bằng bcrypt.compare.
3	Nếu sai thông tin đăng nhập thì ném lỗi nghiệp vụ và kết thúc.
4	Truy vấn RolePermissions + Permissions để lấy toàn bộ permission code của người dùng.
5	Tạo payload JWT gồm id, fullName, role, permissions; ký bằng JWT_SECRET trong 1 ngày.
6	Nếu role là Learner thì thử cộng thưởng daily login thông qua GamificationService.
7	Trả về token, thông tin người dùng và phần thưởng (nếu có).

Giả mã

Hàm login(email, password):

```

user <- query Users + Roles theo email và IsActive = 1
nếu user rỗng: ném lỗi sai tài khoản/mật khẩu
nếu bcrypt.compare(password, user.passwordHash) = false: ném lỗi
permissions <- query RolePermissions/Permissions theo user.id
payload <- {id, fullName, role, permissions}
token <- jwt.sign(payload, JWT_SECRET, expiresIn = 1 day)
nếu user.role = Learner:
    gamification <- awardDailyLogin(user.id)
trả về {token, user, permissions, gamification}
  
```

6.2. Lấy flashcard đến hạn ôn theo SRS (UserService.getDueFlashcards)

Mục tiêu: Xây dựng bộ câu hỏi/flashcard ưu tiên theo lịch ôn, chủ đề và trạng thái học tập của learner.

Đầu vào / đầu ra

Thuộc tính	Mô tả
Đầu vào	userId; topicId (optional); mode = new learned null
Đầu ra	Danh sách flashcard đã trộn và kèm metadata SRS

Luồng xử lý

Bước	Mô tả
1	Đọc SRSReviewLimit của người dùng; nếu chưa có thì dùng mặc định 15.
2	Nếu client chưa truyền topicId, hệ thống lấy chủ đề người học đang enrol đầu tiên còn active/published.
3	Ghép dữ liệu Words với PartOfSpeeches, Questions, ExampleSentences và UserWordProgress.
4	Lọc theo ContentStatus = Published và theo topic hoặc trạng thái đã từng học.
5	Nếu mode = new thì chỉ lấy các từ chưa có repetition count; nếu learned thì chỉ lấy từ đã có progress; nếu null thì lấy từ chưa đến hạn hoặc đã đến hạn tùy luật SRS.
6	Sắp xếp ưu tiên: mục quá hạn trước, rồi theo NextReviewDate, MasteryLevel và random để giảm lặp nhàm chán.
7	Trả về tối đa @Limit bản ghi dùng cho phiên học hiện tại.

Giả mã

```
Hàm getDueFlashcards(userId, topicId, mode):
  limit <- Users.SRSReviewLimit hoặc 15
  nếu topicId rỗng:
    topicId <- chủ đề learner đang theo học đầu tiên còn hiệu lực
  query Words + Questions + Examples + UserWordProgress
  lọc Published và lọc theo topic phù hợp
  nếu mode = 'new': chỉ lấy repetitionCount = 0
  nếu mode = 'learned': chỉ lấy bản ghi đã có UserWordProgress
  nếu mode khác: chỉ lấy từ chưa có lịch hoặc NextReviewDate <= now
  order by overdue first, next review, mastery, random
  trả về TOP(limit)
```

6.3. Gửi kết quả trả lời và cập nhật lịch ôn (UserService.submitAnswer)

Mục tiêu: Lưu lịch sử làm bài, điều chỉnh EaseFactor/NextReviewDate theo rating Again-Hard-Good-Easy và cộng XP khi cần.

Đầu vào / đầu ra

Thuộc tính	Mô tả
Đầu vào	userId; questionId; submittedAnswer; reviewRating; activityType
Đầu ra	Kết quả attempt; feedback SRS; gamification/xpGained

Luồng xử lý

Bước	Mô tả
1	Gọi stored procedure usp_SubmitQuestionAttempt để lưu ExerciseAttempts và lấy WordID chuẩn của câu hỏi.
2	Nếu WordID không ánh xạ được thì trả lỗi vì câu hỏi không gắn với từ vựng hợp lệ.
3	Nếu reviewRating thuộc Again/Hard/Good/Easy thì cập nhật EaseFactor và NextReviewDate trong UserWordProgress.
4	Again giảm EaseFactor mạnh và dời lịch 10 phút; Hard dời 1 ngày; Good/Easy dời 1-30 ngày theo MasteryLevel.
5	Nếu activityType = LearnWord thì gọi GamificationService.awardXP để cộng điểm.
6	Trộn dữ liệu attempt + feedback + gamification thành object trả về cho frontend.

Giải mã

Hàm submitAnswer(payload):

```

result <- execute usp_SubmitQuestionAttempt(userId, questionId,
submittedAnswer)
wordId <- result.WordID
nếu wordId rỗng: ném lỗi
nếu reviewRating hợp lệ:
    update UserWordProgress
        EaseFactor <- tăng/giảm theo rating
        NextReviewDate <- 10 phút / 1 ngày / 1-30 ngày theo rating +
mastery
    nếu activityType = LearnWord:
        gamification <- awardXP(...)
    trả về merge(result, reviewFeedback, gamification)

```

6.4. Cập nhật trực tiếp tiến độ từ vựng (UserService.submitWordReview)

Mục tiêu: Tạo mới hoặc cập nhật UserWordProgress cho learner sau mỗi lần ôn từ độc lập ngoài bài tập.

Đầu vào / đầu ra

Thuộc tính	Mô tả
Đầu vào	userId; wordId; isCorrect; reviewRating; activityType
Đầu ra	masteryLevel; memoryStatus; nextReviewDate; xpGained

Luồng xử lý

Bước	Mô tả
1	Dùng MERGE trên UserWordProgress để xử lý cả trường hợp đã có bản ghi và chưa có bản ghi.
2	Khi bản ghi tồn tại: tăng/giảm MasteryLevel, cộng RepetitionCount, cập nhật chuỗi đúng/sai liên tiếp.
3	Tính NextReviewDate theo reviewRating hoặc theo việc đúng/sai với các mốc 10 phút, 1 ngày, 3-30 ngày.
4	Điều chỉnh EaseFactor trong khoảng tối thiểu/tối đa để phản ánh độ khó của từ với người học.
5	Gán MemoryStatus thành Learning / Reviewing / Mastered / Lapsed tùy mastery và kết quả hiện tại.
6	Nếu chưa có bản ghi thì insert bản ghi mặc định với EaseFactor = 2.50 và lịch ôn ban đầu.
7	Nếu activityType = LearnWord thì cộng XP và trả toàn bộ feedback cho frontend.

Giải mã

Hàm submitWordReview(userId, wordId, isCorrect, reviewRating):

MERGE UserWordProgress theo (UserID, WordID)

nếu MATCHED:

cập nhật mastery, repetition, consecutiveCorrect/Wrong

tính NextReviewDate theo rating + mastery

tính EaseFactor theo Again/Hard/Good/Easy

cập nhật MemoryStatus

nếu NOT MATCHED:

insert bản ghi khởi tạo với mastery ban đầu

nếu activityType = LearnWord: cộng XP

trả về feedback SRS mới nhất

6.5. Tạo từ vựng mới ở khu vực quản trị (AdminService.createWord)

Mục tiêu: Ghi một từ vựng mới cùng liên kết chủ đề và câu ví dụ, bảo đảm tính toàn vẹn bằng transaction.

Đầu vào / đầu ra

Thuộc tính	Mô tả
Đầu vào	wordData: term, meaning, phonetic, partOfSpeechId, topicIds, examples, status; adminId
Đầu ra	id; term; meaning

Luồng xử lý

Bước	Mô tả
1	Kiểm tra status nội dung hợp lệ và chuẩn hóa danh sách topicIds, loại bỏ giá trị trùng/rỗng.
2	Lọc danh sách examples chỉ giữ các ví dụ có sentence không rỗng.
3	Mở transaction SQL Server để bảo đảm các insert xảy ra đồng bộ.
4	Insert bản ghi mới vào bảng Words và lấy WordID vừa sinh.
5	Lặp qua từng topicId để thêm bản ghi WordTopics.
6	Lặp qua từng ví dụ hợp lệ để thêm ExampleSentences.
7	Commit transaction, ghi AdminAuditLog và trả object kết quả.
8	Nếu xảy ra lỗi ở bất kỳ bước nào thì rollback transaction và ném lỗi ra ngoài controller.

Giải mã

```
Hàm createWord(wordData, adminId):
  validate status
  topicIds <- unique(topicIds hợp lệ)
  examples <- lọc câu ví dụ có nội dung
  begin transaction
  thử:
    wordId <- insert Words
    cho mỗi topicId: insert WordTopics(wordId, topicId)
    cho mỗi example: insert ExampleSentences(wordId, ...)
  commit
  logAdminAction('CREATE_WORD')
  trả về {id: wordId, term, meaning}
nếu lỗi:
  rollback
  throw error
```

6.6. Tiễn kiểm import CSV từ vựng (AdminService.previewWordImport)

Mục tiêu: Phân tích dữ liệu import và báo lỗi theo từng dòng trước khi ghi thật vào CSDL.

Đầu vào / đầu ra

Thuộc tính	Mô tả
Đầu vào	input CSV/text/json rows
Đầu ra	total; valid; invalid; rows[] có cờ hợp lệ/lỗi

Luồng xử lý

Bước	Mô tả
------	-------

1	Parse input thành danh sách rows, tự nhận biết delimiter và header aliases.
2	Tải reference data gồm PartOfSpeeches và Topics từ cơ sở dữ liệu.
3	Tạo map tra cứu theo id, name và code đã normalize để hỗ trợ nhập CSV linh hoạt.
4	Duyệt từng dòng, đọc các cột term/meaning/phonetic/partOfSpeech/topic/example qua danh sách alias.
5	Kiểm tra dữ liệu bắt buộc, tra cứu partOfSpeech, ánh xạ topic theo id hoặc tên/mã chủ đề.
6	Thu thập errors theo từng dòng; tăng bộ đếm valid/invalid tương ứng.
7	Trả previewRows cho frontend để hiển thị trước khi gọi bulk-import thật sự.

Giả mã

Hàm `previewWordImport(input)`:

```
rows <- parseWordImport(input)
partsOfSpeech, topics <- query reference data
build map theo id + tên + code
cho từng row:
  term, meaning, phonetic <- lấy theo aliases
  validate partOfSpeech và topics
  nếu lỗi: mark invalid + lưu errors
  ngược lại: mark valid
trả về {total, valid, invalid, rows}
```

6.7. Duyệt nội dung chờ duyệt (ReviewService.approve / reject / archive)

Mục tiêu: Quản lý vòng đời Topic/Word/Question/MiniTest ở trạng thái PendingReview bằng transaction và log lịch sử.

Đầu vào / đầu ra

Thuộc tính	Mô tả
Đầu vào	entityType; entityId; adminId; reason (đối với reject)
Đầu ra	true/false tùy có cập nhật được hay không

Luồng xử lý

Bước	Mô tả
1	Gọi <code>_resolveEntity</code> để ánh xạ entityType thành tên bảng, khóa chính và các cột reviewed/published tương ứng.
2	Bắt đầu transaction để đồng bộ cập nhật trạng thái và log lịch sử duyệt.
3	Approve: đổi ContentStatus thành Published; riêng MiniTest còn bật IsPublished = 1.
4	Reject: đổi ContentStatus thành Rejected và lưu reason vào ContentReviewLogs.
5	Archive: đổi ContentStatus thành Archived, lưu comment mặc định vào log.

6	Nếu không có dòng nào bị cập nhật thì rollback và trả false.
7	Nếu thành công thì insert một dòng vào ContentReviewLogs rồi commit transaction.

Giải mã

```
Hàm approve(entityType, entityId, adminId):
  entityMeta <- resolveEntity(entityType)
  begin transaction
  update bảng đích từ PendingReview -> Published
  nếu entityType = MiniTest: set IsPublished = 1
  insert ContentReviewLogs
  commit hoặc rollback nếu rowsAffected = 0
```

```
Hàm reject(..., reason):
  update PendingReview -> Rejected
  insert log kèm reason
```

```
Hàm archive(...):
  update bất kỳ trạng thái -> Archived
  insert log archive
```

7. THIẾT KẾ DỮ LIỆU BACKEND

Backend hiện tại làm việc chủ yếu trên SQL Server với các bảng lõi phục vụ người dùng, nội dung, học tập, SRS, mini test, duyệt nội dung và gamification. Bảng dưới đây tổng hợp những thực thể quan trọng nhất liên quan trực tiếp tới mã nguồn backend.

Các bảng lõi

Thuộc tính	Mô tả
Users	Lưu tài khoản, vai trò, trạng thái hoạt động và cấu hình học tập của người dùng.
Topics / TopicCategories	Lưu nhóm chủ đề và các chủ đề học từ vựng TOEIC.
Words	Lưu từ vựng, nghĩa, phiên âm, media và trạng thái nội dung.
ExampleSentences	Lưu ví dụ câu cho từng từ vựng.
WordTopics	Bảng nối nhiều-nhiều giữa Words và Topics.
Questions	Lưu câu hỏi trắc nghiệm/fill blank/dictation gắn với từng từ.
MiniTests / MiniTestItems	Lưu bài kiểm tra ngắn và danh sách câu hỏi của từng bài.
UserWordProgress	Lưu tiến độ nhớ từ, mastery, ease factor, next review date và memory status cho SRS.
ExerciseAttempts	Lưu lịch sử làm bài/câu hỏi để tính accuracy và phân tích tiến độ.
UserTopicEnrollments	Lưu các chủ đề mà learner đã đăng ký hoặc đang theo học.
ContentReviewLogs	Lưu lịch sử duyệt nội dung bởi admin/reviewer.
ContentReports	Lưu các báo lỗi, phản hồi nội dung do learner gửi.
UserXPEvents / UserAchievements	Lưu sự kiện cộng điểm, huy hiệu và trạng thái gamification.
Notifications	Lưu thông báo gửi tới learner hoặc do admin tạo.

7.1. Ràng buộc dữ liệu liên quan đến SRS

Dữ liệu hỗ trợ SRS

Thuộc tính	Mô tả
Khóa duy nhất	UserWordProgress có cặp duy nhất (UserID, WordID) để mỗi người học chỉ có một bản ghi tiến độ trên mỗi từ.
Chỉ mục chính	Có index trên (UserID, NextReviewDate) giúp lấy danh sách ôn đến hạn nhanh hơn.
Thuộc tính quan trọng	MasteryLevel, EaseFactor, RepetitionCount, ConsecutiveCorrect, ConsecutiveWrong, NextReviewDate, MemoryStatus.
Ý nghĩa nghiệp vụ	Cho phép backend lập lịch ôn thông minh, ưu tiên từ yếu và giữ lịch sử tiến bộ lâu dài.

8. LIÊN KẾT FRONTEND - BACKEND

Frontend Next.js gọi backend chủ yếu thông qua frontend/src/lib/api-client.ts và các service tương ứng. Bảng dưới đây nêu một số liên kết tiêu biểu để chứng minh backend đã được nối thực tế vào giao diện người dùng.

Liên kết service frontend

Thuộc tính	Mô tả
auth.service.ts -> /api/auth	Đăng ký, đăng nhập, lưu token và cookie phục vụ middleware server-side.
user.service.ts -> /api/user	Phiên học flashcard, mini test, dashboard, notebook, notifications và mục tiêu hằng ngày.
admin.service.ts -> /api/admin	Quản trị chủ đề, từ vựng, câu hỏi, bài kiểm tra, người dùng, báo cáo, thống kê.
creator.service.ts -> /api/creator	Luồng biên tập nội dung, media, gửi duyệt nội dung và analytics cá nhân.
ai.service.ts -> /api/ai	Gọi backend AI để gợi ý nội dung từ vựng trước khi lưu vào CSDL.

9. KẾT LUẬN VÀ KHUYẾN NGHỊ MỞ RỘNG

Phần backend của VocaBoost đã được tổ chức khá rõ theo kiến trúc route-controller-service và đã hiện thực được nhiều yêu cầu quan trọng: xác thực JWT, phân quyền theo Role/Permission, quản trị nội dung, quy trình duyệt, mini test, báo lỗi, thông báo, gamification và đặc biệt là cơ chế SRS.

- Bổ sung test tự động cho các luồng transaction quan trọng như createWord, review approve/reject và submitAnswer.
- Tách bớt các service lớn (đặc biệt admin.service.js và user.service.js) thành nhiều service nhỏ để tăng tính maintainable.
- Chuẩn hóa thêm logging và observability cho các API AI/upstream để dễ truy vết lỗi môi trường.
- Bổ sung tài liệu validate schema và contract API response nếu nhóm muốn hướng tới bàn giao sản phẩm hoàn chỉnh hơn.